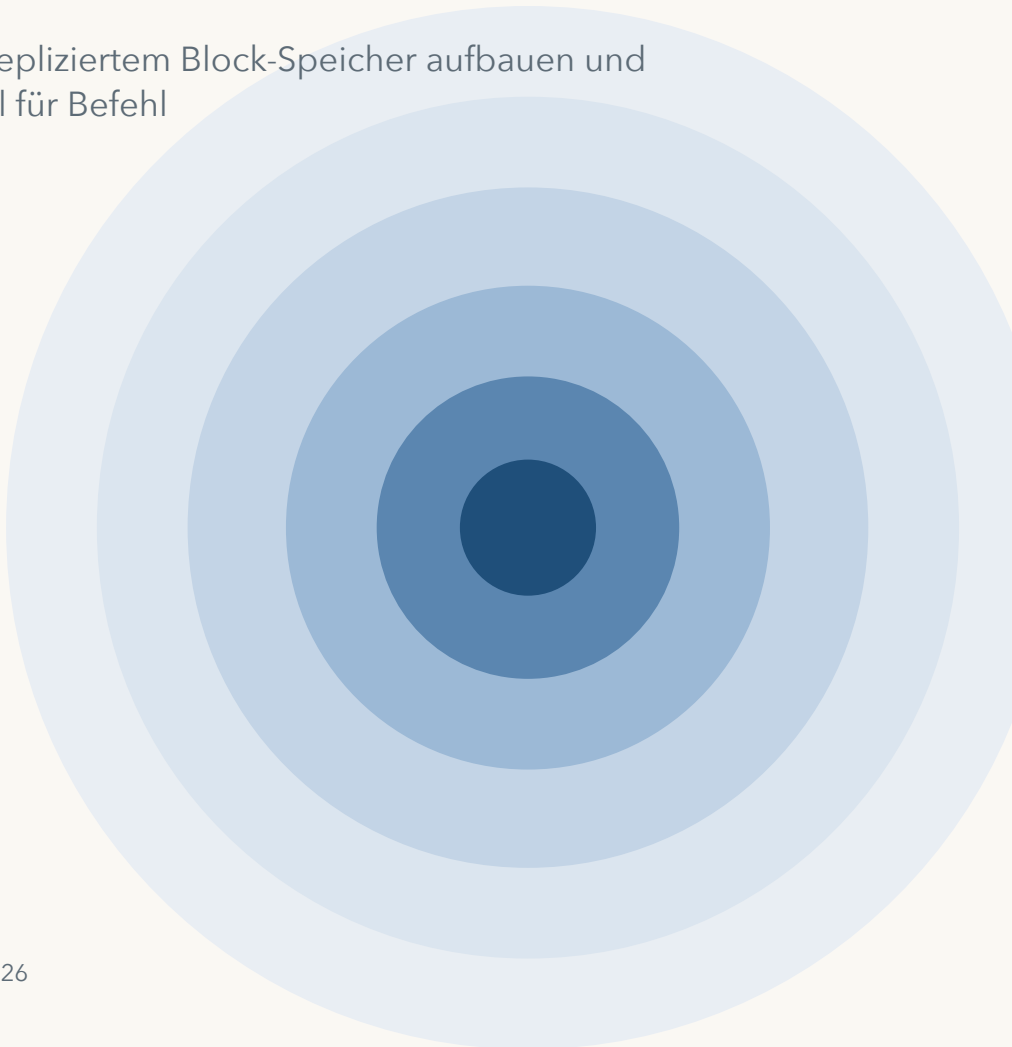
HANDS-ON-HANDBUCH

k0s und LINSTOR

Kubernetes mit repliziertem Block-Speicher aufbauen und betreiben, Befehl für Befehl



Thomas Zachmann

Stand: 12. November 2026

Inhalt

Vorwort **6**
Für wen 6
Was geprüft wurde 6
Versionen 6
Die Architektur 7
Begleit-Repository 7
Wo du einen Befehl ausführst 7
1 Was du aufbaust und warum so **8**
1.1 Das Muster 8
1.2 Was läuft wo 8
1.3 Ein Schreibvorgang von der Datenbank bis zur Platte 9
1.4 Versionen 9
1.5 Der Bauplan 10
2 Vier VMs vorbereiten **12**
2.1 Was du aufbaust 12
2.2 Die interne Bridge in Proxmox 12
2.3 SSH vom Mac 13
2.4 Das Storage-Netz einrichten 14
2.5 Platten für LINSTOR vorbereiten 15
3 k0s mit k0sctl installieren **18**
3.1 Was k0s anders macht 18
3.2 k0sctl installieren 18
3.3 Die Cluster-Beschreibung 19
3.4 Den Cluster aufbauen 20
3.5 kubectl verbinden 20
3.6 Auf den Nodes nachsehen 21
4 k0s verstehen und verwalten **22**
4.1 Was auf welchem Node läuft 22
4.2 Wo k0s seine Dateien ablegt 23
4.3 Werkzeuge auf den Nodes 23
4.4 konnectivity prüfen 24
4.5 Konfiguration ändern 24

5 LINSTOR und DRBD auf den Hosts 28
5.1 Warum LINSTOR außerhalb von Kubernetes 28
5.2 Woher die Pakete kommen 29
5.3 Die Pakete installieren 29
5.4 LVM darf DRBD-Geräte nicht anfassen 30
5.5 Eine Falle direkt nach der Installation 31
5.6 Den Client einrichten 31
5.7 Nodes registrieren 31
5.8 Die Witness über das Management-Netz anbinden 32
5.9 Zonen festlegen 33
6 Der LINSTOR-Controller ohne Single Point of Failure 34
6.1 Das Prinzip 34
6.2 Die Ressource linstor_db 35
6.3 Die Datenbank umziehen 36
6.4 drbd-reactor einrichten 37
6.5 Den Failover ausprobieren 39
7 Storage-Pools, Writecache und Zonen 42
7.1 Storage-Pools anlegen 42
7.2 Was ein Writecache ist 43
7.3 Die Vorlage rg-k8s 43
7.4 Ausprobieren 44
8 Piraeus als CSI-Treiber 46
8.1 Was der Operator hier tut und was nicht 46
8.2 Den Operator installieren 46
8.3 Der Cluster als Helm-Werte 47
8.4 Die StorageClass 52
8.5 Eine Datenbank auf repliziertem Speicher 53
8.6 Der Umzug 54
9 LINSTOR und DRBD lesen 56
9.1 Zwei Sichten auf dasselbe Volume 56
9.2 Die Sicht von LINSTOR 56
9.3 Die Sicht von DRBD 57
9.4 Mehr Details 58
9.5 Die Sicht der Witness 60

9.6 Fehlerberichte des Controllers ... 60
10 Betrieb: Wenn ein Node ausfällt ... 62
10.1 Den Ausfall messbar machen ... 62
10.2 Geplante Wartung ... 62
10.3 Ein Absturz ... 64
10.4 Ein Node, der nicht zurückkommt ... 65
10.5 Der Node kommt zurück ... 67
11 Betrieb: Netzausfälle, Quorum und Split-Brain ... 70
11.1 Der Versuchsaufbau ... 70
11.2 Versuch A: Ein Secondary verliert das Netz ... 71
11.3 Versuch B: Der Primary verliert das Storage-Netz ... 72
11.4 Versuch C: Der Primary verliert alles ... 73
11.5 Versuch D: Die andere Seite übernimmt ... 73
11.6 Versuch E: Ein echtes Split-Brain ... 74
11.7 Das Split-Brain auflösen ... 76
11.8 Zusammenfassung der Versuche ... 77
12 Betrieb: Wenn der Platz ausgeht ... 80
12.1 Ein volles Volume ... 80
12.2 Vergrößern im laufenden Betrieb ... 81
12.3 Schicht für Schicht suchen ... 82
12.4 Der Ausweg von Hand ... 83
12.5 Ein voller Pool ... 85
13 Betrieb: Snapshots und Backup ... 88
13.1 Snapshots: nicht mit klassischem LVM ... 88
13.2 Backup auf Dateiebene ... 89
13.3 Sichern ... 92
13.4 Wiederherstellen ... 92
14 Betrieb: Eine Datenplatte tauschen ... 94
14.1 Den Defekt nachstellen ... 94
14.2 Was der Ausfall auslöst ... 94
14.3 Die Reste abräumen ... 96
14.4 Die Ersatzplatte einrichten ... 97
14.5 Die Kopien neu aufbauen ... 97
15 Betrieb: Updates von LINSTOR, CSI und k0s ... 100

15.1 Die Abhängigkeit ... 100
15.2 Die falsche Reihenfolge ... 100
15.3 Vorbereitung ... 101
15.4 LINSTOR aktualisieren ... 101
15.5 Was mit der Controller-Datenbank passiert ... 103
15.6 CSI aktualisieren ... 104
15.7 Was 1.35 ändert und was nicht ... 105
15.8 k0s aktualisieren ... 106
16 Dasselbe mit Ansible ... 108
16.1 Aufbau ... 108
16.2 Die Rolle linstor ... 109
16.3 Die Rolle linstor-controller-ha ... 111
16.4 Ausprobieren ... 112
16.5 Ein Update mit Ansible ... 113
17 Überwachung: Was du messen und alarmieren solltest ... 114
17.1 Zwei Quellen ... 114
17.2 LINSTOR-Metriken ... 114
17.3 DRBD-Metriken mit drbd-reactor ... 115
17.4 Der Stack im Überblick ... 117
17.5 Prometheus, Alertmanager und Grafana ... 117
17.6 Logs mit Loki und Alloy ... 122
17.7 Alarmregeln ... 125
17.8 Die Regeln testen ... 127
17.9 Das Dashboard ... 128
17.10 Fertige Dashboards ... 130
17.11 Ein Alarm von Anfang bis Ende ... 131
17.12 Was der Versuch über die Regeln lehrt ... 137
17.13 Aufräumen ... 138
18 Vom Lab zur Produktion ... 140
18.1 Lab und Produktion im Vergleich ... 140
18.2 Mehrere k0s-Controller ... 140
18.3 Das Storage-Netz ... 141
18.4 Writecache: Nutzen und Preis ... 141
18.5 Der Piraeus-Operator am externen Controller ... 141

18.6 Fragen an ein bestehendes Setup 142
19 Anhang 143
19.1 Befehle für den Alltag 143
19.2 Fehlerbilder 143
19.3 Ports 144
19.4 Quellen 145

Vorwort

Die offiziellen Anleitungen erklären jedes Werkzeug für sich. Wie k0s, ein LINSTOR außerhalb von Kubernetes, drbd-reactor und der Piraeus-Operator als reiner CSI-Treiber zusammenspielen, und was passiert, wenn dabei etwas ausfällt, steht nirgends an einer Stelle.

Das Buch baut dieses Zusammenspiel auf vier VMs Befehl für Befehl auf und macht es danach absichtlich kaputt: Nodes fallen aus, Netze reißen, Platten laufen voll, Updates gehen schief. Zu jedem Schritt gibt es eine Erklärung, warum er nötig ist, und eine Prüfung, woran du erkennst, dass er gewirkt hat.

Für wen

Für Administratorinnen und Administratoren, die Kubernetes und Linux kennen und mit replizierendem Block-Speicher arbeiten müssen, sei es im Aufbau oder im Betrieb. Du solltest wissen, was ein PVC und ein StatefulSet ist und wie man mit LVM eine Volume-Group anlegt. DRBD und LINSTOR erklärt das Buch von Grund auf.

Was geprüft wurde

Jeder Befehl und jede Ausgabe in diesem Buch stammen aus einem Lauf im November 2026 auf vier Ubuntu-24.04-VMs unter Proxmox. Die Protokolle dieser Läufe liegen im Begleit-Repository unter `lab/protokolle/`. Lange Ausgaben sind gekürzt, das ist mit `...` markiert. Hostnamen, IPs und Ressourcennamen sind die des Labs.

Ausdrücklich **nicht** getestet und im Text so markiert sind:

- ein Export der LINSTOR-Datenbank mit `linstor-database export-db` (scheiterte im Lab, Kapitel 15)
- ein Lauf der Ansible-Rollen gegen einen leeren Node und ein Versionswechsel über die Rollen (Kapitel 16)
- mehrere k0s-Controller und alles in Kapitel 18

Versionen

k0s 1.36.6 und 1.37.2, LINSTOR 1.33.3 und 1.35.3, DRBD 9.3.4, drbd-reactor 1.12.0, Piraeus-Operator 2.12.0, linstor-csi 1.12.0 und 1.13.1. Die vollständige Liste steht in Kapitel 1.

Die Architektur

Das Lab baut ein verbreitetes Produktionsmuster nach: LINSTOR als Host-Pakete mit drbd-reactor, zwei Zonen mit einer Witness, ein eigenes Storage-Netz, Writecache auf NVMe vor HDDs, Piraeus nur als CSI. Kapitel 1 erklärt, warum.

Begleit-Repository

Alle Dateien, die das Buch anlegt, liegen im Repository unter `lab/`. Wo das Buch eine Datei zeigt, steht ihr Pfad im Kopf des Kastens. Alle Befehle mit dem Etikett **Mac** laufen im Wurzelverzeichnis des Repositorys.

Wo du einen Befehl ausführst

Jeder Befehl trägt ein Etikett:

Etikett	Ort
Mac	Dein Rechner mit `kubectl`, `helm`, `k0sctl`, `velero` und `ansible`. `KUBECONFIG` zeigt auf `~/.kube/k0s-lab.yaml`.
k0s-w1 usw.	Eine Shell auf dieser VM, per `ssh k0s-w1`. Die Kurznamen richtet Kapitel 2 ein.

KAPITEL 1

Was du aufbaust und warum so

Du hast das Gesamtbild vor Augen: welche Software auf welcher VM läuft, über welches Netz welche Daten fließen und in welcher Reihenfolge das Buch alles aufbaut.

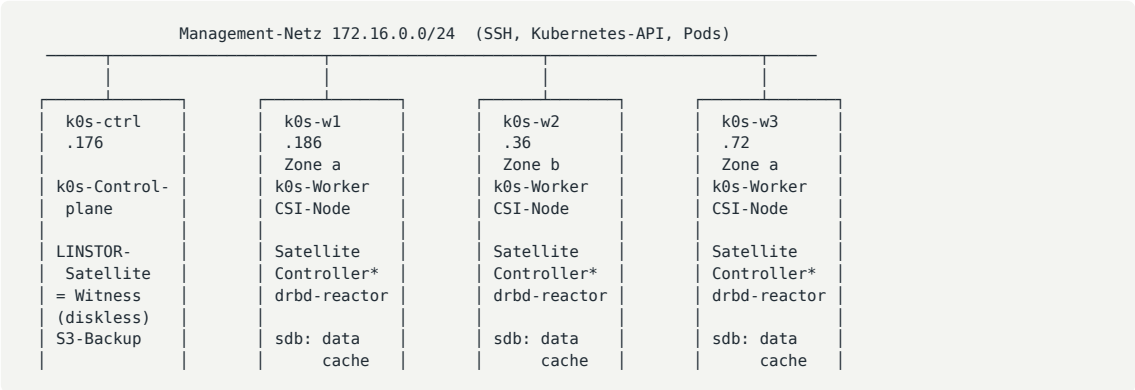
Dauer: 15 Minuten Lesen

1.1 Das Muster

Dieses Buch baut ein Architekturmuster auf, wie es in Rechenzentren mit eigener Hardware verbreitet ist. Kubernetes kommt von k0s, Block-Speicher von LINSTOR und DRBD. Die Besonderheiten dieses Musters bestimmen jedes Kapitel:

- **LINSTOR läuft außerhalb von Kubernetes**, als Pakete auf den Hosts, gestartet von systemd. Kubernetes bekommt nur den CSI-Treiber.
- **Ein aktiver LINSTOR-Controller**, dessen Datenbank selbst auf DRBD liegt. drbd-reactor schwenkt ihn samt virtueller IP auf einen anderen Host, wenn der aktive ausfällt.
- **Zwei Zonen und eine Witness**: Jedes Volume hat eine Kopie in Zone a und eine in Zone b. Eine Witness ohne Daten an einem dritten Standort entscheidet bei Netzproblemen, wer die Mehrheit hat.
- **Ein eigenes Storage-Netz** für die Replikation. Die Witness erreicht es nicht und spricht über das Management-Netz.
- **Klassisches LVM** mit einem schnellen Writecache vor langsamen Platten.

1.2 Was läuft wo



```
└──────────────┘       └──────────────┘       └──────────────┘       └──────────────┘
                              │ .11                │ .12                │ .13
──────────────────────────────┴────────────────────┴────────────────────┴──────────
              Storage-Netz 10.10.0.0/24  (DRBD-Replikation, VIP .100)

 * Controller läuft immer nur auf einem der drei, dort wo linstor_db Primary ist.
```

Verbindung	Netz
kubectl, k0sctl, SSH vom Mac	Management
DRBD zwischen den Workern	Storage
DRBD zwischen Worker und Witness	Management, über eigene Pfade (Kapitel 5)
Controller zu den Satellites der Worker	Storage
Controller zur Witness	Management
CSI-Treiber zum Controller	Storage, über die VIP `10.10.0.100`
Velero zum S3-Speicher	Management

1.3 Ein Schreibvorgang von der Datenbank bis zur Platte

Wenn PostgreSQL auf `k0s-w1` einen Block schreibt:

1. Das Dateisystem gibt den Block an `/dev/drbd1001`.
2. DRBD schreibt ihn in die Writecache-Schicht darunter und schickt ihn gleichzeitig über das Storage-Netz an `k0s-w2`.
3. Auf beiden Nodes landet der Block zuerst im schnellen Cache-Volume und wird im Hintergrund auf das Daten-Volume übertragen.
4. Erst wenn **beide** Nodes den Block bestätigt haben, meldet DRBD der Datenbank: geschrieben.

Die Witness bekommt keine Daten. Sie ist nur mit beiden verbunden und zählt beim Quorum mit.

1.4 Versionen

Komponente	Aufbau	nach Kapitel 15
Ubuntu	24.04.5, Kernel 6.8	unverändert
k0s	v1.36.6+k0s.0	v1.37.2+k0s.1
k0sctl	v0.33.1	unverändert
DRBD	9.3.4 (DKMS), drbd-utils 9.36.0 rc.1	unverändert
LINSTOR	1.33.3	1.35.3

Komponente	Aufbau	nach Kapitel 15
drbd-reactor	1.12.0	unverändert
Piraeus-Operator	2.12.0, Chart `linstor-cluster` 1.2.0	unverändert
linstor-csi	v1.12.0	v1.13.1
Velero	1.18.2 (Chart 12.2.4), AWS-Plugin 1.14.4	–

1.5 Der Bauplan

Kapitel	Ergebnis	So prüfst du es
2	VMs mit Storage-Netz und Volume-Groups	`ip route get` , `vgs`
3, 4	k0s-Cluster, Zonen als Node-Labels	`kubectl get nodes -L topology.kubernetes.io/zone`
5	LINSTOR auf allen Hosts, Witness-Pfade, Zonen	`linstor node list`
6	Controller-HA mit VIP	`drbd-reactorctl status` , `evict`
7	Pools, Writecache, Platzierung	`linstor resource list` , `lsblk`
8	CSI, StorageClass, PostgreSQL	Pod-Umzug ohne Datenverlust
9	Zustände lesen können	`drbdadm status`
10–15	Betrieb: Ausfälle, Quorum, Kapazität, Backup, Plattentausch, Updates	Messungen im Lab
16	Dasselbe mit Ansible	`changed=0`
17	Metriken und Alarme	`promtool test rules`
18	Vom Lab zur Produktion	–

WAS DU GELERNT HAST

- LINSTOR läuft auf den Hosts, Kubernetes nutzt es nur über CSI.
- Zwei Netze: Management für Verwaltung und Witness, Storage für Replikation und VIP.
- Ein Schreibvorgang ist erst fertig, wenn beide Kopien ihn haben.

KAPITEL 2

Vier VMs vorbereiten

Vier Ubuntu-VMs sind per SSH erreichbar, haben ein eigenes Storage-Netz und auf den drei Workern zwei Volume-Groups: eine große für Daten und eine kleine als Schreibcache.

Dauer: 30 Minuten **Voraussetzung:** Vier Ubuntu-24.04-VMs auf Proxmox, je Worker eine leere zweite Platte, eine zweite Netzwerkkarte an einer internen Bridge

2.1 Was du aufbaust

VM	Management-IP	Storage-IP	Rolle	Zone
`k0s-ctrl`	`172.16.0.176`	–	k0s-Controller, LINSTOR-Witness	–
`k0s-w1`	`172.16.0.186`	`10.10.0.11`	Worker, Storage	a
`k0s-w2`	`172.16.0.36`	`10.10.0.12`	Worker, Storage	b
`k0s-w3`	`172.16.0.72`	`10.10.0.13`	Worker, Storage	a

Über `eth0` laufen SSH, die Kubernetes-API und der Verkehr der Pods. Über `ens19` läuft nur die DRBD-Replikation zwischen den Workern. In einer echten Umgebung ist das ein eigenes VLAN auf eigenen Ports, im Lab eine Proxmox-Bridge ohne physischen Port.

Die Witness `k0s-ctrl` bekommt bewusst keine Adresse im Storage-Netz. Sie steht für einen Server an einem dritten Standort, bis zu dem das Storage-Netz nicht reicht. Sie spricht über das Management-Netz mit den anderen, wie das in Kapitel 5 eingerichtet wird.

WARUM DAS WICHTIG IST

DRBD schreibt jeden Block synchron auf ein zweites System. Die Latenz des Netzes geht direkt in jede Schreiboperation der Datenbank ein. Teilt sich die Replikation die Leitung mit Image-Pulls und Backups, wird jede Schreiblast unberechenbar. Ein eigenes Netz trennt diese Lasten, und du siehst im Fehlerfall sofort, welches Netz gemeint ist.

2.2 Die interne Bridge in Proxmox

In der Proxmox-Oberfläche unter *Datacenter → pve → System → Network → Create → Linux Bridge*:

- Name `vmbr1`

- IPv4/CIDR leer lassen, Bridge ports leer lassen
- *Create*, dann *Apply Configuration*

Danach bekommen die drei Worker unter *Hardware* → *Add* → *Network Device* eine zweite Karte an `vmbr1`, Modell VirtIO, Firewall aus. Die drei Worker bekommen außerdem unter *Hardware* → *Add* → *Hard Disk* eine zweite Platte.

STOLPERFALLE

Eine Bridge ohne Port verbindet nur VMs auf demselben Proxmox-Host. Liegen die VMs auf mehreren Hosts, brauchst du ein VLAN auf der physischen Bridge oder ein SDN-Netz.

2.3 SSH vom Mac

Alle Befehle mit dem Etikett „Mac“ laufen auf deinem Rechner. Damit du nicht jedes Mal IP, Benutzer und Schlüssel tippen musst, bekommen die VMs Kurznamen:

DATEI `~/.ssh/config (Ausschnitt)`

```
Host k0s-ctrl
    HostName 172.16.0.176
Host k0s-w1
    HostName 172.16.0.186
Host k0s-w2
    HostName 172.16.0.36
Host k0s-w3
    HostName 172.16.0.72
Host k0s-ctrl k0s-w1 k0s-w2 k0s-w3
    User ubuntu
    IdentityFile ~/.ssh/homelab
    IdentitiesOnly yes
```

`IdentitiesOnly` sorgt dafür, dass SSH nur diesen Schlüssel anbietet. Mit vielen Schlüsseln im Agent lehnt der Server sonst nach zu vielen Versuchen ab.

MAC — Schritt 1

```
ssh k0s-w1 'hostname; ip -br link; lsblk'
```

Zeigt Namen, Netzwerkkarten und Platten von `k0s-w1`.

DAS SOLLTEST DU SEHEN

```
k0s-w1
lo               UNKNOWN        00:00:00:00:00:00 <LOOPBACK,UP,LOWER_UP>
eth0             UP             bc:24:11:bc:2e:3b <BROADCAST,MULTICAST,UP,LOWER_UP>
ens19            DOWN           bc:24:11:ba:60:15 <BROADCAST,MULTICAST>
NAME    MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda       8:0    0   30G  0 disk
├─sda1    8:1    0   29G  0 part /
...
sdb       8:16   0   32G  0 disk
```

`ens19` ist da, aber noch `DOWN` und ohne Adresse. `sdb` ist leer, ohne Partitionen. Wiederhole den Befehl für `k0s-w2` und `k0s-w3`.

2.4 Das Storage-Netz einrichten

Ubuntu konfiguriert Netze mit netplan. Die vorhandene Datei `50-cloud-init.yaml` lässt du in Ruhe und legst eine zweite Datei nur für `ens19` an.

DATEI `lab/netplan/60-storage-k0s-w1.yaml`

```
# Storage-Netz für DRBD (vmbr1 auf Proxmox, ohne Gateway)
network:
  version: 2
  ethernets:
    ens19:
      addresses: [10.10.0.11/24]
      mtu: 1500
```

Kein Gateway und kein DNS: Das Storage-Netz führt nirgendwohin außer zu den anderen Nodes. Die Dateien für die anderen Nodes unterscheiden sich nur in der Adresse.

MAC Schritt 2

```
scp lab/netplan/60-storage-k0s-w1.yaml k0s-w1:/tmp/60-storage.yaml
ssh k0s-w1 'sudo install -m 600 /tmp/60-storage.yaml /etc/netplan/60-storage.yaml'
ssh k0s-w1 'sudo netplan apply'
ssh k0s-w1 'ip -br addr show ens19'
```

`install -m 600` setzt gleich die Rechte, die netplan verlangt. Mit lesbaren Rechten für alle warnt netplan bei jedem Aufruf.

DAS SOLLTEST DU SEHEN

```
ens19            UP             10.10.0.11/24 fe80::be24:11ff:feba:6015/64
```

Dasselbe machst du für `k0s-w2` (`10.10.0.12`) und `k0s-w3` (`10.10.0.13`) mit der jeweiligen Datei aus `lab/netplan/`. `k0s-ctrl` bleibt ohne Adresse auf `ens19`.

MAC Schritt 3

```
ssh k0s-w1 'ping -c2 -q 10.10.0.12'
ssh k0s-w1 'ip route get 10.10.0.12'
```

Der erste Befehl prüft, ob `k0s-w2` erreichbar ist. Der zweite zeigt, über welche Karte der Weg führt.

DAS SOLLTEST DU SEHEN

```
2 packets transmitted, 2 received, 0% packet loss, time 1001ms
10.10.0.12 dev ens19 src 10.10.0.11 uid 1000
```

`dev ens19` ist die wichtige Stelle. Stünde dort `eth0`, liefe die Replikation später über das falsche Netz.

2.5 Platten für LINSTOR vorbereiten

In der Architektur dieses Buchs liegen die Daten auf großen, langsamen Festplatten. Davor sitzt ein kleiner, schneller NVMe-Bereich als Schreibcache. Im Lab gibt es nur eine virtuelle Platte pro Worker, also teilst du sie in zwei Partitionen und tust so, als wären es zwei Geräte:

Partition	Größe	Volume-Group	steht für
`/dev/sdb1`	30 GB	`data`	HDD mit den Daten
`/dev/sdb2`	2 GB	`cache`	NVMe als Schreibcache

NUR IM LAB

Zwei Partitionen auf derselben virtuellen Platte sind gleich schnell. Der Cache bringt im Lab also keine Geschwindigkeit. Du lernst trotzdem den kompletten Aufbau und alle Befehle, und genau darum geht es.

K0S-W1 Schritt 4

```
sudo apt-get install -y lvm2 thin-provisioning-tools
sudo wipefs -a /dev/sdb
sudo parted -s /dev/sdb mklabel gpt mkpart data 1MiB 30GiB mkpart cache 30GiB 100%
sudo pvcreate /dev/sdb1 /dev/sdb2
sudo vgcreate data /dev/sdb1
sudo vgcreate cache /dev/sdb2
sudo vgs
```

`wipefs` entfernt alte Signaturen, falls die Platte vorher benutzt war. `parted` legt die zwei Partitionen an. `pvcreate` macht sie zu physischen LVM-Volumes, `vgcreate` fasst jede in eine eigene Volume-Group. LINSTOR legt später darin für jedes Kubernetes-Volume ein logisches Volume an.

DAS SOLLTEST DU SEHEN

```
  VG    #PV #LV #SN Attr   VSize   VFree
  cache   1   0   0 wz--n-  <2.00g  <2.00g
  data    1   0   0 wz--n- <30.00g <30.00g
```

Wiederhole das auf `k0s-w2` und `k0s-w3`. Auf `k0s-ctrl` legst du keine Volume-Groups an, er speichert später keine Daten.

STOLPERFALLE

`wipefs -a` löscht ohne Rückfrage. Prüfe vorher mit `lsblk`, dass `sdb` wirklich die leere Zusatzplatte ist und nicht die Systemplatte.

WAS DU GELERNT HAST

- Replikation bekommt ein eigenes Netz, weil ihre Latenz in jeden Schreibvorgang eingeht.
- Eine zusätzliche netplan-Datei konfiguriert nur `ens19` und lässt die Cloud-Init-Konfiguration unberührt.
- `ip route get` zeigt, über welche Karte ein Ziel erreicht wird.
- Zwei Volume-Groups bilden Datenplatte und Cache-Platte nach.

KAPITEL 3

k0s mit k0sctl installieren

Ein k0s-Cluster mit einem Controller und drei Workern läuft, und `kubectl` auf dem Mac spricht mit ihm.

Dauer: 20 Minuten **Voraussetzung:** Die vorbereiteten VMs aus Kapitel 2, Homebrew auf dem Mac

3.1 Was k0s anders macht

k0s ist eine Kubernetes-Distribution in einer einzigen Datei. Alles, was ein Node braucht, steckt im Binary `k0s`: API-Server, Scheduler, etcd, containerd, kubelet und das Netzwerk-Plugin. Es gibt keine Abhängigkeiten vom Betriebssystem außer dem Kernel.

Zwei Eigenschaften musst du kennen, weil sie sich von k3s und kubeadm unterscheiden:

- **Der Controller ist kein Node.** Auf `k0s-ctrl` läuft kein kubelet. Er taucht in `kubectl get nodes` nicht auf, und auf ihm landen nie Pods. Die Control Plane ist damit vor Workloads geschützt.
- **konnectivity verbindet API-Server und Worker.** Weil der Controller kein Node ist, kann der API-Server die Pods nicht direkt im Pod-Netz erreichen. Für `kubectl logs`, `exec` und `port-forward` baut jeder Worker einen Tunnel zum Controller auf.

WARUM DAS WICHTIG IST

Wenn in einem Cluster `kubectl logs` hängt, obwohl die Pods laufen, liegt es oft an konnectivity und nicht am Pod. Das Wissen, dass dieser Tunnel existiert, spart dir dort viel Sucherei.

3.2 k0sctl installieren

k0sctl ist das Werkzeug, mit dem du k0s auf mehrere Hosts verteilst. Es läuft auf dem Mac, verbindet sich per SSH zu den VMs und erledigt Download, Konfiguration, Join-Tokens und später auch Updates.

MAC Schritt 1

```
brew install k0sproject/tap/k0sctl
k0sctl version
```

Installiert k0sctl aus dem Homebrew-Tap des Projekts.

DAS SOLLTEST DU SEHEN

```
version: v0.33.1
commit: c4bfa95
```

3.3 Die Cluster-Beschreibung

`k0sctl init` erzeugt ein Gerüst. Das Ergebnis ist lang, weil es jeden Standardwert ausschreibt. Für das Lab reicht eine kurze Datei, in der nur steht, was vom Standard abweicht:

DATEI `lab/k0sctl.yaml`

```yaml
apiVersion: k0sctl.k0sproject.io/v1beta1
kind: Cluster
metadata:
  name: k0s-lab
spec:
  hosts:
    - role: controller
      ssh: {address: 172.16.0.176, user: ubuntu, keyPath: ~/.ssh/homelab}
    - role: worker
      ssh: {address: 172.16.0.186, user: ubuntu, keyPath: ~/.ssh/homelab}
    - role: worker
      ssh: {address: 172.16.0.36, user: ubuntu, keyPath: ~/.ssh/homelab}
    - role: worker
      ssh: {address: 172.16.0.72, user: ubuntu, keyPath: ~/.ssh/homelab}
  k0s:
    version: v1.36.6+k0s.0
    config:
      apiVersion: k0s.k0sproject.io/v1beta1
      kind: ClusterConfig
      metadata:
        name: k0s-lab
      spec:
        api:
          address: 172.16.0.176
          sans: [172.16.0.176]
        network:
          provider: kuberouter
          podCIDR: 10.244.0.0/16
          serviceCIDR: 10.96.0.0/12
        telemetry:
          enabled: false
```

`spec.hosts` legt fest, wer was wird. `role: controller` bedeutet Control Plane ohne Workloads. Die Rolle `controller+worker` gäbe es auch, sie ist aber nur für Ein-Node-Cluster gedacht. `k0s.version` pinnt die Version. Ohne diese Zeile nimmt k0sctl die neueste, und ein späteres `apply` würde den Cluster ungefragt aktualisieren. `k0s.config` ist die k0s-Konfiguration selbst. `kuberouter` ist das Standard-Netz von k0s, `telemetry` schaltet das Senden anonymer Statistiken ab.

STOLPERFALLE

Die Version heißt `v1.36.6+k0s.0`, mit dem Suffix `+k0s.0`. Ohne Suffix findet k0sctl kein Release.

3.4 Den Cluster aufbauen

MAC Schritt 2

```
k0sctl apply --config lab/k0sctl.yaml
```

k0sctl verbindet sich zu allen vier VMs, lädt k0s herunter, richtet den Controller ein, erzeugt für jeden Worker ein Join-Token und wartet, bis alle Nodes bereit sind.

DAS SOLLTEST DU SEHEN

```
level=info msg="==> Running phase: Initialize the k0s cluster"
level=info msg="172.16.0.176:22: installing k0s controller"
level=info msg="172.16.0.176:22: wait for kubernetes to reach ready state"
level=info msg="==> Running phase: Install workers"
level=info msg="172.16.0.176:22: generating a join token for worker 1"
...
level=info msg="172.16.0.186:22: waiting for node to become ready"
level=info msg="==> Finished in 2m42s"
level=info msg="k0s cluster version v1.36.6+k0s.0 is now installed"
```

`k0sctl apply` ist idempotent. Du kannst es jederzeit erneut ausführen. Es vergleicht den Ist-Zustand mit der Datei und ändert nur, was abweicht. Genau so fügst du später Nodes hinzu oder aktualisierst k0s.

3.5 kubectl verbinden

MAC Schritt 3

```
k0sctl kubeconfig --config lab/k0sctl.yaml > ~/.kube/k0s-lab.yaml
chmod 600 ~/.kube/k0s-lab.yaml
export KUBECONFIG=~/.kube/k0s-lab.yaml
kubectl get nodes -o wide
```

Holt die Admin-Kubeconfig vom Controller und legt sie als eigene Datei ab. Deine bestehende `~/.kube/config` bleibt unberührt. Die Variable `KUBECONFIG` gilt nur in diesem Terminal.

DAS SOLLTEST DU SEHEN

```
NAME     STATUS   ROLES    AGE   VERSION       INTERNAL-IP     ... CONTAINER-RUNTIME
k0s-w1   Ready    <none>   50s   v1.36.6+k0s   172.16.0.186    ... containerd://2.3.6
k0s-w2   Ready    <none>   49s   v1.36.6+k0s   172.16.0.36     ... containerd://2.3.6
k0s-w3   Ready    <none>   50s   v1.36.6+k0s   172.16.0.72     ... containerd://2.3.6
```

Drei Nodes, nicht vier. `k0s-ctrl` fehlt, und das ist richtig so.

MAC Schritt 4

```
kubectl get pods -A
```

Zeigt die Systempods. Direkt nach dem Start hängen einige noch in `ContainerCreating`, nach ein bis zwei Minuten sind alle `Running`.

Du siehst `coredns`, `kube-proxy`, `kube-router`, `metrics-server` und je Worker einen `konnectivity-agent`. API-Server, Scheduler und etcd fehlen in der Liste, weil sie auf dem Controller als normale Prozesse laufen, nicht als Pods.

3.6 Auf den Nodes nachsehen

MAC Schritt 5

```
ssh k0s-ctrl 'sudo k0s status'
ssh k0s-ctrl 'sudo k0s etcd member-list'
ssh k0s-w1 'sudo k0s status'
```

`k0s status` zeigt Rolle und Version des lokalen k0s. `k0s etcd member-list` listet die etcd-Mitglieder.

DAS SOLLTEST DU SEHEN

```
Version: v1.36.6+k0s.0
Role: controller
Workloads: false
{"members":{"k0s-ctrl":"https://172.16.0.176:2380"}}

Version: v1.36.6+k0s.0
Role: worker
Workloads: true
Kube-api probing successful: true
```

`Workloads: false` auf dem Controller ist die Bestätigung, dass dort keine Pods laufen. Mit nur einem etcd-Mitglied ist der Controller ein Single Point of Failure. Fällt er aus, laufen die Pods weiter, aber niemand kann den Cluster mehr ändern. Kapitel 18 zeigt, wie drei Controller das lösen.

WAS DU GELERNT HAST

- k0s ist ein einziges Binary, k0sctl verteilt es per SSH.
- Ein reiner Controller ist kein Node und führt keine Pods aus.
- konnectivity tunnelt `logs`, `exec` und `port-forward` vom Controller zu den Workern.
- `k0sctl apply` ist idempotent und gleicht den Cluster mit der Datei ab.
- `k0s status` und `k0s etcd member-list` sind die ersten Befehle auf einem fremden k0s-Node.

KAPITEL 4

k0s verstehen und verwalten

Du weißt, welche Prozesse auf Controller und Workern laufen, wo k0s seine Dateien ablegt und wie du die Konfiguration sicher änderst.

Dauer: 30 Minuten **Voraussetzung:** Den Cluster aus Kapitel 3

4.1 Was auf welchem Node läuft

k0s startet auf jedem Node genau einen systemd-Dienst. Alles andere sind Kindprozesse dieses Dienstes.

MAC Schritt 1

```
ssh k0s-ctrl 'systemctl list-units "k0s*" --no-legend'
ssh k0s-ctrl 'ps -eo comm --no-headers | sort -u | grep -E "kube|etcd|konn"'
```

Zeigt den Dienst auf dem Controller und die Prozesse, die er gestartet hat.

DAS SOLLTEST DU SEHEN

```
  k0scontroller.service loaded active running k0s - Zero Friction Kubernetes
etcd
konnectivity-se
kube-apiserver
kube-controller
kube-scheduler
```

Auf einem Worker heißt der Dienst `k0sworker.service`, und darunter laufen `containerd` und `kubelet`:

MAC Schritt 2

```
ssh k0s-w1 'systemctl list-units "k0s*" --no-legend'
ssh k0s-w1 'ps -eo comm --no-headers | sort -u | grep -E "kubelet|containerd$"'
```

Die Prozesse auf dem Worker.

Node	Dienst	Was darunter läuft
Controller	`k0scontroller`	etcd, kube-apiserver, kube-scheduler, kube-controller-manager, konnectivity-server
Worker	`k0sworker`	containerd, kubelet; als Pods: kube-proxy, kube-router, konnectivity-agent, coredns

WARUM DAS WICHTIG IST

Wenn ein Node Probleme macht, ist die erste Frage: läuft der Dienst? `systemctl status k0sworker` und `journalctl -u k0sworker` sind dort dein Einstieg, nicht `kubectl`. Ein Worker, dessen Dienst steht, meldet sich in Kubernetes nur noch als `NotReady`, ohne zu sagen, warum.

4.2 Wo k0s seine Dateien ablegt

Pfad	Inhalt
`/etc/k0s/k0s.yaml`	Die Cluster-Konfiguration auf dem Controller. k0sctl schreibt sie aus dem Abschnitt `spec.k0s.config`.
`/var/lib/k0s/pki`	Alle Zertifikate des Clusters.
`/var/lib/k0s/etcd`	Die etcd-Daten. Wer das verliert, verliert den Cluster-Zustand.
`/var/lib/k0s/kubelet`	Das Arbeitsverzeichnis des kubelets auf den Workern. **Nicht** `/var/lib/kubelet` wie bei fast allen anderen Distributionen.
`/var/lib/k0s/containerd`	Images und Container-Dateisysteme.
`/etc/k0s/containerd.d/`	Ergänzungen zur containerd-Konfiguration, zum Beispiel für eine eigene Registry.

STOLPERFALLE

Der abweichende kubelet-Pfad `/var/lib/k0s/kubelet` ist der häufigste Grund, warum CSI-Treiber auf k0s nicht funktionieren. Fast alle Helm-Charts gehen von `/var/lib/kubelet` aus. In Kapitel 8 läufst du genau in diese Falle und löst sie.

4.3 Werkzeuge auf den Nodes

Das Binary `k0s` bringt seine eigenen Werkzeuge mit. Du musst auf den Nodes nichts zusätzlich installieren.

MAC — Schritt 3

```
ssh k0s-ctrl 'sudo k0s kubectl get nodes'
ssh k0s-w1 'sudo k0s ctr -n k8s.io containers ls | head -4'
ssh k0s-w1 'sudo k0s sysinfo'
```

`k0s kubectl` ist ein vollwertiges kubectl mit Admin-Rechten, direkt auf dem Controller. `k0s ctr` spricht mit dem containerd des Workers. `k0s sysinfo` prüft, ob der Host alle Voraussetzungen erfüllt.

DAS SOLLTEST DU SEHEN

```
Total memory: 3.8 GiB (pass)
File system of /var/lib/k0s: ext4 (pass)
Disk space available for /var/lib/k0s: 22.5 GiB (pass)
...
  Control Groups: version 2 (pass)
```

`k0s sysinfo` ist der erste Befehl auf einem Node, der nicht starten will. Jede Zeile ohne `(pass)` ist ein Kandidat.

MAC — Schritt 4

```
ssh k0s-ctrl 'sudo journalctl -u k0scontroller --since "-10min" --no-pager | tail'
ssh k0s-w1 'sudo journalctl -u k0sworker --since "-10min" --no-pager | tail'
```

Die Logs aller Komponenten landen im Journal des einen Dienstes. Mit `grep component=` filterst du nach einzelnen Komponenten.

4.4 konnectivity prüfen

Weil der Controller kein Node ist, laufen `kubectl logs`, `exec` und `port-forward` durch die konnectivity-Tunnel der Worker.

MAC — Schritt 5

```
kubectl -n kube-system get pods -l k8s-app=konnectivity-agent -o wide
kubectl -n demo logs pg-0 --tail=2
```

Je Worker ein Agent. Funktioniert `logs`, funktioniert der Tunnel.

STOLPERFALLE

Hängen `kubectl logs` und `exec`, während `kubectl get pods` sofort antwortet, ist fast immer konnectivity schuld: Agent nicht bereit, Port 8132 auf dem Controller blockiert oder ein Controller-Neustart, nach dem sich die Agents noch nicht neu verbunden haben. Prüfe die Agent-Pods und das Journal des Controllers mit `grep konnectivity`.

4.5 Konfiguration ändern

Jede Änderung am Cluster machst du in `lab/k0sctl.yaml` und rollst sie mit `k0sctl apply` aus. Ein Beispiel, das du später brauchst: Die Worker bekommen ihre Zone als Kubernetes-Label, passend zu den Zonen in LINSTOR.

DATEI `lab/k0sctl.yaml (Ausschnitt)`

```
    - role: worker
      ssh: {address: 172.16.0.186, user: ubuntu, keyPath: ~/.ssh/homelab}
      installFlags: [--labels=topology.kubernetes.io/zone=a]
    - role: worker
      ssh: {address: 172.16.0.36, user: ubuntu, keyPath: ~/.ssh/homelab}
      installFlags: [--labels=topology.kubernetes.io/zone=b]
    - role: worker
      ssh: {address: 172.16.0.72, user: ubuntu, keyPath: ~/.ssh/homelab}
      installFlags: [--labels=topology.kubernetes.io/zone=a]
```

`installFlags` sind Parameter für `k0s install worker`. Labels, die so gesetzt werden, stehen in der Dienst-Definition und überleben jeden Neustart. Ein `kubectl label` dagegen ginge verloren, wenn der Node neu registriert wird.

MAC Schritt 6

```
k0sctl apply --config lab/k0sctl.yaml
kubectl get nodes -L topology.kubernetes.io/zone
```

k0sctl erkennt die geänderten Flags und installiert den Dienst auf den drei Workern neu.

DAS SOLLTEST DU SEHEN

```
level=info msg="==> Running phase: Reinstall"
level=info msg="172.16.0.186:22: reinstalling k0s"
level=info msg="172.16.0.36:22: reinstalling k0s"
level=info msg="172.16.0.72:22: reinstalling k0s"
level=info msg="==> Finished in 1m2s"
NAME     STATUS   ROLES    AGE   VERSION       ZONE
k0s-w1   Ready    <none>   73m   v1.36.6+k0s   a
k0s-w2   Ready    <none>   73m   v1.36.6+k0s   b
k0s-w3   Ready    <none>   73m   v1.36.6+k0s   a
```

STOLPERFALLE

k0sctl hat alle drei Worker **gleichzeitig** neu installiert. Dabei startet der kubelet neu, laufende Pods überleben das in der Regel. Bei einem großen Cluster willst du das nicht auf allen Nodes auf einmal. `k0sctl apply` kennt dafür `--concurrency`, zum Beispiel `--concurrency 1`. Bei acht oder mehr Nodes ist das die sichere Wahl.

WAS DU GELERNT HAST

- Ein Dienst pro Node: `k0scontroller` oder `k0sworker`. Alle Logs stehen in dessen Journal.
- k0s legt alles unter `/var/lib/k0s` ab, auch das kubelet.
- `k0s kubectl`, `k0s ctr` und `k0s sysinfo` sind auf jedem Node vorhanden.
- Hängen `logs` und `exec`, prüfe konnectivity.
- Änderungen gehören in `k0sctl.yaml`, ausgerollt mit `k0sctl apply`, bei großen Clustern mit `--concurrency`.

KAPITEL 5

LINSTOR und DRBD auf den Hosts

DRBD 9 und LINSTOR 1.33.3 laufen als Pakete direkt auf den VMs, außerhalb von Kubernetes. Vier Satellites sind beim Controller registriert, die Replikation läuft über das Storage-Netz, die Witness über das Management-Netz.

Dauer: 45 Minuten, davon etwa 25 Minuten Warten auf den Modul-Build **Voraussetzung:** Die VMs aus Kapitel 2

5.1 Warum LINSTOR außerhalb von Kubernetes

Piraeus kann LINSTOR komplett in Kubernetes betreiben: Controller und Satellites als Pods, DRBD per Container geladen. Dieses Buch macht es bewusst anders. LINSTOR läuft als normale systemd-Dienste auf den Hosts, und Kubernetes bekommt nur den CSI-Treiber.

WARUM DAS WICHTIG IST

Storage ist das Fundament unter dem Cluster. Läuft LINSTOR im Cluster, hängt der Speicher von Kubernetes ab und Kubernetes vom Speicher. Bei einem Ausfall des Clusters, bei einem Update von k0s oder bei einem kaputten Operator stehen dann auch die Volumes. Auf den Hosts überlebt LINSTOR all das. Du kannst k0s neu aufsetzen, und die Daten bleiben, wo sie sind. Der Preis: Du verwaltest LINSTOR selbst, mit Paketen, systemd und später Ansible.

Komponente	Wo	Aufgabe
`drbd` (Kernelmodul)	alle vier VMs	Repliziert Blöcke zwischen den Nodes.
`linstor-satellite`	alle vier VMs	Führt aus, was der Controller verlangt: LVs anlegen, DRBD konfigurieren.
`linstor-controller`	w1, w2, w3 (Kandidaten)	Hält den gewünschten Zustand in seiner Datenbank. Läuft immer nur auf einem Node.
`drbd-reactor`	w1, w2, w3	Startet den Controller dort, wo seine Datenbank gerade aktiv ist (Kapitel 6).
`linstor-client`	alle vier VMs	Die Kommandozeile `linstor`.

5.2 Woher die Pakete kommen

Für Ubuntu pflegt LINBIT ein PPA mit DRBD, drbd-utils, drbd-reactor und LINSTOR. Das PPA enthält aber immer nur die neueste Version. Ältere Builds löscht Launchpad. Im November 2026 heißt das: LINSTOR 1.35.3, nicht 1.33.3.

LINBITs öffentliches Debian-Repository hält dagegen auch ältere Versionen vor. Die LINSTOR-Pakete sind reines Java (`_all.deb`) und laufen auf Ubuntu genauso. Das Buch nimmt daher:

Paket	Version	Quelle
`drbd-dkms`	9.3.4	PPA `linbit/linbit-drbd9-stack`
`drbd-utils`	9.36.0 rc.1	PPA; die Ubuntu-Version 9.22 ist zu alt für DRBD 9.3
`drbd-reactor`	1.12.0	PPA
`linstor-client`	1.29.1	PPA
`linstor-common` , `-satellite` , `-controller`	1.33.3	`packages.linbit.com/public` , Zweig bookworm

STOLPERFALLE

Wer in einer Umgebung mit fester LINSTOR-Version arbeitet, kann sich nicht auf das PPA verlassen. Ein `apt upgrade` zieht dort sofort die neueste Version, und eine einmal ersetzte Version ist weg. In Produktion legt die Automatisierung, etwa Ansible (Kapitel 16), die Version fest. Im Lab pinnst du sie mit `apt-mark hold` .

5.3 Die Pakete installieren

Die folgenden Befehle zeigt das Buch für `k0s-w1` . Auf `k0s-w2` und `k0s-w3` sind sie identisch. Auf `k0s-ctrl` lässt du das Paket `linstor-controller` weg, denn die Witness wird nie Controller.

K0S-W1 Schritt 1

```
sudo add-apt-repository -y ppa:linbit/linbit-drbd9-stack
sudo apt-get update
sudo apt-get install -y linux-headers-$(uname -r) drbd-dkms drbd-utils \
  drbd-reactor linstor-client lvm2 thin-provisioning-tools default-jre-headless
```

`drbd-dkms` baut das DRBD-Modul passend zum laufenden Kernel, deshalb die Header. Das dauert auf einer kleinen VM sechs bis sieben Minuten. `default-jre-headless` ist die Java-Laufzeit für LINSTOR.

K0S-W1 Schritt 2

```
cd /tmp
B=https://packages.linbit.com/public/dists/bookworm/misc/pool
curl -sSfLO $B/linstor-common_1.33.3-1_all.deb
curl -sSfLO $B/linstor-satellite_1.33.3-1_all.deb
curl -sSfLO $B/linstor-controller_1.33.3-1_all.deb
sudo apt-get install -y ./linstor-common_1.33.3-1_all.deb \
  ./linstor-satellite_1.33.3-1_all.deb ./linstor-controller_1.33.3-1_all.deb
sudo apt-mark hold linstor-common linstor-satellite linstor-controller drbd-dkms drbd-utils
```

`apt-get install ./datei.deb` installiert lokale Pakete und löst ihre Abhängigkeiten trotzdem aus den Repositories auf. `apt-mark hold` schützt die Versionen vor `apt upgrade`. Updates machst du in Kapitel 15 bewusst.

K0S-W1 Schritt 3

```
sudo modprobe drbd
echo drbd | sudo tee /etc/modules-load.d/drbd.conf
cat /proc/drbd
```

Lädt das Modul jetzt und bei jedem Start.

DAS SOLLTEST DU SEHEN

```
version: 9.3.4 (api:2/proto:118-125)
GIT-hash: e59b287b199f7224468af85b68a10e83c68213f5 build by root@k0s-w1, ...
```

Steht dort `version: 8.4.x`, ist das alte Modul aus dem Ubuntu-Kernel geladen. Dann hat DKMS nicht gebaut. Prüfe `dkms status`.

5.4 LVM darf DRBD-Geräte nicht anfassen

LVM durchsucht beim Start alle Blockgeräte nach Volume-Groups. Liegt auf einem DRBD-Gerät wieder LVM, etwa in einer VM, sieht der Host dieselbe Volume-Group zweimal. Ein Filter verhindert das:

K0S-W1 Schritt 4

```
sudo sed -i 's|^\(\s*\)# global_filter = .*|\1global_filter = [ "r\|^/dev/drbd\|" ]|' /etc/lvm/lvm.conf
sudo lvmconfig devices/global_filter
```

Ersetzt die auskommentierte Beispielzeile durch einen Filter, der alle `/dev/drbd*` ausschließt. Das gilt für alle vier Nodes.

DAS SOLLTEST DU SEHEN

```
global_filter="r|^/dev/drbd|"
```

5.5 Eine Falle direkt nach der Installation

K0S-W1 Schritt 5

```
systemctl is-active linstor-satellite linstor-controller drbd-reactor
```

Prüft, welche Dienste die Pakete gestartet haben.

DAS SOLLTEST DU SEHEN

```
active
active
active
```

Das Paket startet den Controller sofort, auf **jedem** Node, auf dem es installiert ist. Du hast jetzt drei unabhängige Controller mit je einer leeren Datenbank. Registrierst du einen Node bei einem davon, wissen die anderen nichts davon.

K0S-W2 Schritt 6

```
sudo systemctl disable --now linstor-controller
```

Dasselbe auf `k0s-w3`. Auf `k0s-w1` bleibt der Controller vorerst aktiv. In Kapitel 6 übernimmt drbd-reactor ihn.

STOLPERFALLE

Zwei aktive LINSTOR-Controller sind kein Ausfallschutz, sondern ein Split-Brain auf Verwaltungsebene. Es darf immer nur einen geben. Nach jeder Paketinstallation auf einem Controller-Kandidaten gilt: `systemctl is-enabled linstor-controller` muss `disabled` melden.

5.6 Den Client einrichten

Der Client fragt standardmäßig `localhost`. Auf jedem Node legst du fest, wo der Controller ist:

DATEI `/etc/linstor/linstor-client.conf`

```
[global]
controllers=10.10.0.11
```

Vorerst die Storage-IP von `k0s-w1`. In Kapitel 6 wird daraus die VIP `10.10.0.100`.

5.7 Nodes registrieren

Ab hier laufen alle `linstor`-Befehle auf einem beliebigen Worker. Das Buch nutzt `k0s-w1`.

K0S-W1 Schritt 7

```
linstor node create k0s-w1 10.10.0.11 --node-type satellite
linstor node create k0s-w2 10.10.0.12 --node-type satellite
linstor node create k0s-w3 10.10.0.13 --node-type satellite
linstor node create k0s-ctrl 172.16.0.176 --node-type satellite
linstor node list
```

Die IP ist die Standard-Schnittstelle des Nodes. Über sie spricht der Controller mit dem Satellite, und über sie läuft DRBD, solange nichts anderes festgelegt ist. Die Worker bekommen deshalb ihre Storage-IP, die Witness ihre Management-IP.

DAS SOLLTEST DU SEHEN

```
| Node     | NodeType  | Addresses                  | State  |
| k0s-ctrl | SATELLITE | 172.16.0.176:3366 (PLAIN) | Online |
| k0s-w1   | SATELLITE | 10.10.0.11:3366 (PLAIN)   | Online |
| k0s-w2   | SATELLITE | 10.10.0.12:3366 (PLAIN)   | Online |
| k0s-w3   | SATELLITE | 10.10.0.13:3366 (PLAIN)   | Online |
```

Beim Anlegen meldet jeder Node, welche Speicherarten und Schichten er kann. Wichtig sind `lvm` und die Schichten `drbd` und `writecache`. Die Meldungen zu ZFS und SPDK kannst du ignorieren.

5.8 Die Witness über das Management-Netz anbinden

Die Witness sitzt in dieser Architektur an einem dritten Standort, bis zu dem das Storage-Netz nicht reicht. Im Lab hat `k0s-ctrl` deshalb keine Adresse im Storage-Netz. Für eine Verbindung zwischen `k0s-w1` und `k0s-ctrl` würde DRBD aber die Standard-Schnittstelle von `k0s-w1` nehmen, also `10.10.0.11`. Die Witness kann darauf nicht antworten.

Die Lösung ist ein **Pfad**: Für bestimmte Node-Paare legst du fest, über welche Schnittstellen sie sich verbinden.

K0S-W1 Schritt 8

```
linstor node interface create k0s-w1 mgmt 172.16.0.186
linstor node interface create k0s-w2 mgmt 172.16.0.36
linstor node interface create k0s-w3 mgmt 172.16.0.72
linstor node interface create k0s-ctrl mgmt 172.16.0.176
linstor node-connection path create k0s-w1 k0s-ctrl witness mgmt mgmt
linstor node-connection path create k0s-w2 k0s-ctrl witness mgmt mgmt
linstor node-connection path create k0s-w3 k0s-ctrl witness mgmt mgmt
```

Jeder Node bekommt eine zweite Schnittstelle namens `mgmt`. Die drei Pfade sagen: Zwischen einem Worker und der Witness läuft DRBD über `mgmt` auf beiden Seiten.

K0S-W1 Schritt 9

```
linstor node-connection path list k0s-w1 k0s-ctrl
```

Zeigt den Pfad.

DAS SOLLTEST DU SEHEN

```
| Key                    | Value |
| Paths/witness/k0s-ctrl | mgmt  |
| Paths/witness/k0s-w1   | mgmt  |
```

Ob DRBD den Pfad wirklich nutzt, siehst du erst an einer Ressource, in Kapitel 6.

5.9 Zonen festlegen

Die Server stehen in zwei Brandabschnitten, Zone a und Zone b. Ein Volume soll je ein Replikat in jeder Zone haben. LINSTOR kennt Zonen nicht von sich aus. Du hängst sie als freie Eigenschaft an die Nodes:

K0S-W1 Schritt 10

```
linstor node set-property k0s-w1 Aux/zone a
linstor node set-property k0s-w2 Aux/zone b
linstor node set-property k0s-w3 Aux/zone a
linstor node list-properties k0s-w2
```

`Aux/` markiert eigene Eigenschaften. LINSTOR wertet sie nicht aus, du kannst sie aber in Platzierungsregeln verwenden (Kapitel 7).

DAS SOLLTEST DU SEHEN

```
| Aux/zone        | b       |
| CurStltConnName | default |
| NodeUname       | k0s-w2  |
```

WAS DU GELERNT HAST

- LINSTOR auf den Hosts überlebt jeden Ausfall und jedes Update von Kubernetes.
- Das PPA hat nur die neueste Version. Feste Versionen kommen aus LINBITs Debian-Repository und werden mit `apt-mark hold` gesichert.
- Nach der Installation laufen Controller auf allen Kandidaten. Es darf nur einen geben.
- Die Node-IP ist die Standard-Schnittstelle für Satellite-Verkehr und DRBD.
- Pfade legen für einzelne Node-Paare ein anderes Netz fest, hier für die Witness.
- Zonen sind `Aux/`-Eigenschaften an den Nodes.

KAPITEL 6

Der LINSTOR-Controller ohne Single Point of Failure

Die Datenbank des Controllers liegt auf einer eigenen DRBD-Ressource. drbd-reactor startet den Controller samt VIP dort, wo diese Ressource aktiv ist, und schwenkt bei einem Ausfall in Sekunden auf einen anderen Node.

Dauer: 40 Minuten **Voraussetzung:** Die registrierten Nodes aus Kapitel 5, die Storage-Pools aus Kapitel 7 Abschnitt „Storage-Pools anlegen"

REIHENFOLGE

`linstor_db` ist selbst eine LINSTOR-Ressource und braucht deshalb einen Storage-Pool. Lege zuerst die beiden Pools aus dem Abschnitt „Storage-Pools anlegen" in Kapitel 7 an und kehre dann hierher zurück.

6.1 Das Prinzip

Der LINSTOR-Controller ist zustandslos bis auf seine Datenbank, eine H2-Datei unter `/var/lib/linstor`. Wer diese Datei hat, kann Controller sein. Die Idee:

1. Die Datenbank liegt auf einer kleinen DRBD-Ressource `linstor_db`, die auf allen drei Workern repliziert ist.
2. Auf dem Node, auf dem `linstor_db` Primary ist, wird sie nach `/var/lib/linstor` gemountet und der Controller gestartet.
3. Eine virtuelle IP zieht mit. Clients und der CSI-Treiber sprechen immer mit der VIP.
4. drbd-reactor überwacht `linstor_db`. Fällt der aktive Node aus, macht ein anderer Node die Ressource zum Primary und startet die Kette dort.

WARUM DAS WICHTIG IST

DRBD sorgt selbst dafür, dass nur ein Node Primary sein kann, solange er Quorum hat. Damit gibt es nie zwei Controller gleichzeitig, auch nicht bei einer Netztrennung. Ein Pacemaker-Cluster ist dafür nicht nötig. Das ist das Muster dieses Buchs: ein aktiver Controller, die Datenbank auf mehreren Hosts repliziert, eine disklose Witness am dritten Standort.

6.2 Die Ressource linstor_db

K0S-W1 — Schritt 1

```
linstor resource-group create --storage-pool pool_data --place-count 3 \
  --diskless-on-remaining true linstor-db-grp
linstor resource-group drbd-options --auto-promote=no --quorum=majority \
  --on-suspended-primary-outdated=force-secondary \
  --on-no-quorum=io-error --on-no-data-accessible=io-error linstor-db-grp
linstor volume-group create linstor-db-grp
linstor resource-group spawn-resources linstor-db-grp linstor_db 200M
```

Eine Resource-Group ist eine Vorlage. `--place-count 3` legt drei Kopien mit Daten an, `--diskless-on-remaining` gibt allen übrigen Nodes, hier der Witness, eine Kopie ohne Daten, die beim Quorum mitzählt.

Option	Bedeutung
`auto-promote=no`	Niemand wird automatisch Primary, nur weil er das Gerät öffnet. Das übernimmt allein drbd-reactor.
`quorum=majority`	Schreiben nur, wenn die Mehrheit der Knoten erreichbar ist. Bei vier Stimmen also mindestens drei.
`on-no-quorum=io-error`	Verliert der Primary sein Quorum, bekommt der Controller sofort Fehler statt zu hängen. drbd-reactor erkennt das und gibt ab.
`on-suspended-primary-outdated=force-secondary`	Ein abgeschnittener Primary, der wieder auftaucht und veraltet ist, wird zwangsweise Secondary.

K0S-W1 — Schritt 2

```
linstor resource list -r linstor_db
```

Zeigt, wo die Ressource liegt.

DAS SOLLTEST DU SEHEN

```
| ResourceName | Node     | Layers       | Usage  | Conns |    State | Vote |
| linstor_db   | k0s-ctrl | DRBD,STORAGE | Unused | Ok    | Diskless | Yes  |
| linstor_db   | k0s-w1   | DRBD,STORAGE | Unused | Ok    | UpToDate | Yes  |
| linstor_db   | k0s-w2   | DRBD,STORAGE | Unused | Ok    | UpToDate | Yes  |
| linstor_db   | k0s-w3   | DRBD,STORAGE | Unused | Ok    | UpToDate | Yes  |
```

Drei Kopien mit Daten, eine ohne. Alle vier haben eine Stimme (`Vote: Yes`).

Jetzt siehst du auch, ob der Witness-Pfad aus Kapitel 5 wirkt:

K0S-W1 Schritt 3

```
sudo drbdsetup show linstor_db | grep -E "_this_host ipv4|_remote_host ipv4"
```

Zeigt für jede Verbindung die lokale und die entfernte Adresse.

DAS SOLLTEST DU SEHEN

```
            _this_host ipv4 172.16.0.186:7000;
            _remote_host ipv4 172.16.0.176:7000;
            _this_host ipv4 10.10.0.11:7000;
            _remote_host ipv4 10.10.0.12:7000;
            _this_host ipv4 10.10.0.11:7000;
            _remote_host ipv4 10.10.0.13:7000;
```

Zur Witness über `172.16.0.x` , zu den anderen Workern über `10.10.0.x` . So soll es sein.

6.3 Die Datenbank umziehen

Der Controller läuft noch auf `k0s-w1` mit seiner Datenbank auf der Systemplatte. Jetzt wird sie auf `linstor_db` kopiert.

DATEI `lab/linstor/var-lib-linstor.mount`

```
[Unit]
Description=Filesystem for the LINSTOR controller

[Mount]
What=/dev/drbd/by-res/linstor_db/0
Where=/var/lib/linstor
```

Eine systemd-Mount-Unit. drbd-reactor startet sie, bevor er den Controller startet. Der Dateiname muss zum Pfad passen: `/var/lib/linstor` wird zu `var-lib-linstor.mount` .

MAC Schritt 4

```
scp lab/linstor/var-lib-linstor.mount k0s-w1:/tmp/
ssh k0s-w1 'sudo install -m 644 /tmp/var-lib-linstor.mount /etc/systemd/system/'
ssh k0s-w1 'sudo systemctl daemon-reload'
```

Dasselbe für `k0s-w2` und `k0s-w3` . Aktiviere die Unit **nicht**, sie wird nur von drbd-reactor gestartet.

K0S-W1 Schritt 5

```
sudo systemctl disable --now linstor-controller
sudo mv /var/lib/linstor /var/lib/linstor.orig
sudo mkdir /var/lib/linstor
sudo drbdadm primary linstor_db
sudo mkfs.ext4 -b 4096 /dev/drbd/by-res/linstor_db/0
sudo systemctl start var-lib-linstor.mount
sudo cp -a /var/lib/linstor.orig/. /var/lib/linstor/
sudo umount /var/lib/linstor
sudo drbdadm secondary linstor_db
```

Controller stoppen, alte Datenbank zur Seite legen, `linstor_db` zum Primary machen, ein Dateisystem anlegen, mounten, die Datenbank hineinkopieren und alles wieder freigeben. Ab jetzt ist der Controller aus, bis drbd-reactor ihn startet.

STOLPERFALLE

`/var/lib/linstor.orig` bleibt als Sicherung liegen. Lösche es erst, wenn der Controller einige Tage stabil auf `linstor_db` läuft. Und starte nie wieder einen Controller, der darauf zeigt: Er hätte einen veralteten Stand.

6.4 drbd-reactor einrichten

Für die VIP nutzt drbd-reactor den bewährten OCF-Agenten `IPaddr2` aus dem Pacemaker-Umfeld. Er steckt im Paket `resource-agents-base`.

K0S-W1 Schritt 6

```
sudo apt-get install -y resource-agents-base
```

Auf allen drei Controller-Kandidaten.

DATEI `lab/linstor/linstor_db.toml`

```
# drbd-reactor: genau ein aktiver LINSTOR-Controller, dort wo linstor_db Primary ist
[[promoter]]
id = "linstor_db"
[promoter.resources.linstor_db]
start = [
  "var-lib-linstor.mount",
  "ocf:heartbeat:IPaddr2 linstor_vip ip=10.10.0.100 cidr_netmask=24 nic=ens19",
  "linstor-controller.service",
]
on-drbd-demote-failure = "reboot"
stop-services-on-exit = true
```

`start` ist eine Kette: Mount, dann VIP, dann Controller. Beim Abschalten geht es rückwärts. Die VIP liegt im Storage-Netz. Der CSI-Treiber erreicht sie, weil jeder Worker eine Adresse in diesem Netz hat. `on-drbd-demote-failure = "reboot"`: Lässt sich `linstor_db` beim Abgeben nicht zum Secondary machen, weil etwas das Gerät festhält, startet der Node neu. Das ist hart, aber besser als ein Node, der die

Datenbank nicht loslässt. `stop-services-on-exit`: Wird drbd-reactor selbst gestoppt, fährt er auch die Kette herunter.

MAC Schritt 7

```
scp lab/linstor/linstor_db.toml k0s-w1:/tmp/
ssh k0s-w1 'sudo install -m 644 /tmp/linstor_db.toml /etc/drbd-reactor.d/'
```

Dasselbe für `k0s-w2` und `k0s-w3`.

Bevor drbd-reactor startet, stellst du alle Clients auf die VIP um:

DATEI `/etc/linstor/linstor-client.conf (auf w1, w2, w3)`

```
[global]
controllers=10.10.0.100
```

DATEI `/etc/linstor/linstor-client.conf (auf k0s-ctrl)`

```
[global]
controllers=172.16.0.186,172.16.0.36,172.16.0.72
```

Die Witness hat keine Adresse im Storage-Netz und erreicht die VIP nicht. Sie bekommt die Liste der Kandidaten und probiert sie der Reihe nach durch.

K0S-W1 Schritt 8

```
sudo systemctl enable drbd-reactor
sudo systemctl restart drbd-reactor
```

Auf allen drei Kandidaten. Der erste, der `linstor_db` zum Primary machen kann, startet die Kette.

K0S-W1 Schritt 9

```
sudo drbd-reactorctl status linstor_db
ip -br addr show ens19
```

Zeigt die Kette und ob die VIP auf diesem Node liegt.

DAS SOLLTEST DU SEHEN

```
Promoter: Resource linstor_db currently active on this node
● drbd-services@linstor_db.target
● ├─ drbd-promote@linstor_db.service
● ├─ var-lib-linstor.mount
● ├─ ocf.rs@linstor_vip_linstor_db.service
● └─ linstor-controller.service
ens19            UP             10.10.0.11/24 10.10.0.100/24 ...
```

Auf `k0s-w2` und `k0s-w3` steht stattdessen `currently active on node 'k0s-w1'` und leere Kreise.

K0S-W2 Schritt 10

```
linstor node list
```

Ein Befehl von einem anderen Node über die VIP. Antwortet er mit allen vier Nodes `Online`, ist der Umzug gelungen.

6.5 Den Failover ausprobieren

`drbd-reactorctl evict` zwingt den aktiven Node, abzugeben. Das ist der geplante Weg, etwa vor einer Wartung.

K0S-W1 Schritt 11

```
sudo drbd-reactorctl evict linstor_db
```

Fährt die Kette auf `k0s-w1` herunter und wartet, bis ein anderer Node übernommen hat.

DAS SOLLTEST DU SEHEN

```
Node 'k0s-w3' took over
Re-enabling linstor_db
```

Im Lab war der Controller nach **15 Sekunden** wieder über die VIP erreichbar. Laufende Volumes merken davon nichts: DRBD repliziert weiter, auch ohne Controller. Nur neue Volumes, Löschungen und Änderungen müssen warten.

K0S-W3 Schritt 12

```
sudo drbd-reactorctl status linstor_db | head -2
linstor resource list -r linstor_db
```

Die Ressource ist jetzt auf `k0s-w3` in Benutzung.

DAS SOLLTEST DU SEHEN

```
Promoter: Resource linstor_db currently active on this node
| linstor_db   | k0s-w3   | DRBD,STORAGE | InUse  | Ok    | UpToDate | Yes  |
```

WARUM DAS WICHTIG IST

Der wichtigste Satz dieses Kapitels: **Fällt der Controller aus, laufen alle Volumes weiter.** Der Controller verwaltet nur. Die Daten fließen direkt zwischen den DRBD-Modulen der Nodes. Wenn der Controller hängt, hast du Zeit. Panik ist erst angebracht, wenn `drbdadm status` auf den Nodes Probleme zeigt.

Den ungeplanten Fall, einen hart ausgeschalteten Node mit aktivem Controller, spielst du in Kapitel 10 durch.

WAS DU GELERNT HAST

- Der Controller ist austauschbar, solange seine Datenbank repliziert ist.
- drbd-reactor startet Mount, VIP und Controller als Kette auf dem Primary von `linstor_db`.
- DRBD-Quorum verhindert zwei Controller gleichzeitig.
- `drbd-reactorctl status` zeigt, wo der Controller läuft. `evict` schiebt ihn geplant weiter.
- Ein Controller-Ausfall stoppt keine laufenden Volumes.

KAPITEL 7

Storage-Pools, Writecache und Zonen

Zwei Storage-Pools auf jedem Worker, eine Vorlage für replizierte Volumes mit Writecache und eine Platzierung, die je ein Replikat pro Zone erzwingt.

Dauer: 30 Minuten **Voraussetzung:** Die Volume-Groups aus Kapitel 2, die Nodes aus Kapitel 5

7.1 Storage-Pools anlegen

Ein Storage-Pool sagt LINSTOR, wo es auf einem Node Speicher anlegen darf. Jeder Worker bekommt zwei:

K0S-W1 Schritt 1

```
linstor storage-pool create lvm k0s-w1 pool_data data
linstor storage-pool create lvm k0s-w2 pool_data data
linstor storage-pool create lvm k0s-w3 pool_data data
linstor storage-pool create lvm k0s-w1 pool_nvme cache
linstor storage-pool create lvm k0s-w2 pool_nvme cache
linstor storage-pool create lvm k0s-w3 pool_nvme cache
linstor storage-pool list
```

`lvm` ist der Treiber, `pool_data` der Name in LINSTOR, `data` die Volume-Group auf dem Node. Gleiche Pool-Namen auf allen Nodes sind wichtig: Platzierungsregeln verweisen nur auf den Namen.

DAS SOLLTEST DU SEHEN

```
| StoragePool          | Node     | Driver   | PoolName | FreeCapacity | TotalCapacity |
| DfltDisklessStorPool | k0s-ctrl | DISKLESS |          |              |               |
| ...                                                                                  |
| pool_data            | k0s-w1   | LVM      | data     |    30.00 GiB |     30.00 GiB |
| pool_data            | k0s-w2   | LVM      | data     |    30.00 GiB |     30.00 GiB |
| pool_data            | k0s-w3   | LVM      | data     |    30.00 GiB |     30.00 GiB |
| pool_nvme            | k0s-w1   | LVM      | cache    |     2.00 GiB |      2.00 GiB |
| pool_nvme            | k0s-w2   | LVM      | cache    |     2.00 GiB |      2.00 GiB |
| pool_nvme            | k0s-w3   | LVM      | cache    |     2.00 GiB |      2.00 GiB |
```

`DfltDisklessStorPool` legt LINSTOR auf jedem Node selbst an. Er steht für Kopien ohne Daten.

LVM ODER LVM-THIN?

`lvm` reserviert den vollen Platz eines Volumes sofort. `lvmthin` reserviert nur, was geschrieben wird, und kann Snapshots. Dieses Buch nutzt klassisches LVM: berechenbar, kein Überbuchen, kein voller Thin-Pool, der alle Volumes gleichzeitig einfriert. Der Preis sind fehlende Snapshots, darum geht es in Kapitel 13.

7.2 Was ein Writecache ist

`dm-writecache` ist eine Funktion des Linux-Kernels. Schreibvorgänge landen zuerst auf dem schnellen Gerät und werden im Hintergrund auf das langsame übertragen. Lesen bleibt beim langsamen Gerät. LINSTOR baut das als eigene Schicht zwischen DRBD und LVM ein:

Schicht	Aufgabe
DRBD	Repliziert jeden Block auf den zweiten Node.
WRITECACHE	Puffert Schreibvorgänge auf `pool_nvme`.
STORAGE	Das logische Volume in `pool_data`.

`Writecache/Size` legt fest, wie groß der Cache je Volume ist. `5%` bedeutet: Ein 2-GB-Volume bekommt etwa 100 MB Cache.

WARUM DAS WICHTIG IST

Der Cache sitzt **unter** DRBD. DRBD bestätigt einen Schreibvorgang erst, wenn beide Nodes ihn haben. Mit Writecache heißt „haben“: auf der NVMe des jeweiligen Nodes. Die langsame Platte bremst damit nicht mehr jeden Schreibvorgang. Was im Cache steht, ist trotzdem auf beiden Seiten sicher.

7.3 Die Vorlage rg-k8s

K0S-W1 — Schritt 2

```
linstor resource-group create rg-k8s --storage-pool pool_data --place-count 2 \
  --replicas-on-different zone --layer-list drbd,writecache,storage
linstor resource-group set-property rg-k8s Writecache/PoolName pool_nvme
linstor resource-group set-property rg-k8s Writecache/Size 5%
linstor resource-group drbd-options --quorum=majority --on-no-quorum=suspend-io \
  --on-suspended-primary-outdated=force-secondary rg-k8s
linstor volume-group create rg-k8s
```

Zwei Replikate in `pool_data`, auf Nodes mit unterschiedlichem Wert für `Aux/zone`, mit Writecache aus `pool_nvme`.

Einstellung	Wirkung
`--place-count 2`	Zwei Kopien mit Daten.
`--replicas-on-different zone`	Nie zwei Kopien in derselben Zone. Der Client ergänzt `Aux/` selbst.
`on-no-quorum=suspend-io`	Verliert ein Primary das Quorum, hält DRBD alle Schreibvorgänge an, statt Fehler zu liefern. Die Anwendung hängt, statt ihr Dateisystem zu verlieren. Kommt das Quorum zurück, läuft sie weiter.

ANDERS ALS BEI LINSTOR_DB

Für `linstor_db` gilt `io-error`, damit drbd-reactor sofort umschalten kann. Für Anwendungsdaten gilt `suspend-io`: Ein kurzer Netzaussetzer soll keine Datenbank zerstören. Kapitel 11 zeigt beide Verhaltensweisen live.

7.4 Ausprobieren

K0S-W1 Schritt 3

```
linstor resource-group spawn-resources rg-k8s test1 1G
linstor resource list -r test1
```

Erzeugt ein Test-Volume aus der Vorlage.

DAS SOLLTEST DU SEHEN

```
| ResourceName | Node     | Layers                  | Usage  | Conns |
State | Vote |
| test1        | k0s-ctrl | DRBD,STORAGE            | Unused | Ok    |
Diskless | Yes  |
| test1        | k0s-w1   | DRBD,WRITECACHE,STORAGE | Unused | Ok    |
UpToDate | Yes  |
| test1        | k0s-w2   | DRBD,WRITECACHE,STORAGE | Unused | Ok    |
SyncTarget(62.30%) | Yes  |
```

Ein Replikat in Zone a (`k0s-w1`), eines in Zone b (`k0s-w2`). `k0s-w3` liegt ebenfalls in Zone a und bleibt deshalb leer. Auf der Witness hat LINSTOR von sich aus einen **Tiebreaker** angelegt, eine Kopie ohne Daten, die beim Quorum mitstimmt. Mit nur zwei Stimmen gäbe es bei einer Netztrennung keine Mehrheit.

Wie das auf der Platte aussieht, zeigt `lsblk` auf `k0s-w1`:

K0S-W1 Schritt 4

```
lsblk /dev/sdb
sudo dmsetup table | grep writecache
```

Der Gerätestapel des Test-Volumes.

DAS SOLLTEST DU SEHEN

```
sdb                                     8:16    0    32G  0 disk
├─sdb1                                  8:17    0    30G  0 part
│ ├─data-linstor_db_00000             252:0     0   204M  0 lvm
│ │ └─drbd1000                        147:1000  0 203.9M  0 disk
│ └─data-test1_00000                  252:1     0     1G  0 lvm
│   └─linstor_writecache_test1_00000 252:3     0     1G  0 dm
│     └─drbd1001                      147:1001  0     1G  0 disk
└─sdb2                                  8:18    0     2G  0 part
  └─cache-test1.cache_00000           252:2     0    52M  0 lvm
    └─linstor_writecache_test1_00000 252:3     0     1G  0 dm
      └─drbd1001                      147:1001  0     1G  0 disk
linstor_writecache_test1_00000: 0 2105344 writecache s 252:1 252:2 4096 0
```

`linstor_writecache_test1_00000` setzt sich aus dem Daten-LV (`252:1`) und dem 52-MB-Cache-LV (`252:2`) zusammen. Darauf sitzt `drbd1001` . Das sind 5 % von 1 GB, aufgerundet auf LVM-Blöcke.

K0S-W1 Schritt 5

```
linstor resource-definition delete test1
```

Räumt das Test-Volume wieder ab.

STOLPERFALLE

Die Vorlage `rg-k8s` ist für LINSTOR selbst gedacht. Sobald der CSI-Treiber sie benutzt, überschreibt er ihre Einstellungen mit den Parametern der StorageClass. Was dort fehlt, setzt er auf seine Standardwerte. Kapitel 8 zeigt, wie das im Lab aus zwei Replikaten eines gemacht hat.

WAS DU GELERNT HAST

- Storage-Pools verbinden einen Namen in LINSTOR mit einer Volume-Group auf dem Node.
- Klassisches LVM ist berechenbar, kann aber keine Snapshots.
- Die Writecache-Schicht puffert Schreibvorgänge unter DRBD auf schnellem Speicher.
- `--replicas-on-different zone` verteilt Replikate über Zonen.
- LINSTOR legt Tiebreaker auf der Witness selbst an.

KAPITEL 8

Piraeus als CSI-Treiber

Kubernetes legt Volumes über den externen LINSTOR-Controller an. Der Piraeus-Operator liefert nur den CSI-Treiber, keine Satellites. Eine PostgreSQL-Datenbank läuft auf einem replizierten Volume und zieht ohne Datenverlust auf einen anderen Node um.

Dauer: 45 Minuten **Voraussetzung:** Den Controller hinter der VIP aus Kapitel 6, `helm` auf dem Mac

8.1 Was der Operator hier tut und was nicht

Der Piraeus-Operator kann alles: Controller, Satellites, DRBD-Modul, CSI. Hier soll er nur eines tun: den CSI-Treiber bereitstellen, der mit dem LINSTOR auf den Hosts spricht.

Komponente des Operators	Hier	Warum
LINSTOR-Controller	aus	Läuft auf den Hosts, erreichbar über die VIP.
Satellites	aus	Laufen auf den Hosts als systemd-Dienste.
CSI-Controller und CSI-Node	an	Übersetzen PVCs in LINSTOR-Aufrufe und mounten die Volumes.
HA-Controller	aus	Würde Pods bei DRBD-Problemen zwangsweise verschieben. Dieses Buch verzichtet bewusst darauf.
Affinity-Controller	aus	Passt PV-Affinitäten an, wenn LINSTOR Replikate verschiebt. Nicht genutzt.
NFS-Server	aus	Für ReadWriteMany-Volumes. Nicht genutzt.

8.2 Den Operator installieren

Das Helm-Chart des Operators liegt im Git-Repository des Projekts, nicht in einem Chart-Repository. Seine Version entspricht der Operator-Version.

MAC Schritt 1

```
git clone --depth 1 --branch v2.12.0 https://github.com/piraeusdatastore/piraeus-operator.git
helm install piraeus-operator piraeus-operator/charts/piraeus \
  --namespace piraeus-datastore --create-namespace --set installCRDs=true
kubectl -n piraeus-datastore rollout status deploy/piraeus-operator-controller-manager
```

`installCRDs=true` ist nötig. Der Standardwert `false` installiert den Operator ohne seine Ressourcen-Typen.

DAS SOLLTEST DU SEHEN

```
deployment "piraeus-operator-controller-manager" successfully rolled out
```

8.3 Der Cluster als Helm-Werte

Die Ressource `LinstorCluster` beschreibt, was der Operator betreiben soll. Das Chart `linstor-cluster` erzeugt sie aus Helm-Werten.

MAC Schritt 2

```
helm repo add piraeus-charts https://piraeus.io/helm-charts/
helm repo update
```

Das Chart-Repository für `linstor-cluster`.

Die Werte entstehen in diesem Kapitel Schritt für Schritt, weil jeder Schritt eine Falle offenlegt. Am Ende steht die vollständige Datei `lab/piraeus/linstor-cluster-values.yaml`.

Erster Versuch: externer Controller, keine Satellites

DATEI `Erster Stand der Werte`

```
linstorCluster:
  externalController:
    url: http://10.10.0.100:3370
  nodeSelector:
    piraeus.io/managed-satellite: "true"
  highAvailabilityController: {enabled: false}
  affinityController: {enabled: false}
  nfsServer: {enabled: false}
  csiController:
    podTemplate:
      spec:
        containers:
          - name: linstor-csi
            image: quay.io/piraeusdatastore/piraeus-csi:v1.12.0
  csiNode:
    podTemplate:
      spec:
        containers:
          - name: linstor-csi
            image: quay.io/piraeusdatastore/piraeus-csi:v1.12.0
```

`externalController` zeigt auf die VIP. Der Operator startet dann keinen eigenen Controller. Der `nodeSelector` nennt ein Label, das kein Node trägt. Damit findet der Operator keinen Node für Satellites. Das CSI-Image ist auf v1.12.0 gepinnt. Der Operator 2.12 bringt v1.13.1 mit, und CSI ab 1.13 setzt LINSTOR 1.35 voraus. Mit LINSTOR 1.33 muss CSI auf 1.12 bleiben. In Kapitel 15 werden beide gemeinsam aktualisiert.

MAC Schritt 3

```
helm install linstor-cluster piraeus-charts/linstor-cluster --version 1.2.0 \
  -n piraeus-datastore -f werte.yaml
kubectl -n piraeus-datastore get pods
```

Installiert die Cluster-Beschreibung.

DAS SOLLTEST DU SEHEN

```
NAME                                                READY   STATUS    RESTARTS   AGE
linstor-csi-controller-d58ff7b44-4vg29              0/6     Pending   0          59s
piraeus-operator-controller-manager-54fd7c49f9-767dh   1/1     Running   0          100s
```

Der Controller ist erreichbar, aber der CSI-Controller hängt in `Pending`, und CSI-Node-Pods gibt es gar keine.

STOLPERFALLE

Der `nodeSelector` gilt nicht nur für Satellites, sondern auch für alle CSI-Pods. `kubectl describe` am Pod sagt: `3 node(s) didn't match Pod's node affinity/selector`. Im Quellcode des Operators sieht man, dass derselbe Selector auf Satellites, CSI-Controller, CSI-Node und weitere Komponenten angewendet wird.

Die Lösung: Den Selector in den Pod-Vorlagen der CSI-Komponenten wieder entfernen. Die `podTemplate` wird als Strategic-Merge-Patch angewendet. Darin löscht `null` einen Schlüssel:

DATEI `Ergänzung für csiController und csiNode`

```
    podTemplate:
      spec:
        nodeSelector:
          piraeus.io/managed-satellite: null
```

Zweiter Versuch: der kubelet-Pfad von k0s

Nach dem Update laufen der CSI-Controller und drei CSI-Node-Pods an. Die Node-Pods bleiben aber in `Init:0/1` stehen.

MAC Schritt 4

```
kubectl -n piraeus-datastore describe pod -l app.kubernetes.io/component=linstor-csi-node | tail -3
```

Die Ereignisse eines CSI-Node-Pods.

DAS SOLLTEST DU SEHEN

```
Warning  FailedMount  ...  MountVolume.SetUp failed for volume "registration-dir" :
  hostPath type check failed: /var/lib/kubelet/plugins_registry is not a directory
```

STOLPERFALLE

k0s legt das kubelet unter `/var/lib/k0s/kubelet` ab (Kapitel 4). Der CSI-Treiber sucht es unter `/var/lib/kubelet`. Piraeus beschreibt die Korrektur in seiner Anleitung für k0s: drei Host-Pfade, ein zusätzlicher Mount und der Registrierungspfad.

DATEI `Ergänzung für csiNode`

```
      containers:
        - name: linstor-csi
          volumeMounts:
            - name: publish-dir
              mountPath: /var/lib/k0s/kubelet
              mountPropagation: Bidirectional
        - name: csi-node-driver-registrar
          env:
            - name: KUBELET_REGISTRATION_PATH
              value: /var/lib/k0s/kubelet/plugins/linstor.csi.linbit.com/csi.sock
      volumes:
        - name: publish-dir
          hostPath: {path: /var/lib/k0s/kubelet}
        - name: registration-dir
          hostPath: {path: /var/lib/k0s/kubelet/plugins_registry}
        - name: plugin-dir
          hostPath: {path: /var/lib/k0s/kubelet/plugins/linstor.csi.linbit.com}
```

Der zusätzliche Mount unter `/var/lib/k0s/kubelet` muss im Container denselben Pfad haben wie auf dem Host. Der kubelet übergibt dem Treiber absolute Pfade, und die müssen im Container stimmen.

Dritter Versuch: die Neustart-Schleife

Jetzt laufen die CSI-Node-Pods, aber nie lange. Alle paar Sekunden werden sie gelöscht und neu erzeugt.

MAC — Schritt 5

```
kubectl -n piraeus-datastore get pods -l app.kubernetes.io/component=linstor-csi-node
kubectl get csinodes -o 'custom-columns=NODE:.metadata.name,DRIVERS:.spec.drivers[*].name'
```

Alter der Pods und welche Nodes den Treiber registriert haben.

DAS SOLLTEST DU SEHEN

```
NAME                    READY   STATUS            RESTARTS   AGE
linstor-csi-node-5n4fm  0/3     PodInitializing   0          2s
linstor-csi-node-8wgdg  0/3     PodInitializing   0          1s
linstor-csi-node-98gms  3/3     Running           0          3s
NODE     DRIVERS
k0s-w1   <none>
k0s-w2   linstor.csi.linbit.com
k0s-w3   <none>
```

Keine Abstürze, keine Fehler in den Logs. Die Ereignisse zeigen nur `Killing`.

Der Grund steht im Quellcode des Operators. Für jeden CSI-Node vergleicht er die Topologie-Schlüssel, die der Treiber in Kubernetes meldet, mit den Eigenschaften `Aux/topology/*` des passenden LINSTOR-Nodes. Weichen sie ab, löscht er den Pod, damit der Treiber sich neu registriert. Für Nodes, die der Operator selbst verwaltet, setzt er diese Eigenschaften selbst. Unsere Nodes verwaltet er nicht.

Der Treiber meldet den Schlüssel `linbit.com/hostname`. Den bekommen die LINSTOR-Nodes von Hand:

K0S-W1 Schritt 6

```
linstor node set-property k0s-w1 Aux/topology/linbit.com/hostname k0s-w1
linstor node set-property k0s-w2 Aux/topology/linbit.com/hostname k0s-w2
linstor node set-property k0s-w3 Aux/topology/linbit.com/hostname k0s-w3
```

Der Wert ist der Node-Name. Die Witness bekommt keinen Eintrag, auf ihr läuft kein CSI-Node.

DAS SOLLTEST DU SEHEN

```
NAME                       READY   STATUS    RESTARTS   AGE
linstor-csi-node-8t9g8     3/3     Running   0          93s
linstor-csi-node-lfwd7     3/3     Running   0          92s
linstor-csi-node-x6m4r     3/3     Running   0          93s
NODE     DRIVERS                    KEYS
k0s-w1   linstor.csi.linbit.com     [linbit.com/hostname]
k0s-w2   linstor.csi.linbit.com     [linbit.com/hostname]
k0s-w3   linstor.csi.linbit.com     [linbit.com/hostname]
```

STOLPERFALLE

Diese Eigenschaft gehört in die Automatisierung, die neue Nodes in LINSTOR anlegt. Ein neuer Worker ohne sie bringt sofort wieder die Neustart-Schleife, und zwar nur auf diesem einen Node.

Die vollständigen Werte

DATEI `lab/piraeus/linstor-cluster-values-linstor-1.33.yaml`

```yaml
# Piraeus nur als CSI-Treiber gegen den externen LINSTOR-Controller (VIP im Storage-Netz)
linstorCluster:
  externalController:
    url: http://10.10.0.100:3370
  # Kein Node trägt dieses Label: der Operator startet keine eigenen Satellites.
  nodeSelector:
    piraeus.io/managed-satellite: "true"
  highAvailabilityController:
    enabled: false
  affinityController:
    enabled: false
  nfsServer:
    enabled: false
  # Derselbe nodeSelector gilt sonst auch für die CSI-Pods. null entfernt ihn dort wieder.
```

```
# CSI 1.13 setzt LINSTOR 1.35 voraus, der Controller läuft mit 1.33: CSI auf 1.12.0 pinnen.
csiController:
  podTemplate:
    spec:
      nodeSelector:
        piraeus.io/managed-satellite: null
      containers:
        - name: linstor-csi
          image: quay.io/piraeusdatastore/piraeus-csi:v1.12.0
csiNode:
  podTemplate:
    spec:
      nodeSelector:
        piraeus.io/managed-satellite: null
      containers:
        - name: linstor-csi
          image: quay.io/piraeusdatastore/piraeus-csi:v1.12.0
          # k0s legt den kubelet-Zustand unter /var/lib/k0s/kubelet ab, nicht /var/lib/kubelet.
          volumeMounts:
            - name: publish-dir
              mountPath: /var/lib/k0s/kubelet
              mountPropagation: Bidirectional
        - name: csi-node-driver-registrar
          env:
            - name: KUBELET_REGISTRATION_PATH
              value: /var/lib/k0s/kubelet/plugins/linstor.csi.linbit.com/csi.sock
      volumes:
        - name: publish-dir
          hostPath:
            path: /var/lib/k0s/kubelet
        - name: registration-dir
          hostPath:
            path: /var/lib/k0s/kubelet/plugins_registry
        - name: plugin-dir
          hostPath:
            path: /var/lib/k0s/kubelet/plugins/linstor.csi.linbit.com
```

Alle drei Korrekturen zusammen.

MAC

Schritt 7

```
helm upgrade linstor-cluster piraeus-charts/linstor-cluster --version 1.2.0 \
  -n piraeus-datastore -f lab/piraeus/linstor-cluster-values-linstor-1.33.yaml
kubectl get linstorcluster -o \
  jsonpath='{range .items[].status.conditions[*]}{.type}={.status} {.message}{"\n"}{end}'
```

Prüft den Zustand der Cluster-Beschreibung.

DAS SOLLTEST DU SEHEN

```
Applied=True Resources applied
Available=True Controller 1.33.3 (API: 1.27.0, Git: 1ff3a1ea...) reachable at
'http://10.10.0.100:3370'
Configured=True Properties applied
```

Was der Operator am Controller verändert

`Configured=True Properties applied` ist wörtlich gemeint. Der Operator schreibt eigene Einstellungen in den externen Controller:

K0S-W1 Schritt 8

```
linstor controller list-properties | grep -E "piraeus|DrbdOptions"
```

Die globalen Eigenschaften des Controllers.

DAS SOLLTEST DU SEHEN

```
| Aux/piraeus.io/managed-by                          | piraeus-operator |
| DrbdOptions/AutoEvictAllowEviction                 | false            |
| DrbdOptions/Net/rr-conflict                        | retry-connect    |
| DrbdOptions/Resource/on-no-data-accessible         | suspend-io       |
| DrbdOptions/Resource/on-no-quorum                  | suspend-io       |
| DrbdOptions/Resource/on-suspended-primary-outdated | force-secondary  |
```

`Aux/piraeus.io/last-applied` listet, welche davon vom Operator stammen.

STOLPERFALLE

Diese Werte gelten für **alle** Ressourcen des Controllers, auch für solche außerhalb von Kubernetes. Wichtig ist vor allem `AutoEvictAllowEviction=false`: Die automatische Evakuierung ausgefallener Nodes ist damit abgeschaltet (Kapitel 10). Wer am Controller etwas anderes einstellen will, muss es über `linstorCluster.properties` in den Helm-Werten tun. Sonst setzt der Operator es beim nächsten Abgleich zurück. Im Lab war ein von Hand gesetztes `true` nach spätestens zwei Minuten wieder `false`.

8.4 Die StorageClass

DATEI `lab/k8s/storageclass.yaml`

```yaml
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: linstor-replicated
  annotations:
    storageclass.kubernetes.io/is-default-class: "true"
provisioner: linstor.csi.linbit.com
parameters:
  # Der CSI-Treiber schreibt diese Werte in die Resource-Group. Was hier fehlt,
  # setzt er auf seine Standardwerte zurück (1 Replikat, nur drbd+storage).
  linstor.csi.linbit.com/resourceGroup: rg-k8s
  linstor.csi.linbit.com/storagePool: pool_data
  linstor.csi.linbit.com/placementCount: "2"
  linstor.csi.linbit.com/replicasOnDifferent: Aux/zone
  linstor.csi.linbit.com/layerList: drbd writecache storage
  property.linstor.csi.linbit.com/Writecache/PoolName: pool_nvme
  property.linstor.csi.linbit.com/Writecache/Size: "5%"
  property.linstor.csi.linbit.com/DrbdOptions/Resource/quorum: majority
  property.linstor.csi.linbit.com/DrbdOptions/Resource/on-no-quorum: suspend-io
  property.linstor.csi.linbit.com/DrbdOptions/Resource/on-suspended-primary-outdated: force-secondary
  csi.storage.k8s.io/fstype: ext4
allowVolumeExpansion: true
```

```
volumeBindingMode: WaitForFirstConsumer
reclaimPolicy: Delete
```

`WaitForFirstConsumer` legt das Volume erst an, wenn klar ist, auf welchem Node der Pod landet. LINSTOR kann dann eine Kopie direkt dort platzieren. Die `property.linstor.csi.linbit.com/`-Parameter werden zu Eigenschaften der Resource-Group.

STOLPERFALLE

Im ersten Versuch stand in der StorageClass nur `resourceGroup: rg-k8s`, in der Annahme, die Vorlage aus Kapitel 7 liefere den Rest. Das Ergebnis war ein Volume mit **einem** Replikat ohne Writecache, und `rg-k8s` war danach umgeschrieben:

```
| rg-k8s | PlaceCount: 1                    | ...
|        | LayerStack: ['DRBD', 'STORAGE']  | ...
```

Der CSI-Treiber schreibt bei jedem neuen Volume die Parameter der StorageClass in die Resource-Group. Was fehlt, setzt er auf seine Standardwerte. **Die StorageClass ist die Quelle der Wahrheit**, nicht die Resource-Group. Prüfe nach jeder neuen StorageClass mit `linstor resource-group list`, was wirklich gilt.

8.5 Eine Datenbank auf repliziertem Speicher

DATEI `lab/k8s/postgres.yaml (Ausschnitt)`

```
  volumeClaimTemplates:
    - metadata: {name: data}
      spec:
        accessModes: [ReadWriteOnce]
        resources: {requests: {storage: 2Gi}}
```

Ein StatefulSet mit einer PostgreSQL-Instanz. Ohne `storageClassName` nimmt es die Standardklasse `linstor-replicated`. Das Passwort steht der Einfachheit halber im Klartext, nur fürs Lab.

MAC Schritt 9

```
kubectl apply -f lab/k8s/storageclass.yaml -f lab/k8s/postgres.yaml
kubectl -n demo rollout status sts/pg
kubectl -n demo get pvc,pod -o wide
```

Legt StorageClass und Datenbank an.

K0S-W1 Schritt 10

```
linstor resource list -r pvc-0d903f58-5691-4cda-bb3e-a155de0ced03
```

Den Namen der Ressource zeigt `kubectl -n demo get pvc` in der Spalte `VOLUME`.

DAS SOLLTEST DU SEHEN

```
| pvc-0d903f58-... | k0s-ctrl | DRBD,STORAGE            | Unused | Ok    | Diskless
| Yes  |
| pvc-0d903f58-... | k0s-w1   | DRBD,WRITECACHE,STORAGE | Unused | Ok    |
SyncTarget(36.32%) | Yes  |
| pvc-0d903f58-... | k0s-w2   | DRBD,WRITECACHE,STORAGE | InUse  | Ok    | UpToDate
| Yes  |
```

Zwei Zonen, Writecache, Tiebreaker auf der Witness. Der Pod läuft auf `k0s-w2`, dort ist das Volume `InUse`.

8.6 Der Umzug

MAC Schritt 11

```
kubectl -n demo exec pg-0 -- psql -U postgres \
  -c "create table notizen(id serial primary key, text text, zeit timestamptz default now());" \
  -c "insert into notizen(text) values ('geschrieben auf k0s-w2');"
kubectl cordon k0s-w2
kubectl -n demo delete pod pg-0
kubectl -n demo wait --for=condition=Ready pod/pg-0
kubectl -n demo get pod pg-0 -o wide
kubectl -n demo exec pg-0 -- psql -U postgres -c "select * from notizen;"
kubectl uncordon k0s-w2
```

Eine Zeile schreiben, `k0s-w2` sperren, den Pod löschen. Das StatefulSet startet ihn auf einem anderen Node neu.

DAS SOLLTEST DU SEHEN

```
NAME   READY   STATUS    RESTARTS   AGE   IP           NODE     ...
pg-0   1/1     Running   0          31s   10.244.0.6   k0s-w1   ...
 id |          text          |             zeit
----+------------------------+-------------------------------
  1 | geschrieben auf k0s-w2 | 2026-11-12 18:24:41.505541+00
```

Der Pod läuft auf `k0s-w1`, wo das zweite Replikat liegt, und die Zeile ist da. In LINSTOR ist jetzt `k0s-w1` `InUse`.

WAS DU GELERNT HAST

- Piraeus kann reiner CSI-Treiber für ein LINSTOR außerhalb von Kubernetes sein.
- Der `nodeSelector` des LinstorCluster gilt auch für die CSI-Pods. `null` in der `podTemplate` hebt ihn auf.
- Auf k0s braucht der CSI-Node die Pfade unter `/var/lib/k0s/kubelet`.
- Fremd verwaltete Nodes brauchen `Aux/topology/linbit.com/hostname`, sonst startet der Operator die CSI-Pods endlos neu.
- Die StorageClass überschreibt die Resource-Group. Alle Platzierungsregeln gehören in die StorageClass.
- CSI 1.13 braucht LINSTOR 1.35. Mit 1.33 bleibt CSI auf 1.12.

KAPITEL 9

LINSTOR und DRBD lesen

Du liest aus den Ausgaben von `linstor` und `drbdadm`, ob ein Volume gesund ist, wer schreibt, wer mitstimmt und über welches Netz die Daten fließen. Das ist das Handwerkszeug für alle Betriebskapitel.

Dauer: 30 Minuten **Voraussetzung:** Die PostgreSQL-Datenbank aus Kapitel 8

9.1 Zwei Sichten auf dasselbe Volume

LINSTOR und DRBD sehen ein Volume unterschiedlich:

- **LINSTOR** kennt den Soll-Zustand: welche Ressource auf welchem Node mit welchen Schichten liegen soll. Den Ist-Zustand fragt es bei den Satellites ab.
- **DRBD** kennt nur den Ist-Zustand auf diesem einen Node: Rolle, Platte, Verbindungen.

Wenn die beiden sich widersprechen, gilt DRBD. LINSTOR kann veraltet sein, etwa wenn ein Satellite nicht erreichbar ist.

Für die folgenden Befehle brauchst du den Namen der Ressource:

MAC Schritt 1

```
kubectl -n demo get pvc data-pg-0 -o jsonpath='{.spec.volumeName}{"\n"}'
```

Der PV-Name ist der Name der LINSTOR-Ressource.

DAS SOLLTEST DU SEHEN

```
pvc-0d903f58-5691-4cda-bb3e-a155de0ced03
```

Im Folgenden kurz `pvc-0d90...`.

9.2 Die Sicht von LINSTOR

K0S-W1 Schritt 2

```
linstor volume list -r pvc-0d903f58-5691-4cda-bb3e-a155de0ced03
```

Eine Zeile pro Kopie.

DAS SOLLTEST DU SEHEN

```
| Node     | StoragePool          | MinorNr | DeviceName    | Allocated | InUse  |
State | Repl           |
| k0s-ctrl | DfltDisklessStorPool |    1001 | /dev/drbd1001 |           | Unused |
Diskless | Established(2) |
| k0s-w1   | pool_data            |    1001 | /dev/drbd1001 |  2.11 GiB | InUse  |
UpToDate | Established(2) |
| k0s-w2   | pool_data            |    1001 | /dev/drbd1001 |  2.11 GiB | Unused |
UpToDate | Established(2) |
```

Spalte	Was sie sagt
`MinorNr`, `DeviceName`	Das Gerät auf jedem Node, hier überall `/dev/drbd1001`.
`Allocated`	Belegter Platz. 2 GiB plus DRBD-Metadaten plus Writecache-Anteil.
`InUse`	Wer das Gerät offen hat. Genau ein Node, solange der Pod läuft.
`State`	`UpToDate` heißt: aktuelle Daten. `Diskless` heißt: keine eigenen Daten.
`Repl`	Zustand der Replikation zu den anderen Kopien. `Established(2)` heißt: zu beiden verbunden.

K0S-W1 Schritt 3

```
linstor resource-definition list-properties pvc-0d903f58-5691-4cda-bb3e-a155de0ced03
```

Eigenschaften, die der CSI-Treiber gesetzt hat.

DAS SOLLTEST DU SEHEN

```
| Aux/csi-provisioning-completed-by    | linstor-csi/v1.12.0 |
| Aux/csi.storage.k8s.io/pvc/name      | data-pg-0           |
| Aux/csi.storage.k8s.io/pvc/namespace | demo                |
| DrbdOptions/Resource/quorum          | majority            |
| DrbdOptions/auto-verify-alg          | crct10dif           |
```

Über `Aux/csi.storage.k8s.io/pvc/name` findest du zu jeder LINSTOR-Ressource das PVC, ohne Kubernetes zu fragen.

9.3 Die Sicht von DRBD

K0S-W1 Schritt 4

```
sudo drbdadm status pvc-0d903f58-5691-4cda-bb3e-a155de0ced03
```

Der wichtigste Befehl im Betrieb. Ohne Ressourcennamen zeigt er alle Ressourcen.

DAS SOLLTEST DU SEHEN

```
pvc-0d903f58-5691-4cda-bb3e-a155de0ced03 role:Primary
  disk:UpToDate open:yes
  k0s-ctrl role:Secondary
    peer-disk:Diskless peer-client:yes
  k0s-w2 role:Secondary
    peer-disk:UpToDate
```

So liest du das:

Zeile	Bedeutung
`role:Primary`	Dieser Node darf schreiben. Es gibt immer höchstens einen.
`disk:UpToDate open:yes`	Die lokale Kopie ist aktuell, und jemand hat das Gerät geöffnet, hier das Dateisystem des Pods.
`k0s-ctrl role:Secondary`	Verbindung zur Witness, die nur liest und mitstimmt.
`peer-disk:Diskless`	Die Gegenseite hat keine Daten. Bei `k0s-w2` steht `UpToDate`.

Ein gesundes Volume sieht immer so aus. Jede Abweichung ist ein Hinweis:

Du siehst	Das heißt
`connection:Connecting`	Die Gegenseite ist nicht erreichbar. Netz oder Node prüfen.
`connection:StandAlone`	DRBD versucht nicht mehr, sich zu verbinden. Fast immer nach einem Split-Brain (Kapitel 11).
`disk:Outdated`	Diese Kopie weiß, dass sie veraltet ist. Sie wird nie Primary.
`disk:Inconsistent`	Wird gerade synchronisiert und ist unvollständig.
`replication:SyncSource` / `SyncTarget`	Resync läuft. `done:` zeigt den Fortschritt.
`quorum:no`	Keine Mehrheit erreichbar. Je nach Einstellung hängen die Schreibvorgänge oder schlagen fehl.
`suspended:quorum`	Der Primary hat die Ein- und Ausgabe angehalten, weil das Quorum fehlt.

9.4 Mehr Details

K0S-W1 — Schritt 5

```
sudo drbdsetup status --verbose --statistics pvc-0d903f58-5691-4cda-bb3e-a155de0ced03
```

Dieselbe Ressource mit allen Zählern.

DAS SOLLTEST DU SEHEN

```
pvc-0d903f58-... node-id:1 role:Primary suspended:no
  volume:0 minor:1001 disk:UpToDate
      backing_dev:/dev/mapper/linstor_writecache_pvc-0d903f58-..._00000
      quorum:yes open:yes
      size:2100760 read:6596 written:2174708 al-writes:11 bm-writes:250 upper-pending:0
      lower-pending:0 al-suspended:no blocked:no
  k0s-ctrl node-id:2 connection:Connected role:Secondary tls:no congested:no
    volume:0 replication:Established peer-disk:Diskless peer-client:yes
        received:0 sent:0 out-of-sync:0 pending:0 unacked:0
  k0s-w2 node-id:0 connection:Connected role:Secondary tls:no congested:no
    volume:0 replication:Established peer-disk:UpToDate resync-suspended:no
        received:58844 sent:2936 out-of-sync:0 pending:0 unacked:0
```

Drei Werte sind im Betrieb besonders nützlich:

- `backing_dev` zeigt, worauf DRBD schreibt. Hier das Writecache-Gerät, darunter liegen die beiden LVs.
- `quorum:yes` bestätigt, dass dieser Node eine Mehrheit sieht.
- `out-of-sync` zählt die KiB, die der Gegenseite fehlen. Im Normalbetrieb ist der Wert 0. Steigt er bei einer verbundenen Gegenseite, kommt das Netz nicht hinterher.

K0S-W1 Schritt 6

```
sudo drbdsetup show pvc-0d903f58-5691-4cda-bb3e-a155de0ced03
```

Die wirksame Konfiguration, wie DRBD sie gerade benutzt.

DAS SOLLTEST DU SEHEN

```
resource "pvc-0d903f58-..." {
    options {
        on-no-data-accessible suspend-io;
        quorum            majority;
        on-suspended-primary-outdated force-secondary;
    }
    ...
    connection {
        _peer_node_id 2;
        path {
            _this_host ipv4 172.16.0.186:7001;
            _remote_host ipv4 172.16.0.176:7001;
        }
        net {
            cram-hmac-alg     "sha1";
            shared-secret     "<geschwärzt>";
            ...
```

`on-no-quorum` taucht nicht auf, weil `suspend-io` der Standardwert ist. `drbdsetup show` zeigt nur Abweichungen.

STOLPERFALLE

`drbdsetup show` gibt das gemeinsame Geheimnis jeder Verbindung im Klartext aus. Kopiere die Ausgabe nie ungefiltert in ein Ticket oder einen Chat.

9.5 Die Sicht der Witness

K0S-CTRL Schritt 7

```
sudo drbdadm status pvc-0d903f58-5691-4cda-bb3e-a155de0ced03
```

Dieselbe Ressource von der Witness aus.

DAS SOLLTEST DU SEHEN

```
pvc-0d903f58-... role:Secondary
  disk:Diskless client:yes open:no
  k0s-w1 role:Primary
    peer-disk:UpToDate
  k0s-w2 role:Secondary
    peer-disk:UpToDate
```

Von hier aus siehst du auf einen Blick, wer Primary ist. Das ist praktisch, wenn du nicht weißt, auf welchem Worker der Pod läuft.

9.6 Fehlerberichte des Controllers

Wenn LINSTOR eine Aktion nicht ausführen kann, legt es einen Fehlerbericht an:

K0S-W1 Schritt 8

```
linstor error-reports list
```

Listet alle Berichte. Im Moment ist die Liste leer.

Mit `linstor error-reports show <Id>` liest du einen Bericht im Detail. Er enthält den Stacktrace und meist den genauen Befehl auf dem Satellite, der gescheitert ist. Bei Problemen mit LINSTOR ist das die zweite Anlaufstelle nach `journalctl -u linstor-satellite` auf dem betroffenen Node.

WAS DU GELERNT HAST

- LINSTOR kennt den Soll-Zustand, DRBD den Ist-Zustand. Im Zweifel gilt DRBD.
- `drbdadm status` ist der wichtigste Befehl im Betrieb.
- `out-of-sync`, `quorum` und `backing_dev` aus `drbdsetup status --verbose --statistics` zeigen Netz, Mehrheit und Gerätestapel.
- `drbdsetup show` zeigt die wirksame Konfiguration samt Geheimnis.
- Fehlerberichte und das Satellite-Journal erklären, warum LINSTOR etwas nicht getan hat.

KAPITEL 10

Betrieb: Wenn ein Node ausfällt

Du wartest einen Node geplant ohne Ausfall, weißt, was bei einem Absturz passiert, und holst eine Datenbank zurück, deren Node nicht wiederkommt.

Dauer: 60 Minuten **Voraussetzung:** Den Cluster aus Kapitel 8 mit der PostgreSQL-Datenbank

10.1 Den Ausfall messbar machen

Ein Client fügt jede Sekunde eine Zeile in die Datenbank ein und protokolliert Erfolg oder Fehler. So siehst du später auf die Sekunde, wie lange die Datenbank weg war.

DATEI `lab/k8s/pg-client.yaml (Ausschnitt)`

```
          while true; do
            if timeout 3 psql -h pg -U postgres -qc "insert into puls default values" 2>/dev/null; then
              echo "$(date +%T) ok"
            else
              echo "$(date +%T) FEHLER"
            fi
            sleep 1
          done
```

Dazu gehört ein Service `pg`, damit der Client die Datenbank unabhängig vom Node findet. Der Client ist ein Deployment, damit er einen Drain übersteht (Kapitel 15). Bei den Messungen in diesem Kapitel lief er als einzelner Pod auf `k0s-w3`.

MAC Schritt 1

```
kubectl apply -f lab/k8s/pg-client.yaml
kubectl -n demo logs deploy/pg-client --tail=3
```

Startet den Client.

10.2 Geplante Wartung

Ein Node soll neu starten, etwa für ein Kernel-Update. Im Lab trifft es `k0s-w3`, auf dem gerade der LINSTOR-Controller läuft.

MAC Schritt 2

```
kubectl drain k0s-w3 --ignore-daemonsets --delete-emptydir-data
```

Verschiebt alle Pods außer den DaemonSets auf andere Nodes und sperrt den Node für neue Pods.

K0S-W3 Schritt 3

```
sudo drbd-reactorctl status linstor_db | head -2
sudo drbd-reactorctl evict linstor_db
```

Läuft der Controller hier, gibt der Node ihn geplant ab.

DAS SOLLTEST DU SEHEN

```
Node 'k0s-w1' took over
```

STOLPERFALLE

Direkt nach `evict` meldet `linstor` noch einige Sekunden `Connection refused` von der VIP. Der Controller startet auf dem neuen Node gerade, Java braucht dafür ein paar Sekunden. Warte, bis `linstor node list` antwortet, bevor du den Node neu startest.

K0S-W3 Schritt 4

```
sudo systemctl reboot
```

Der eigentliche Neustart.

Während `k0s-w3` weg ist, zeigt `drbdadm status linstor_db` auf den anderen Nodes `k0s-w3 connection:Connecting`. Nach etwa 30 Sekunden ist der Node zurück:

K0S-W1 Schritt 5

```
linstor node list
linstor resource list --faulty
```

`--faulty` zeigt nur Ressourcen mit Problemen.

DAS SOLLTEST DU SEHEN

```
| k0s-w3   | SATELLITE | 10.10.0.13:3366 (PLAIN)   | OFFLINE |
| linstor_db   | k0s-w3 | DRBD,STORAGE |       |       | Unknown | Yes  |
```

Ein paar Sekunden nach dem Neustart steht der Node noch auf `OFFLINE`, obwohl DRBD schon wieder verbunden ist. Der Controller verbindet sich in Intervallen neu. Kurz darauf:

```
| k0s-w3   | SATELLITE | 10.10.0.13:3366 (PLAIN)   | Online |
```

und `resource list --faulty` ist leer.

MAC Schritt 6

```
kubectl uncordon k0s-w3
```

Gibt den Node wieder frei.

Drain, Abgabe des Controllers, Neustart, Resync und Freigabe dauerten im Lab zusammen etwa zwei Minuten. Die Datenbank war in dieser Zeit durchgehend erreichbar.

WARUM DAS WICHTIG IST

Die Reihenfolge ist wichtig: erst die Pods weg, dann der Controller, dann der Neustart. Würdest du einen Node mit aktivem Controller einfach neu starten, ginge der Controller trotzdem auf einen anderen Node über, aber ungeplant, mit einem Timeout statt in Sekunden.

10.3 Ein Absturz

Jetzt der unschöne Fall: `k0s-w1` stürzt ab. Auf ihm laufen sowohl PostgreSQL als auch der LINSTOR-Controller. Ein Absturz lässt sich mit SysRq nachstellen. `b` startet den Kernel sofort neu, ohne Sync und ohne Herunterfahren, wie der Reset-Knopf.

K0S-W1 — Schritt 7

```
echo 1 | sudo tee /proc/sys/kernel/sysrq
echo b | sudo tee /proc/sysrq-trigger
```

Die SSH-Verbindung reißt sofort ab.

STOLPERFALLE

SysRq ist ein Notfallwerkzeug. Führe diese Befehle nur im Lab aus und nur auf dem Node, den du meinst. Es gibt keine Rückfrage.

Was dann geschah, in Zahlen aus dem Lab:

Zeitpunkt	Ereignis
0 s	Absturz von `k0s-w1`. Der Client meldet ab jetzt `FEHLER`.
38 s	LINSTOR-Controller auf `k0s-w2` gestartet und über die VIP erreichbar.
ca. 30 s	`k0s-w1` bootet wieder, DRBD verbindet sich und gleicht ab.
bis 120 s	`pg-0` bleibt auf `k0s-w1` und wartet. Fehler: `driver name linstor.csi.linbit.com not found in the list of registered CSI drivers`.
ca. 122 s	CSI-Node-Pod auf `k0s-w1` läuft, das Volume wird gemountet, PostgreSQL startet. Der Client meldet wieder `ok`.

MAC — Schritt 8

```
kubectl -n demo describe pod pg-0 | tail -8
```

Die Ereignisse am Pod nach dem Neustart des Nodes.

DAS SOLLTEST DU SEHEN

```
Warning  NodeNotReady   node-controller  Node is not ready
Warning  FailedMount    kubelet          MountVolume.MountDevice failed ... driver name
         linstor.csi.linbit.com not found in the list of registered CSI drivers
Normal   SandboxChanged kubelet          Pod sandbox changed, it will be killed and re-
created.
Normal   Started        kubelet          spec.containers{postgres}: Container started
```

Zwei Minuten Ausfall, kein Datenverlust, keine Handarbeit. Kubernetes hat den Pod nicht verschoben, weil der Node schnell genug zurück war. Die meiste Zeit verging mit dem Warten auf den CSI-Treiber nach dem Neustart.

10.4 Ein Node, der nicht zurückkommt

Der schwerste Fall: `k0s-w1` fällt aus und bleibt aus, etwa wegen eines defekten Netzteils. Im Lab schaltet SysRq `o` die VM sofort aus.

K0S-W1 Schritt 9

```
sudo sysctl -w kernel.sysrq=1
sudo systemd-run --on-active=2 sh -c "echo o > /proc/sysrq-trigger"
```

Schaltet die VM in zwei Sekunden hart ab. Die Verzögerung sorgt dafür, dass die SSH-Sitzung den Befehl noch sauber übergibt. Wieder einschalten kannst du sie nur über Proxmox.

MAC Schritt 10

```
kubectl get nodes
kubectl -n demo get pod -o wide
```

Nach einer Minute und nach sechs Minuten.

DAS SOLLTEST DU SEHEN

```
## nach 1 Minute
k0s-w1   NotReady   ...
pg-0        1/1   Running       ...   k0s-w1
## nach 6 Minuten
k0s-w1   NotReady   ...
pg-0        1/1   Terminating   ...   k0s-w1
```

Der Client meldet die ganze Zeit `FEHLER`.

Nach fünf Minuten markiert Kubernetes den Pod zum Löschen. Löschen kann ihn aber nur das kubelet auf `k0s-w1`, und das ist aus. Der Pod bleibt in `Terminating`, das StatefulSet startet keinen Ersatz. Eine zweite `pg-0` darf es nie geben. Kubernetes kann nicht unterscheiden, ob der Node tot ist oder nur nicht erreichbar und die alte Instanz noch schreibt.

WARUM DAS WICHTIG IST

Auf der Speicherseite ist alles bereit: `k0s-w2` hat eine aktuelle Kopie und mit der Witness die Mehrheit. Was fehlt, ist die Bestätigung, dass `k0s-w1` wirklich nicht mehr schreibt. Diese Bestätigung muss ein Mensch geben. Der Piraeus-HA-Controller könnte das automatisch tun. In diesem Buch ist er bewusst abgeschaltet, also braucht es jemanden, der den Knopf drückt.

Der Knopf ist das **Out-of-Service-Taint**. Du setzt es erst, wenn du sicher weißt, dass der Node aus ist, etwa weil du ihn in Proxmox ausgeschaltet siehst:

MAC — Schritt 11

```
kubectl taint node k0s-w1 node.kubernetes.io/out-of-service=nodeshutdown:NoExecute
```

Kubernetes löscht daraufhin alle Pods des Nodes zwangsweise und gibt ihre Volumes frei.

DAS SOLLTEST DU SEHEN

```
pg-0   1/1   Running   0   11s   10.244.1.10   k0s-w3
Normal  Scheduled              Successfully assigned demo/pg-0 to k0s-w3
Normal  SuccessfulAttachVolume  AttachVolume.Attach succeeded for volume
"pvc-0d903f58-..."
Normal  Started                spec.containers{postgres}: Container started
```

30 Sekunden nach dem Taint lief die Datenbank wieder, mit allen Daten.

STOLPERFALLE

Setze das Taint **nie** auf einen Node, der vielleicht noch läuft. Lebt die alte Instanz noch und erreicht ihr Storage-Netz, schützt dich DRBD-Quorum vor doppeltem Schreiben, aber die Anwendung auf dem alten Node bekommt I/O-Fehler (Kapitel 11, Versuch D). Erst ausschalten, dann das Taint setzen.

Ein Blick auf LINSTOR zeigt eine Überraschung:

K0S-W2 — Schritt 12

```
linstor resource list -r pvc-0d903f58-5691-4cda-bb3e-a155de0ced03
```

Wo die Kopien jetzt liegen.

DAS SOLLTEST DU SEHEN

```
| k0s-ctrl | DRBD,STORAGE            | Unused | Connecting(k0s-w1) | Diskless | Yes  |
| k0s-w1   | DRBD,WRITECACHE,STORAGE |        |                    |  Unknown | Yes  |
| k0s-w2   | DRBD,WRITECACHE,STORAGE | Unused | Connecting(k0s-w1) | UpToDate | Yes  |
| k0s-w3   | DRBD,STORAGE            | InUse  | Connecting(k0s-w1) | Diskless | Yes  |
```

Der Scheduler hat `pg-0` nach `k0s-w3` gesetzt, wo keine Kopie liegt. LINSTOR hat dort automatisch einen disklosen Zugang angelegt. Die Datenbank liest und schreibt übers Netz auf `k0s-w2`. Das funktioniert, kostet aber Latenz. Und es gibt gerade nur **eine** aktuelle Kopie der Daten.

10.5 Der Node kommt zurück

Nach der Reparatur startest du `k0s-w1` in Proxmox. Die Kopie auf `k0s-w1` ist veraltet. DRBD erkennt das und gleicht sie ab, ohne dass du etwas tun musst:

K0S-W2 Schritt 13

```
sudo drbdadm status pvc-0d903f58-5691-4cda-bb3e-a155de0ced03
```

Eine Minute nach dem Start von `k0s-w1`.

DAS SOLLTEST DU SEHEN

```
pvc-0d903f58-... role:Secondary
  disk:UpToDate open:no
  k0s-ctrl role:Secondary
    peer-disk:Diskless peer-client:yes
  k0s-w1 role:Secondary
    peer-disk:UpToDate
  k0s-w3 role:Primary
    peer-disk:Diskless peer-client:yes
```

Der Resync war bereits durch, als SSH wieder ging. Er überträgt nur die Blöcke, die sich in der Zwischenzeit geändert haben.

Kubernetes meldet den Node als `Ready`, aber auf ihm läuft fast nichts:

MAC Schritt 14

```
kubectl get pods -n piraeus-datastore -o wide --field-selector spec.nodeName=k0s-w1
```

Welche Piraeus-Pods auf `k0s-w1` laufen.

DAS SOLLTEST DU SEHEN

```
No resources found in piraeus-datastore namespace.
```

STOLPERFALLE

Das Out-of-Service-Taint hat den Effekt `NoExecute` und verdrängt **alle** Pods ohne passende Toleranz, auch den CSI-Node-DaemonSet. Solange es gesetzt ist, kann auf dem Node kein LINSTOR-Volume gemountet werden. Nimm es herunter, sobald der Node repariert ist.

MAC Schritt 15

```
kubectl taint node k0s-w1 node.kubernetes.io/out-of-service=nodeshutdown:NoExecute-
kubectl get csinode k0s-w1 -o jsonpath='{.spec.drivers[*].name}{"\n"}'
```

Das Minus am Ende entfernt das Taint. Danach registriert sich der CSI-Treiber wieder.

DAS SOLLTEST DU SEHEN

```
node/k0s-w1 untainted
linstor.csi.linbit.com
```

Bleibt die Datenbank auf `k0s-w3` ohne eigene Kopie. Das ist kein Fehler, aber jeder Lese- und Schreibzugriff geht übers Netz. Du holst sie zurück auf einen Node mit Daten:

MAC Schritt 16

```
kubectl cordon k0s-w3
kubectl -n demo delete pod pg-0
kubectl -n demo wait --for=condition=Ready pod/pg-0
kubectl uncordon k0s-w3
```

Mit `k0s-w3` gesperrt bleiben nur Nodes mit Kopie, und der Scheduler muss einen davon nehmen.

K0S-W2 Schritt 17

```
linstor resource list -r pvc-0d903f58-5691-4cda-bb3e-a155de0ced03
```

Der Zustand danach.

DAS SOLLTEST DU SEHEN

```
| k0s-ctrl | DRBD,STORAGE            | Unused | Ok    | Diskless | Yes  |
| k0s-w1   | DRBD,WRITECACHE,STORAGE | InUse  | Ok    | UpToDate | Yes  |
| k0s-w2   | DRBD,WRITECACHE,STORAGE | Unused | Ok    | UpToDate | Yes  |
```

Den disklosen Zugang auf `k0s-w3` hat LINSTOR beim Aushängen selbst entfernt. Der Umzug kostete die Datenbank etwa 6 Sekunden.

WARUM LINSTOR NICHT VON SELBST NEUE KOPIEN ANLEGT

LINSTOR kennt eine automatische Evakuierung: Ist ein Node länger als `DrbdOptions/AutoEvictAfterTime` offline (Standard 60 Minuten), legt es seine Kopien auf anderen Nodes neu an. In diesem Cluster ist das abgeschaltet, und zwar vom Piraeus-Operator. Er setzt am Controller `DrbdOptions/AutoEvictAllowEviction=false` (Kapitel 8). Ein ausgefallener Node bleibt also ausgefallen, bis du ihn reparierst oder seine Kopien bewusst verschiebst (Kapitel 14). Abgewartet wurde das im Lab nicht.

WAS DU GELERNT HAST

- Geplante Wartung: drain, Controller mit `evict` abgeben, neu starten, warten bis `--faulty` leer ist, uncordon.
- Nach einem Absturz mit schnellem Neustart heilt sich alles selbst. Die meiste Zeit kostet der CSI-Treiber.
- Bleibt ein Node aus, hängt ein StatefulSet-Pod dauerhaft in `Terminating`.
- Das Out-of-Service-Taint ist die menschliche Bestätigung, dass ein Node tot ist. Danach läuft der Pod in Sekunden woanders.
- Ein Pod kann auf einem Node ohne Kopie landen. LINSTOR hängt das Volume dann diskless an.

KAPITEL 11

Betrieb: Netzausfälle, Quorum und Split-Brain

Du weißt aus eigener Anschauung, was DRBD tut, wenn Verbindungen reißen: wann weitergeschrieben wird, wann alles anhält, wann eine Seite zwangsweise zurücktritt. Und du löst ein echtes Split-Brain auf, ohne die falschen Daten zu behalten.

Dauer: 60 Minuten **Voraussetzung:** Den Cluster aus Kapitel 8. Alle Versuche laufen an eigenen Test-Ressourcen.

11.1 Der Versuchsaufbau

Für die Versuche gibt es eine eigene Ressource aus der Vorlage `rg-k8s`. Auf dem Primary läuft ein Schreiber, der jede Sekunde einen Zeitstempel anhängt und sofort `sync` aufruft. An Lücken in dieser Datei siehst du später genau, wann nicht geschrieben werden konnte.

K0S-W1 Schritt 1

```
linstor resource-group spawn-resources rg-k8s qtest 300M
linstor resource list -r qtest
```

Zwei Replikate, Tiebreaker auf der Witness.

DAS SOLLTEST DU SEHEN

```
| qtest        | k0s-ctrl | DRBD,STORAGE            | Unused | Ok    | Diskless | Yes  |
| qtest        | k0s-w2   | DRBD,WRITECACHE,STORAGE | Unused | Ok    | UpToDate | Yes  |
| qtest        | k0s-w3   | DRBD,WRITECACHE,STORAGE | Unused | Ok    | UpToDate | Yes  |
```

K0S-W3 Schritt 2

```
sudo mkfs.ext4 /dev/drbd1002
sudo mkdir -p /mnt/qtest
sudo mount /dev/drbd1002 /mnt/qtest
sudo systemd-run --unit=schreiber bash -c \
  'while true; do date +%T.%N | cut -c1-12 >> /mnt/qtest/log; sync /mnt/qtest/log; sleep 1; done'
```

Die Gerätenummer zeigt `linstor volume list -r qtest`. Beim Mounten wird `k0s-w3` automatisch Primary. `systemd-run` startet den Schreiber als eigenen Dienst, der weiterläuft, wenn deine SSH-Sitzung endet.

Netzausfälle stellst du nach, indem du `ens19` abschaltest. SSH läuft weiter über `eth0`. Damit das Netz auch dann zurückkommt, wenn etwas hängt, plant ein zweiter Dienst die Rückkehr gleich mit ein:

K0S-W2 Schritt 3

```
sudo systemd-run --unit=netzweg bash -c \
  'ip link set ens19 down; sleep 60; ip link set ens19 up; netplan apply'
```

60 Sekunden ohne Storage-Netz.

11.2 Versuch A: Ein Secondary verliert das Netz

Wie oben, auf `k0s-w2`. Nach 15 Sekunden:

K0S-W3 Schritt 4

```
sudo drbdadm status qtest
```

Die Sicht des Primary.

DAS SOLLTEST DU SEHEN

```
qtest role:Primary
  disk:UpToDate open:yes
  k0s-ctrl role:Secondary
    peer-disk:Diskless peer-client:yes
  k0s-w2 connection:Connecting
```

Auf `k0s-w2` steht `disk:Outdated`. w2 weiß, dass es den Anschluss verloren hat, und wird mit diesem Stand nie Primary.

Der Primary hat mit sich selbst und der Witness zwei von drei Stimmen, also Quorum. Er schreibt weiter. Nach der Rückkehr des Netzes synchronisiert DRBD die verpassten Blöcke von selbst, ohne dass du etwas tun musst.

Die Lücken in der Datei des Schreibers zeigen aber etwas, das man leicht übersieht:

MAC Schritt 5

```
ssh k0s-w3 'cat /mnt/qtest/log' | awk 'NR>1{split(p,a,":");split($0,b,":");
  d=(b[2]*60+b[3])-(a[2]*60+a[3]); if(d>2.5) print "Lücke", p, "->", $0} {p=$0}'
```

Sucht Abstände über 2,5 Sekunden.

DAS SOLLTEST DU SEHEN

```
Lücke 19:08:11.460 -> 19:08:22.181
```

WARUM DAS WICHTIG IST

Fällt ein Partner weg, frieren die Schreibzugriffe für etwa 10 Sekunden ein. DRBD bestätigt einen Schreibvorgang erst, wenn der Partner ihn hat. Antwortet der Partner nicht mehr, wartet DRBD, bis das Timeout der Verbindung abläuft, und schreibt erst dann allein weiter. Für eine Datenbank sind das 10 Sekunden ohne Commit. Wer über „kurze Hänger“ bei Netzproblemen klagt, sieht genau das.

11.3 Versuch B: Der Primary verliert das Storage-Netz

Jetzt fällt `ens19` auf `k0s-w3` aus, dem Primary. Die Witness erreicht w3 weiter über das Management-Netz. Damit sieht jede Seite außer sich selbst noch die Witness, beide hätten rechnerisch zwei von drei Stimmen.

DAS SOLLTEST DU SEHEN

```
## w3 (Primary)
qtest role:Primary
  disk:UpToDate open:yes
  k0s-w2 connection:Connecting
## w2
qtest role:Secondary
  disk:Outdated open:no
## Witness
qtest role:Secondary
  disk:Diskless client:yes open:no
  k0s-w2 role:Secondary
    peer-disk:Outdated
  k0s-w3 role:Primary
    peer-disk:UpToDate
```

Kein Split-Brain. Der Primary bleibt Primary, w2 setzt sich auf `Outdated`. Die Witness ist mit beiden verbunden und weiß, wer Primary ist. Eine Seite, die nicht Primary ist, kann über sie keine Mehrheit gegen den Primary bilden. Auch hier: eine Lücke von etwa 10 Sekunden, danach Resync von selbst.

WARUM DAS WICHTIG IST

Das ist der wahrscheinlichste Fall in dieser Architektur: Das Storage-VLAN hat ein Problem, das Management-Netz nicht. Die Witness im Management-Netz verhindert, dass daraus ein Split-Brain wird.

11.4 Versuch C: Der Primary verliert alles

Jetzt ist `k0s-w3` komplett isoliert: kein Storage-Netz und auch keine Verbindung zur Witness. Die Witness-Verbindung kappt eine Firewall-Regel auf den DRBD-Ports:

K0S-W3 Schritt 6

```
sudo systemd-run --unit=isoliert bash -c '
  iptables -I OUTPUT -d 172.16.0.176 -p tcp --dport 7000:7999 -j DROP
  iptables -I INPUT  -s 172.16.0.176 -p tcp --sport 7000:7999 -j DROP
  ip link set ens19 down; sleep 60; ip link set ens19 up; netplan apply
  iptables -D OUTPUT -d 172.16.0.176 -p tcp --dport 7000:7999 -j DROP
  iptables -D INPUT  -s 172.16.0.176 -p tcp --sport 7000:7999 -j DROP'
sleep 25
sudo drbdsetup status qtest --verbose | head -5
```

60 Sekunden allein.

DAS SOLLTEST DU SEHEN

```
qtest node-id:1 role:Primary suspended:quorum force-io-failures:no
  volume:0 minor:1002 disk:UpToDate
      backing_dev:/dev/mapper/linstor_writecache_qtest_00000 quorum:no open:yes
      blocked:upper
```

`quorum:no`, `suspended:quorum`, `blocked:upper`: DRBD hat alle Schreibvorgänge angehalten. Das ist die Einstellung `on-no-quorum=suspend-io` aus der Vorlage. Selbst `tail` auf das Dateisystem hängt jetzt. Nach der Rückkehr des Netzes läuft alles von selbst weiter:

DAS SOLLTEST DU SEHEN

```
Lücke 19:11:34.582 -> 19:12:45.308
```

71 Sekunden Stillstand, kein Datenverlust, keine Fehlermeldung in der Anwendung.

11.5 Versuch D: Die andere Seite übernimmt

Was, wenn während der Isolation jemand die andere Seite benutzt? Genau das tut Kubernetes, wenn es den Pod auf einen anderen Node verschiebt. `k0s-w2` hat mit der Witness zwei von drei Stimmen und darf Primary werden.

K0S-W2 Schritt 7

```
sudo mount /dev/drbd1002 /mnt/qtest
echo "w2 hat übernommen $(date +%T)" | sudo tee -a /mnt/qtest/log
sudo umount /mnt/qtest
```

Während `k0s-w3` isoliert ist und seine Schreibvorgänge angehalten hat.

Kommt `k0s-w3` zurück, hat es einen veralteten Stand und hängende Schreibvorgänge.

K0S-W3 Schritt 8

```
sudo drbdsetup status qtest --verbose | head -4
tail -2 /mnt/qtest/log
```

Der zurückgekehrte alte Primary.

DAS SOLLTEST DU SEHEN

```
qtest node-id:1 role:Secondary suspended:no force-io-failures:yes
  volume:0 minor:1002 disk:UpToDate
      quorum:yes open:yes
tail: cannot open '/mnt/qtest/log' for reading: Input/output error
```

`on-suspended-primary-outdated=force-secondary` hat gegriffen: DRBD hat den alten Primary zum Secondary gemacht, seinen Stand mit den Daten von w2 überschrieben und lässt alle Zugriffe des alten Dateisystems mit Fehlern scheitern (`force-io-failures:yes`). Die Anwendung auf w3 bekommt I/O-Fehler statt veralteter Daten.

K0S-W3 Schritt 9

```
sudo systemctl stop schreiber
sudo umount /mnt/qtest
```

Aufräumen auf dem alten Primary. Erst danach ist das Gerät wieder normal nutzbar.

DAS SOLLTEST DU SEHEN

Auf `k0s-w2` gemountet:

```
19:13:23.181
19:13:24.214
w2 hat übernommen 19:13:45
```

Alles bis zur Isolation, dann der Eintrag von w2. Was w3 während der Isolation schreiben wollte, ist nie angekommen. Es gibt kein Split-Brain.

WARUM DAS WICHTIG IST

Das ist der Grund für die drei Einstellungen der Vorlage: `quorum=majority` verhindert, dass die Minderheit schreibt. `suspend-io` lässt die Minderheit warten, statt Fehler zu werfen. `force-secondary` räumt auf, wenn die Mehrheit inzwischen weitergemacht hat. Zusammen machen sie ein Split-Brain unmöglich, solange es eine dritte Stimme gibt.

11.6 Versuch E: Ein echtes Split-Brain

Ohne Quorum gibt es diesen Schutz nicht. Um zu sehen, wie ein Split-Brain aussieht und wie du es auflöst, bekommt eine Test-Ressource bewusst kein Quorum:

K0S-W1 Schritt 10

```
linstor resource-group spawn-resources rg-k8s sbtest 200M
linstor resource-definition drbd-options --quorum off sbtest
linstor resource list -r sbtest
```

Mit `quorum off` entfernt LINSTOR den Tiebreaker, er wäre nutzlos.

DAS SOLLTEST DU SEHEN

```
| sbtest       | k0s-w2 | DRBD,WRITECACHE,STORAGE | Unused | Ok    | UpToDate | Yes  |
| sbtest       | k0s-w3 | DRBD,WRITECACHE,STORAGE | Unused | Ok    | UpToDate | Yes  |
```

Der Ablauf: auf w2 ein Dateisystem mit dem Inhalt `basis` anlegen, das Storage-Netz von w3 für 60 Sekunden trennen, während der Trennung auf **beiden** Seiten mounten und je eine Zeile schreiben, dann warten.

K0S-W2 Schritt 11

```
sudo mount /dev/drbd1003 /mnt/sb
echo 'seite w2' | sudo tee -a /mnt/sb/datei
sudo umount /mnt/sb
```

Auf `k0s-w3` dasselbe mit `seite w3`. Ohne Quorum wird jede Seite beim Mounten Primary.

Nach der Rückkehr des Netzes:

K0S-W2 Schritt 12

```
sudo drbdadm status sbtest
sudo journalctl -k --since "-2min" | grep -i split
```

Was DRBD beim Wiederverbinden festgestellt hat.

DAS SOLLTEST DU SEHEN

```
sbtest role:Secondary
  disk:UpToDate open:no
  k0s-w3 connection:StandAlone
... drbd sbtest/0 drbd1003 k0s-w3: Split-Brain detected but unresolved, dropping
connection!
... drbd sbtest k0s-w3: conn( Disconnecting -> StandAlone ) [disconnected]
```

In LINSTOR: `StandAlone(k0s-w3)` bei w2 und `Connecting(k0s-w2)` bei w3.

Beide Seiten sind `UpToDate`, und beide haben recht, jede für sich. DRBD kann nicht entscheiden, welche Änderungen zählen, und trennt die Verbindung dauerhaft: `StandAlone` heißt, es wird nicht mehr versucht.

11.7 Das Split-Brain auflösen

ERST ENTSCHEIDEN, DANN TIPPEN

Beim Auflösen verwirft eine Seite **alle** Änderungen seit der Trennung, unwiderruflich. Prüfe vorher auf beiden Seiten, was dort steht. Im Zweifel sicherst du die Daten der Verliererseite, bevor du sie verwirfst.

K0S-W2 Schritt 13

```
sudo mount -o ro /dev/drbd1003 /mnt/sb && cat /mnt/sb/datei; sudo umount /mnt/sb
```

Auf beiden Nodes nur lesend mounten und nachsehen.

DAS SOLLTEST DU SEHEN

```
w2:  basis          w3:  basis
     seite w2            seite w3
```

Hier gewinnt `k0s-w2`. Auf der **Verliererseite** `k0s-w3`:

K0S-W3 Schritt 14

```
sudo drbdadm disconnect sbtest
sudo drbdadm secondary sbtest
sudo drbdadm connect --discard-my-data sbtest
```

`--discard-my-data` sagt: Meine Änderungen seit der Trennung sind wertlos, übernimm alles von der Gegenseite.

Auf der **Gewinnerseite** `k0s-w2`, die im Zustand `StandAlone` nicht von selbst wieder verbindet:

K0S-W2 Schritt 15

```
sudo drbdadm connect sbtest
sudo drbdadm status sbtest
```

Verbindet wieder.

DAS SOLLTEST DU SEHEN

```
sbtest role:Secondary
  disk:UpToDate open:no
  k0s-w3 role:Secondary
    peer-disk:UpToDate
```

Im Kernel-Log von w3: `repl( SyncTarget -> Established ) [resync-finished]`. Auf w3 steht jetzt `basis` und `seite w2`. `seite w3` ist weg, wie beabsichtigt.

STOLPERFALLE

Den Befehl auf der falschen Seite auszuführen, ist der klassische Fehler. `--discard-my-data` gehört **immer** auf den Node, dessen Daten weg sollen. Lies den Befehl vor dem Absenden noch einmal mit dem Hostnamen im Prompt.

K0S-W1 Schritt 16

```
linstor resource-definition delete qtest
linstor resource-definition delete sbtest
```

Räumt die Test-Ressourcen ab.

11.8 Zusammenfassung der Versuche

Fall	Verhalten	Schreibpause
Secondary verliert Storage-Netz	Primary schreibt weiter, Secondary `Outdated`, Resync automatisch	ca. 10 s
Primary verliert Storage-Netz, Witness erreichbar	Wie oben, kein Split-Brain	ca. 10 s
Primary komplett isoliert	`suspended:quorum`, Anwendung hängt bis zur Rückkehr	so lange wie der Ausfall
Gegenseite übernimmt während der Isolation	Alter Primary wird `Secondary`, I/O-Fehler, Daten der Mehrheit gelten	–

Fall	Verhalten	Schreibpause
Ressource ohne Quorum getrennt beschrieben	Split-Brain, `StandAlone`, manuelle Auflösung	keine, dafür Datenverlust auf einer Seite

WAS DU GELERNT HAST

- Mit Quorum und einer dritten Stimme entsteht kein Split-Brain.
- Fällt ein Partner aus, frieren Schreibzugriffe etwa 10 Sekunden ein.
- Ohne Quorum hält der Primary mit `suspend-io` alles an und macht danach weiter.
- `force-secondary` räumt einen veralteten Primary ab, die Anwendung dort bekommt I/O-Fehler.
- Ein Split-Brain zeigt sich als `StandAlone`. Aufgelöst wird es mit `connect --discard-my-data` auf der Verliererseite und `connect` auf der Gewinnerseite.

KAPITEL 12

Betrieb: Wenn der Platz ausgeht

Du erkennst ein volles Volume und einen vollen Pool, vergrößerst Volumes im laufenden Betrieb und kennst die Falle, in die Writecache-Volumes dabei laufen.

Dauer: 40 Minuten **Voraussetzung:** Den Cluster mit StorageClass aus Kapitel 8

Alle Versuche in diesem Kapitel laufen in einem eigenen Namespace mit einem kleinen Test-Volume:

DATEI `lab/k8s/fueller.yaml (Ausschnitt)`

```
kind: PersistentVolumeClaim
metadata: {name: klein, namespace: kapazitaet}
spec:
  accessModes: [ReadWriteOnce]
  resources: {requests: {storage: 1Gi}}
---
kind: Pod
metadata: {name: fueller, namespace: kapazitaet}
spec:
  containers:
    - name: shell
      image: busybox:1.36
      command: [sleep, "86400"]
      volumeMounts: [{name: daten, mountPath: /daten}]
```

MAC Schritt 1

```
kubectl apply -f lab/k8s/fueller.yaml
kubectl -n kapazitaet wait --for=condition=Ready pod/fueller
```

Legt Namespace, PVC und einen Pod mit einer Shell an.

12.1 Ein volles Volume

MAC Schritt 2

```
kubectl -n kapazitaet exec fueller -- dd if=/dev/zero of=/daten/gross bs=1M count=2000
kubectl -n kapazitaet exec fueller -- df -h /daten
kubectl -n kapazitaet exec fueller -- sh -c 'echo test > /daten/neu'
```

Schreibt 2 GB in ein 1-GB-Volume.

DAS SOLLTEST DU SEHEN

```
1007722496 bytes (961.0MB) copied, 3.630850 seconds, 264.7MB/s
command terminated with exit code 1
Filesystem                Size      Used Available Use% Mounted on
/dev/drbd1002           977.1M    961.1M         0 100% /daten
sh: write error: No space left on device
```

Für DRBD und LINSTOR ist das kein Fehler. Das Dateisystem ist voll, das Gerät darunter ist gesund. `drbdadm status` zeigt weiter `UpToDate`. Die Anwendung sieht `No space left on device` (ENOSPC). Eine Datenbank stoppt dann in der Regel oder geht in einen Nur-Lese-Modus.

WARUM DAS WICHTIG IST

Im Betrieb heißt das: Ein voller PVC ist ein Fall für die Anwendung und das Monitoring, nicht für den Storage. Die Lösung ist Vergrößern. Wie gut das klappt, hängt allerdings vom Storage ab, wie der nächste Abschnitt zeigt.

12.2 Vergrößern im laufenden Betrieb

Die StorageClass erlaubt das Vergrößern (`allowVolumeExpansion: true`). Du änderst nur die Größe im PVC:

MAC Schritt 3

```
kubectl -n kapazitaet patch pvc klein -p '{"spec":{"resources":{"requests":{"storage":"3Gi"}}}}'
kubectl -n kapazitaet describe pvc klein | tail -3
```

Fordert 3 GiB statt 1 GiB an.

DAS SOLLTEST DU SEHEN

```
Warning  VolumeResizeFailed  external-resizer linstor.csi.linbit.com  resize volume
  "pvc-370b3c44-..." failed: ... Cannot grow the volume definition by 2097152KiB,
  as the following storage pool do not have enough free space:
Node: k0s-w1, StorPool: pool_nvme
Node: k0s-w2, StorPool: pool_nvme
```

STOLPERFALLE

LINSTOR prüft beim Vergrößern den vollen Zuwachs gegen jeden Pool des Volumes, auch gegen den Cache-Pool. Der Writecache braucht nur 5 % des Zuwachses, also gut 100 MB. LINSTOR verlangt aber 2 GiB freien Platz in `pool_nvme`, und dort sind nur 1,84 GiB frei. Mit einer Cache-VG von etwa 5 % der Datenplatte heißt das: **Jede Vergrößerung, die größer ist als der freie Platz auf der NVMe, scheitert**, auch wenn auf der HDD Terabytes frei sind.

Kubernetes erlaubt seit Version 1.34, eine gescheiterte Vergrößerung auf einen kleineren Wert zurückzunehmen, solange er größer ist als die aktuelle Größe:

MAC Schritt 4

```
kubectl -n kapazitaet patch pvc klein -p '{"spec":{"resources":{"requests":{"storage":"2Gi"}}}}'
kubectl -n kapazitaet get pvc klein
```

1 GiB Zuwachs passt in den Cache-Pool.

DAS SOLLTEST DU SEHEN

```
NAME    STATUS   VOLUME             CAPACITY   ACCESS MODES   STORAGECLASS
klein   Bound    pvc-370b3c44-...   2Gi        RWO            linstor-replicated
```

Kubernetes meldet 2 GiB, und die Ereignisse zeigen `FileSystemResizeSuccessful`.

Jetzt die Probe im Pod:

MAC Schritt 5

```
kubectl -n kapazitaet exec fueller -- df -h /daten
```

Wie groß ist das Dateisystem wirklich?

DAS SOLLTEST DU SEHEN

```
Filesystem                Size      Used Available Use% Mounted on
/dev/drbd1002           977.1M    961.1M         0 100% /daten
```

Immer noch 977 MB.

12.3 Schicht für Schicht suchen

Kubernetes sagt 2 GiB, das Dateisystem 1 GiB. Irgendwo dazwischen hängt die Größe. Du gehst den Gerätestapel von unten nach oben durch, auf dem Node, auf dem der Pod läuft:

K0S-W1 Schritt 6

```
sudo lvs -o lv_name,lv_size data cache | grep pvc-370b
sudo dmsetup table | grep pvc-370b
sudo blockdev --getsize64 /dev/drbd1002
```

LVs, Writecache-Gerät und DRBD-Gerät.

DAS SOLLTEST DU SEHEN

```
  pvc-370b3c44-....cache_00000 104.00m
  pvc-370b3c44-..._00000         2.00g
linstor_writecache_pvc-370b3c44-..._00000: 0 2105344 writecache s 252:4 252:5 4096 0
1077665792
```

Schicht	Größe	Gewachsen?
Daten-LV	2,00 GiB	ja
Cache-LV	104 MB (5 %)	ja
dm-writecache	2105344 Sektoren = 1 GiB	**nein**
DRBD	1 GiB	nein, weil die Schicht darunter nicht gewachsen ist
Dateisystem	977 MB	nein

Das Satellite-Journal bestätigt es:

K0S-W1 Schritt 7

```
sudo journalctl -u linstor-satellite --since "-10min" | grep resized
```

Was LINSTOR beim Vergrößern getan hat.

DAS SOLLTEST DU SEHEN

```
... Volume number 0 of resource 'pvc-370b3c44-...' [LVM] resized
... Volume number 0 of resource 'pvc-370b3c44-....cache' [LVM] resized
... DRBD resized pvc-370b3c44-.../0
```

Beide LVs vergrößert, DRBD „vergrößert“. Für die Writecache-Schicht steht nichts da, und es gibt keinen Fehlerbericht.

EIN STILLER FEHLER IN LINSTOR 1.33.3

Mit der Schicht WRITECACHE wird das Device-Mapper-Gerät beim Vergrößern nicht neu geladen. LINSTOR, CSI und Kubernetes melden Erfolg, die Anwendung bekommt keinen zusätzlichen Platz. Im Changelog von LINSTOR bis 1.35.3 ist dazu kein Fix vermerkt, und ein Test nach dem Update auf 1.35.3 in Kapitel 15 zeigt dasselbe Verhalten. Prüfe nach jeder Vergrößerung eines Writecache-Volumes mit `df` im Pod, ob sie wirklich angekommen ist.

12.4 Der Ausweg von Hand

Das Writecache-Gerät lässt sich im laufenden Betrieb mit der neuen Länge neu laden. Das machst du auf **jedem** Node mit einer Datenkopie, hier `k0s-w1` und `k0s-w2`:

K0S-W1 Schritt 8

```
R=pvc-370b3c44-87e5-478e-a380-adb44aa460e9
D=linstor_writecache_${R}_00000
sudo dmsetup table $D
sudo blockdev --getsz /dev/data/${R}_00000
```

Die alte Tabelle und die neue Länge des Daten-LVs in Sektoren.

DAS SOLLTEST DU SEHEN

```
0 2105344 writecache s 252:4 252:5 4096 0
4202496
```

K0S-W1 Schritt 9

```
sudo dmsetup suspend $D
echo "0 4202496 writecache s 252:4 252:5 4096 0" | sudo dmsetup reload $D
sudo dmsetup resume $D
sudo dmsetup table $D
```

Die Tabelle bleibt gleich bis auf die zweite Zahl, die Länge. `suspend` hält kurz alle Zugriffe an, `reload` lädt die neue Tabelle, `resume` gibt das Gerät wieder frei.

STOLPERFALLE

Übernimm die Gerätenummern (`252:4 252:5`) aus deiner eigenen Ausgabe von `dmsetup table` , nicht aus dem Buch. Sie sind auf jedem Node anders. Eine falsche Nummer verbindet das Writecache-Gerät mit einem fremden LV.

Danach, nur auf dem Primary:

K0S-W1 Schritt 10

```
sudo drbdadm resize $R
sudo drbdadm status $R
sudo resize2fs /dev/drbd1002
```

`drbdadm resize` lässt DRBD die neue Größe der Geräte darunter übernehmen. Den neuen Bereich synchronisiert DRBD zur Gegenseite. `resize2fs` vergrößert das gemountete ext4 online.

DAS SOLLTEST DU SEHEN

```
  k0s-w2 role:Secondary
    replication:SyncSource peer-disk:Inconsistent done:19.73
The filesystem on /dev/drbd1002 is now 525190 (4k) blocks long.
```

Die Gegenseite gleicht den neuen Bereich ab. Im Pod:

```
Filesystem                Size      Used Available Use% Mounted on
/dev/drbd1002             1.9G    961.1M    921.6M  51% /daten
```

WARUM DAS WICHTIG IST

Diesen Weg gehst du in Produktion nur, wenn eine Vergrößerung wirklich gebraucht wird. Die Alternative ohne Handarbeit: ein neues, größeres PVC anlegen und die Daten auf Anwendungsebene umziehen, etwa mit `pg_dump` . Melde den Fehler außerdem an LINBIT. Ein Volume, das laut allen Werkzeugen größer ist, als es ist, gehört behoben und nicht dauerhaft umgangen.

12.5 Ein voller Pool

MAC — Schritt 11

```
kubectl apply -f lab/k8s/zu-gross.yaml
kubectl -n kapazitaet get pvc zu-gross
kubectl -n kapazitaet get events --field-selector involvedObject.name=zu-gross,reason=ProvisioningFailed
```

Ein PVC mit 40 GiB, die Pools haben je 30 GiB.

DAS SOLLTEST DU SEHEN

```
NAME       STATUS    VOLUME   CAPACITY   ...
zu-gross   Pending                       ...
... ResourceExhausted desc = failed to enough replicas on requisite nodes:
  Message: 'Not enough available nodes'; Details: 'Not enough nodes fulfilling
  the following auto-place criteria:
 * has a deployed storage pool named [pool_data]
 * the storage pools have to have at least '41943040' free space
 ...
   Replica count: 2
   Replicas on nodes with different property:
      Aux/zone
```

Die Meldung ist ausführlich und ehrlich: kein Node mit 40 GiB frei. Der Pod bleibt `Pending`, nichts ist halb angelegt. Bei knappem Platz scheitert die Platzierung allerdings oft schon früher, als die freien Gigabytes vermuten lassen:

- Wegen `--replicas-on-different zone` braucht es **in jeder Zone** einen Node mit genug Platz. Zone b hat im Lab nur `k0s-w2`.
- Auch `pool_nvme` muss die 5 % für den Cache haben.

K0S-W1 — Schritt 12

```
linstor storage-pool list -s pool_data pool_nvme
```

Der freie Platz pro Node und Pool. Das ist die Zahl, die das Monitoring im Blick haben muss.

MAC — Schritt 13

```
kubectl delete namespace kapazitaet
```

Räumt die Tests ab. LINSTOR löscht die Volumes mit.

WAS DU GELERNT HAST

- Ein volles Dateisystem ist kein Storage-Fehler. Die Anwendung sieht ENOSPC, DRBD bleibt gesund.
- LINSTOR verlangt den vollen Zuwachs auch im Writecache-Pool.
- Mit Writecache wächst das Volume in LINSTOR 1.33.3 nicht wirklich mit. Kontrolle mit `df` im Pod ist Pflicht.
- Den Gerätestapel prüfst du von unten nach oben: `lvs`, `dmsetup table`, `blockdev`, `df`.
- Ein voller Pool lässt das PVC `Pending`. Die Meldung nennt die Platzierungskriterien.
- Platz muss pro Zone und auch im Cache-Pool reichen.

KAPITEL 13

Betrieb: Snapshots und Backup

Du weißt, warum es mit klassischem LVM keine Snapshots gibt, und sicherst Volumes stattdessen mit Velero auf Dateiebene in einen S3-Speicher an einem anderen Ort. Eine gelöschte Datenbank stellst du vollständig wieder her.

Dauer: 60 Minuten **Voraussetzung:** Den Cluster aus Kapitel 8, Platz auf `k0s-ctrl` für den S3-Speicher, `velero` und `aws` auf dem Mac

WARUM DAS WICHTIG IST

DRBD ist kein Backup. Es repliziert jeden Schreibvorgang sofort, auch ein versehentliches `DROP TABLE` oder ein `rm -rf`. Gegen Bedienfehler, Ransomware und gelöschte Namespaces hilft nur eine Kopie, die an einem anderen Ort liegt und älter ist als der Fehler.

13.1 Snapshots: nicht mit klassischem LVM

Kubernetes kennt Volume-Snapshots über CSI. Dafür braucht es einen Snapshot-Controller und seine CRDs. k0s bringt beides nicht mit:

MAC Schritt 1

```
helm install snapshot-controller piraeus-charts/snapshot-controller --version 5.4.0 -n kube-system
kubectl get crd | grep snapshot.storage
```

Installiert Controller und CRDs.

DATEI `lab/k8s/snapshot.yaml`

```
apiVersion: snapshot.storage.k8s.io/v1
kind: VolumeSnapshotClass
metadata: {name: linstor-snapshots}
driver: linstor.csi.linbit.com
deletionPolicy: Delete
---
apiVersion: snapshot.storage.k8s.io/v1
kind: VolumeSnapshot
metadata: {name: pg-vor-update, namespace: demo}
spec:
  volumeSnapshotClassName: linstor-snapshots
  source: {persistentVolumeClaimName: data-pg-0}
```

MAC Schritt 2

```
kubectl apply -f lab/k8s/snapshot.yaml
kubectl get volumesnapshotcontent -o jsonpath='{range .items[*]}{.status.error.message}{"\n"}{end}'
```

Versucht einen Snapshot des PostgreSQL-Volumes.

DAS SOLLTEST DU SEHEN

```
failed to take snapshot of the volume pvc-0d903f58-...: "rpc error: code = Internal
desc = failed to create snapshot: failed to create snapshots: Message: 'Storage driver
'LVM' does not support snapshots.'; Details: 'Used for storage pool 'pool_data' on 'k0s-
w1'.
```

Das stand schon in `linstor storage-pool list` in der Spalte `CanSnapshots: False`. Klassisches LVM kann in LINSTOR 1.33 keine Snapshots. Das hat Folgen:

- Kein schneller Rückweg vor einem riskanten Update der Anwendung.
- Keine Backups über LINSTOR selbst. LINSTOR kann Snapshots nach S3 versenden, aber eben nur Snapshots.
- Velero kann keine CSI-Snapshots nutzen.

MAC Schritt 3

```
kubectl -n demo delete volumesnapshot pg-vor-update
```

Räumt den gescheiterten Versuch ab.

AUSBLICK

Mit LINSTOR 1.35 gehen Snapshots auch mit klassischem LVM. Kapitel 15 zeigt das nach dem Update, samt dem Haken: Jeder Snapshot belegt sofort die volle Größe des Volumes.

13.2 Backup auf Dateiebene

Velero sichert Kubernetes-Objekte und, mit seinem Node-Agent, den Inhalt von Volumes Datei für Datei. Dafür liest der Node-Agent die gemounteten Volumes direkt auf dem Node, verschlüsselt und dedupliziert sie mit Kopia und legt sie in S3 ab. Snapshots braucht er dafür nicht.

Ein S3-Speicher am dritten Standort

Ein Backup gehört an einen anderen Ort als die Daten. Im Lab ist das `k0s-ctrl`, die Witness am „dritten Standort“. Dort läuft SeaweedFS als einzelnes Programm, außerhalb von Kubernetes:

DATEI `lab/backup/seaweedfs.service`

```
[Unit]
Description=SeaweedFS S3 für Velero-Backups (dritter Standort)
```

```
After=network-online.target

[Service]
ExecStart=/usr/local/bin/weed server -dir=/var/lib/seaweedfs -ip=172.16.0.176 -s3 -s3.port=8333 -
s3.config=/etc/seaweedfs/s3.json -master.volumeSizeLimitMB=1024
Restart=on-failure

[Install]
WantedBy=multi-user.target
```

`weed server` startet Master, Volume-Server und Filer in einem Prozess, `-s3` dazu die S3-Schnittstelle auf Port 8333. Die Zugangsdaten stehen in `/etc/seaweedfs/s3.json`.

DATEI `/etc/seaweedfs/s3.json`

```
{"identities":[{"name":"velero",
  "credentials":[{"accessKey":"velero","secretKey":"<40 Zeichen Zufall>"}],
  "actions":["Admin","Read","Write","List","Tagging"]}]}
```

Den Schlüssel erzeugst du mit `openssl rand -hex 20`. Dieselben Werte landen in `lab/backup/velero-credentials` im Format einer AWS-Credentials-Datei. Beide Dateien stehen in `.gitignore`.

K0S-CTRL — Schritt 4

```
cd /tmp
curl -sSfL -o weed.tgz https://github.com/seaweedfs/seaweedfs/releases/download/4.53/linux_amd64.tar.gz
tar xzf weed.tgz weed
sudo install -m 755 weed /usr/local/bin/weed
sudo install -d -m 700 /etc/seaweedfs /var/lib/seaweedfs
sudo install -m 600 /tmp/s3.json /etc/seaweedfs/s3.json
sudo install -m 644 /tmp/seaweedfs.service /etc/systemd/system/
sudo systemctl daemon-reload
sudo systemctl enable --now seaweedfs
```

`s3.json` und die Unit-Datei hast du vorher mit `scp` nach `/tmp` kopiert.

MAC — Schritt 5

```
export AWS_ACCESS_KEY_ID=velero AWS_SECRET_ACCESS_KEY=<Schlüssel>
aws --endpoint-url http://172.16.0.176:8333 s3 mb s3://velero
aws --endpoint-url http://172.16.0.176:8333 s3 ls
```

Legt den Bucket an.

DAS SOLLTEST DU SEHEN

```
make_bucket: velero
2026-11-12 21:06:33 velero
```

Velero installieren

DATEI `lab/backup/velero-values.yaml`

```yaml
# Velero mit Dateisicherung (Kopia) in SeaweedFS auf k0s-ctrl
initContainers:
  - name: velero-plugin-for-aws
    image: velero/velero-plugin-for-aws:v1.14.4
    volumeMounts: [{mountPath: /target, name: plugins}]
configuration:
  backupStorageLocation:
    - name: default
      provider: aws
      bucket: velero
      config:
        region: us-east-1
        s3ForcePathStyle: "true"
        s3Url: http://172.16.0.176:8333
  # LVM ohne thin kann keine Snapshots: alle Volumes per Dateisicherung
  defaultVolumesToFsBackup: true
snapshotsEnabled: false
deployNodeAgent: true
nodeAgent:
  # k0s: kubelet liegt unter /var/lib/k0s/kubelet
  podVolumePath: /var/lib/k0s/kubelet/pods
  pluginVolumePath: /var/lib/k0s/kubelet/plugins
credentials:
  existingSecret: velero-s3
```

`defaultVolumesToFsBackup: true` sichert jedes Volume per Kopia, ohne dass du Pods einzeln markieren musst. `snapshotsEnabled: false`, weil es keine Snapshots gibt.

STOLPERFALLE

Der kubelet-Pfad schlägt wieder zu. Der Node-Agent sucht die Volumes der Pods unter `/var/lib/kubelet/pods`. Auf k0s liegen sie unter `/var/lib/k0s/kubelet/pods`. Ohne `nodeAgent.podVolumePath` findet der Node-Agent die Volumes nicht. Im Lab wurde gleich mit dem richtigen Pfad installiert, die Fehlermeldung ohne ihn ist hier also nicht gezeigt.

MAC — Schritt 6

```
helm repo add vmware-tanzu https://vmware-tanzu.github.io/helm-charts
kubectl create namespace velero
kubectl -n velero create secret generic velero-s3 --from-file=cloud=lab/backup/velero-credentials
helm install velero vmware-tanzu/velero --version 12.2.4 -n velero -f lab/backup/velero-values.yaml
velero -n velero backup-location get
```

Installiert Velero 1.18 mit dem AWS-Plugin für S3.

DAS SOLLTEST DU SEHEN

```
NAME      PROVIDER   BUCKET/PREFIX   PHASE       LAST VALIDATED                  ACCESS
MODE   DEFAULT
default   aws        velero          Available   2026-11-12 21:07:23 +0100 CET
ReadWrite     true
```

13.3 Sichern

MAC Schritt 7

```
velero -n velero backup create demo-1 --include-namespaces demo --wait
velero -n velero backup describe demo-1 --details
```

Sichert den Namespace `demo` mit der Datenbank.

DAS SOLLTEST DU SEHEN

```
Backup completed with status: Completed.
...
  Pod Volume Backups - kopia:
    Completed:
      demo/pg-0: data (size: 40459477, incremental size: 40459477)
```

Die erste Sicherung überträgt alles, 40 MB. Weitere Sicherungen übertragen nur die Änderungen.

KONSISTENZ

Velero kopiert die Dateien einer laufenden Datenbank. PostgreSQL verkraftet das wie einen Stromausfall: Beim Start spielt es sein Write-Ahead-Log nach. Für eine wirklich konsistente Sicherung gehört ein Pre-Hook dazu, der vor dem Kopieren einen `CHECKPOINT` auslöst oder die Datenbank per `pg_dump` sichert. Velero unterstützt das über Annotationen am Pod.

13.4 Wiederherstellen

Jetzt der Ernstfall: Jemand löscht den Namespace.

MAC Schritt 8

```
kubectl delete namespace demo
```

Mit dem Namespace verschwinden PVC, PV und das LINSTOR-Volume.

K0S-W2 Schritt 9

```
linstor resource-definition list
```

Das Volume der Datenbank ist weg, nur `linstor_db` steht noch da.

MAC Schritt 10

```
velero -n velero restore create demo-1-restore --from-backup demo-1 --wait
kubectl -n demo rollout status sts/pg
kubectl -n demo exec pg-0 -- psql -U postgres -c "select text from notizen;" -c "select count(*) from
puls;"
```

Stellt alle Objekte wieder her. Für das PVC legt LINSTOR ein neues Volume an, der Node-Agent kopiert die Dateien hinein, bevor PostgreSQL startet.

DAS SOLLTEST DU SEHEN

```
Restore completed with status: Completed.
          text
-----------------------
 geschrieben auf k0s-w2
 count
-------
  2027
```

Die erste Zeile aus Kapitel 8 und alle Einträge des Pulsmessers bis zum Zeitpunkt der Sicherung.

K0S-W2 Schritt 11

```
linstor resource list -r pvc-55d1f760-7f7b-4d71-8a03-0e1e8cd11250
```

Das neue Volume.

DAS SOLLTEST DU SEHEN

```
| k0s-ctrl | DRBD,STORAGE            | Unused | Ok    |           Diskless | Yes  |
| k0s-w2   | DRBD,WRITECACHE,STORAGE | Unused | Ok    | SyncTarget(36.36%) | Yes  |
| k0s-w3   | DRBD,WRITECACHE,STORAGE | InUse  | Ok    |           UpToDate | Yes  |
```

Ein neues Volume mit neuem Namen, wieder in zwei Zonen und mit Writecache. Die Zuordnung zu Nodes ist eine andere als vorher, das ist normal.

Zeitbedarf im Lab: Sicherung 15 Sekunden, Wiederherstellung 21 Sekunden, bei 40 MB Daten. Bei einer echten Datenbank mit vielen Gigabyte bestimmt der Durchsatz zwischen Cluster und S3 die Zeit. Miss das in deiner Umgebung einmal, bevor du es im Ernstfall brauchst.

STOLPERFALLE

Ein Backup, das nie wiederhergestellt wurde, ist eine Hoffnung. Plane die Wiederherstellung in einen eigenen Namespace als regelmäßige Übung ein: `velero restore create --namespace-mappings demo:demo-test`.

WAS DU GELERNT HAST

- DRBD ist kein Backup. Es repliziert auch Fehler.
- Mit klassischem LVM gibt es in LINSTOR 1.33 keine Snapshots.
- Velero mit Node-Agent sichert Volumes Datei für Datei mit Kopia in S3.
- Auf k0s braucht der Node-Agent den Pfad `/var/lib/k0s/kubelet/pods`.
- Das Backup-Ziel liegt außerhalb des Clusters und an einem anderen Ort.
- Ein Restore legt neue LINSTOR-Volumes an und füllt sie vor dem Start der Anwendung.

KAPITEL 14

Betrieb: Eine Datenplatte tauschen

Du erkennst eine ausgefallene Datenplatte, räumst ihre Reste ab, richtest die Ersatzplatte ein und lässt LINSTOR die Kopien darauf neu aufbauen, ohne dass die Anwendung etwas merkt.

Dauer: 30 Minuten **Voraussetzung:** Zugriff auf den Hypervisor, um eine Platte zu entfernen und eine neue einzuhängen

14.1 Den Defekt nachstellen

Im Lab trifft es `k0s-w2`, den einzigen Node in Zone b. Auf ihm liegen Kopien von `linstor_db` und der PostgreSQL-Datenbank, der Primary der Datenbank läuft auf `k0s-w3`.

In Proxmox, bei der VM `k0s-w2`, im laufenden Betrieb:

1. *Hardware* → die Datenplatte (`scsi1`, 32 GB) → *Detach*. Für das System ist die Platte damit schlagartig weg, wie bei einem Defekt.
2. Die jetzt als „Unused“ markierte Platte → *Remove*.
3. *Add* → *Hard Disk*, 32 GB, gleicher Bus. Das ist die Ersatzplatte.

14.2 Was der Ausfall auslöst

K0S-W2 Schritt 1

```
sudo journalctl -k --since "-30min" | grep -E "sdb|drbd1001"
```

Was der Kernel gesehen hat.

DAS SOLLTEST DU SEHEN

```
kernel: sd 0:0:0:1: [sdb] Synchronizing SCSI cache
kernel: sd 0:0:0:1: [sdb] Add. Sense: Logical unit not supported
kernel: drbd pvc-55d1f760-.../0 drbd1001: disk( UpToDate -> Failed ) [local-io-error]
kernel: drbd pvc-55d1f760-.../0 drbd1001: Local IO failed in drbd_endio_write_sec_final.
Detaching...
```

DRBD hat den ersten Schreibfehler bemerkt und die lokale Platte abgehängt. Die Ressource läuft auf `k0s-w2` jetzt **diskless** weiter: Sie ist noch mit den Partnern verbunden, hat aber keine eigenen Daten mehr.

K0S-W2 Schritt 2

```
sudo drbdadm status pvc-55d1f760-7f7b-4d71-8a03-0e1e8cd11250
```

Die Sicht von DRBD auf `k0s-w2`.

DAS SOLLTEST DU SEHEN

```
pvc-55d1f760-... role:Secondary
  disk:Diskless client:no open:no
  k0s-ctrl role:Secondary
    peer-disk:Diskless peer-client:yes
  k0s-w3 role:Primary
    peer-disk:UpToDate
```

`client:no` heißt: Diese Kopie **sollte** Daten haben, hat sie aber nicht. Eine gewollt disklose Kopie wie auf der Witness zeigt `client:yes`.

K0S-W3 Schritt 3

```
linstor resource list
```

Die Sicht von LINSTOR.

DAS SOLLTEST DU SEHEN

```
| linstor_db       | k0s-w2   | DRBD,STORAGE            | Unused | Ok    | Diskless,
SkipDisk (R) | Yes  |
| pvc-55d1f760-... | k0s-w2   | DRBD,WRITECACHE,STORAGE | Unused | Ok    | Diskless,
SkipDisk (R) | Yes  |
SkipDisk:
  At least one resource has 'DrbdOptions/SkipDisk' enabled. This indicates an IO error
on the
  affected resource(s). Remove this property (using 'linstor resource set-property $node
$rsc
  DrbdOptions/SkipDisk') to instruct LINSTOR and DRBD to adjust (and recreate if
necessary)
  the affected logical volumes again.
```

LINSTOR hat bei jeder betroffenen Ressource `SkipDisk` gesetzt. Damit versucht es nicht ständig, die kaputte Platte wieder anzuhängen. Die Meldung sagt gleich, wie es weitergeht.

Die Anwendung hat davon nichts gemerkt. Der Client aus Kapitel 10 meldete in dieser Zeit keinen einzigen Fehler, weil der Primary auf `k0s-w3` liegt und die Kopie dort intakt ist.

WARUM DAS WICHTIG IST

Bis die Platte getauscht ist, gibt es von jeder Ressource auf `k0s-w2` nur noch **eine** Kopie mit Daten. Fällt jetzt auch `k0s-w3` aus, sind die Daten weg. Ein Plattendefekt ist deshalb immer dringend, auch wenn alles weiterläuft. `drbd_device_unintentionaldiskless` aus Kapitel 17 meldet genau diesen Zustand.

K0S-W3 Schritt 4

```
linstor storage-pool list -n k0s-w2
```

Wie sieht LINSTOR den Pool der kaputten Platte?

DAS SOLLTEST DU SEHEN

```
| pool_data            | k0s-w2 | LVM      | data     |    27.79 GiB |     30.00 GiB |
True | Ok |
| pool_nvme            | k0s-w2 | LVM      | cache    |     1.89 GiB |      2.00 GiB |
True | Ok |
```

Der Pool steht noch auf `Ok`, mit den Werten von vor dem Ausfall. Verlass dich nicht auf diese Anzeige, sondern auf DRBD.

14.3 Die Reste abräumen

Die Ersatzplatte heißt `sdc`, nicht `sdb`. Der Kernel vergibt Namen in der Reihenfolge, in der Geräte erscheinen. Von der alten Platte sind noch Device-Mapper-Geräte übrig:

K0S-W2 Schritt 5

```
lsblk
sudo dmsetup info -c -o name,open
```

Die Platten und die verbliebenen Device-Mapper-Geräte.

DAS SOLLTEST DU SEHEN

```
sdc         32G disk
...
Name                                                        Open
cache-pvc--55d1f760--...cache_00000                            1
data-linstor_db_00000                                          0
data-pvc--55d1f760--..._00000                                  1
linstor_writecache_pvc-55d1f760-..._00000                      0
```

Diese Geräte zeigen auf eine Platte, die es nicht mehr gibt. Die neuen LVs bekommen dieselben Namen und würden mit ihnen kollidieren. Du entfernst sie von oben nach unten, also erst den Writecache, dann die LVs darunter:

K0S-W2 Schritt 6

```
sudo dmsetup remove linstor_writecache_pvc-55d1f760-7f7b-4d71-8a03-0e1e8cd11250_00000
sudo dmsetup remove cache-pvc--55d1f760--7f7b--4d71--8a03--0e1e8cd11250.cache_00000
sudo dmsetup remove data-pvc--55d1f760--7f7b--4d71--8a03--0e1e8cd11250_00000
sudo dmsetup remove data-linstor_db_00000
sudo dmsetup ls
```

Danach meldet `dmsetup ls` `No devices found`.

STOLPERFALLE

Entferne nur Geräte, die zu der ausgefallenen Platte gehören. Auf einem Node mit mehreren Platten liegen daneben gesunde Geräte. `lsblk` zeigt, welche Geräte noch an einer vorhandenen Platte hängen.

14.4 Die Ersatzplatte einrichten

Genau wie in Kapitel 2, nur mit `sdc`:

K0S-W2 Schritt 7

```
sudo wipefs -a /dev/sdc
sudo parted -s /dev/sdc mklabel gpt mkpart data 1MiB 30GiB mkpart cache 30GiB 100%
sudo pvcreate /dev/sdc1 /dev/sdc2
sudo vgcreate data /dev/sdc1
sudo vgcreate cache /dev/sdc2
sudo vgs
```

Die Volume-Groups bekommen dieselben Namen wie vorher. Die Storage-Pools in LINSTOR verweisen auf diese Namen und bleiben dadurch unverändert.

DAS SOLLTEST DU SEHEN

```
  VG    #PV #LV #SN Attr   VSize   VFree
  cache   1   0   0 wz--n-  <2.00g  <2.00g
  data    1   0   0 wz--n- <30.00g <30.00g
```

GERÄTENAMEN

Dass die Platte jetzt `sdc` heißt, ist für LVM egal. LVM erkennt seine Physical Volumes an ihrer UUID, nicht am Namen. Nach dem nächsten Neustart kann die Platte auch wieder `sdb` heißen.

14.5 Die Kopien neu aufbauen

Jetzt tust du, was LINSTOR in seiner Meldung vorgeschlagen hat:

K0S-W3 Schritt 8

```
linstor resource set-property k0s-w2 linstor_db DrbdOptions/SkipDisk
linstor resource set-property k0s-w2 pvc-55d1f760-7f7b-4d71-8a03-0e1e8cd11250 DrbdOptions/SkipDisk
```

Ohne Wert gesetzt, wird die Eigenschaft gelöscht. LINSTOR legt daraufhin die fehlenden LVs neu an, baut den Writecache auf und hängt die Platte wieder unter DRBD. DRBD synchronisiert die Daten von der Gegenseite.

Eine halbe Minute später:

K0S-W3 Schritt 9

```
linstor resource list
linstor resource list --faulty
```

Der Zustand danach.

DAS SOLLTEST DU SEHEN

```
| linstor_db       | k0s-w2   | DRBD,STORAGE            | Unused | Ok    | UpToDate |
Yes  |
| pvc-55d1f760-... | k0s-w2   | DRBD,WRITECACHE,STORAGE | Unused | Ok    | UpToDate |
Yes  |
```

`--faulty` ist leer.

K0S-W2 Schritt 10

```
lsblk /dev/sdc
```

Der neu aufgebaute Gerätestapel.

DAS SOLLTEST DU SEHEN

```
sdc                                          32G  disk
├─sdc1                                       30G  part
│ ├─data-linstor_db_00000                   204M  lvm
│ │ └─drbd1000                            203.9M  disk
│ └─data-pvc--55d1f760-..._00000              2G  lvm
│   └─linstor_writecache_pvc-55d1f760-..._00000  2G  dm
│     └─drbd1001                              2G  disk
└─sdc2                                       2G  part
  └─cache-pvc--55d1f760-...cache_00000     104M  lvm
    └─linstor_writecache_pvc-55d1f760-..._00000  2G  dm
```

Derselbe Aufbau wie in Kapitel 7, auf der neuen Platte. Der Cache hat 104 MB, 5 % von 2 GB.

WAS DU GELERNT HAST

- Bei einem Plattendefekt hängt DRBD die Platte ab und läuft diskless weiter. Die Anwendung merkt nichts.
- LINSTOR setzt `SkipDisk` und zeigt `Diskless, SkipDisk (R)`.
- Bis zum Tausch gibt es nur eine Kopie. Das ist dringend, auch wenn alles läuft.
- Alte Device-Mapper-Geräte von oben nach unten mit `dmsetup remove` abräumen.
- Volume-Groups mit denselben Namen anlegen, dann `SkipDisk` löschen. LINSTOR baut alles neu auf.

KAPITEL 15

Betrieb: Updates von LINSTOR, CSI und k0s

LINSTOR von 1.33.3 auf 1.35.3, den CSI-Treiber von 1.12 auf 1.13 und k0s von 1.36.6 auf 1.37.2, in der richtigen Reihenfolge und ohne die Datenbank zu verlieren. Dazu der Blick darauf, was dabei schiefgehen kann.

Dauer: 90 Minuten **Voraussetzung:** Den Cluster aus den vorigen Kapiteln mit Velero aus Kapitel 13

15.1 Die Abhängigkeit

Der CSI-Treiber spricht mit dem LINSTOR-Controller über dessen REST-API. Jede CSI-Version verlangt eine Mindestversion dieser API:

LINSTOR	REST-API	passender CSI-Treiber
1.33.3	1.27.0	bis 1.12.x
1.35.3	1.29.1	1.13.x

Daraus folgt die Reihenfolge: **erst LINSTOR, dann CSI.** Umgekehrt geht es nicht, wie der nächste Abschnitt zeigt.

15.2 Die falsche Reihenfolge

Was passiert, wenn jemand den Pin auf CSI 1.12 entfernt, bevor LINSTOR aktualisiert ist?

MAC Schritt 1

```
helm upgrade linstor-cluster piraeus-charts/linstor-cluster --version 1.2.0 \
  -n piraeus-datastore -f werte-mit-csi-1.13.1.yaml
kubectl -n piraeus-datastore get pods
kubectl -n piraeus-datastore logs deploy/linstor-csi-controller -c linstor-csi --tail=1
```

Die Werte aus Kapitel 8, nur mit `piraeus-csi:v1.13.1` statt `v1.12.0`.

DAS SOLLTEST DU SEHEN

```
linstor-csi-controller-6c9bc687dd-bhsv5   5/6   CrashLoopBackOff   11 (8s ago)   70s
linstor-csi-node-647g6                    2/3   CrashLoopBackOff   5 (6s ago)    66s
...
level=fatal msg="LINSTOR 1.33.3 (REST API 1.27.0) is not supported: the driver requires
  REST API 1.29 or newer in the 1.x series, shipped with LINSTOR 1.35"
```

Die Meldung ist eindeutig. Die laufende Datenbank war davon nicht betroffen: Gemountete Volumes brauchen den Treiber nicht mehr. Neue PVCs, Mounts nach einem Pod-Neustart und Vergrößerungen hätten aber nicht funktioniert. Zurück auf 1.12 mit den alten Werten, und alles läuft wieder.

STOLPERFALLE

Das ist eine typische Lage: Der Operator 2.12 bringt CSI 1.13 mit, und nur ein Pin hält den Treiber auf 1.12. Wer den Pin bei einem Aufräumen der Helm-Werte entfernt, legt das Anlegen neuer Volumes still. Der Pin gehört kommentiert, und er darf erst nach dem LINSTOR-Update fallen.

15.3 Vorbereitung

MAC Schritt 2

```
velero -n velero backup create vor-linstor-update --include-namespaces demo --wait
```

Eine frische Sicherung der Anwendungsdaten. Das LINSTOR-Update selbst fasst keine Daten an, aber wer ein Update ohne aktuelle Sicherung beginnt, hat Glück oder Pech.

K0S-W1 Schritt 3

```
sudo drbd-reactorctl status linstor_db | head -2
linstor controller version
```

Wo läuft der Controller, und welche Version hat er?

DAS SOLLTEST DU SEHEN

```
Promoter: Resource linstor_db currently active on node 'k0s-w2'
linstor controller 1.33.3; GIT-hash: 1ff3a1ea5cb5c77100265ec442bb9a492b225145
```

15.4 LINSTOR aktualisieren

Controller und Satellites müssen dieselbe Version haben. Während des Updates gibt es deshalb eine Phase, in der aktualisierte Satellites als `OFFLINE` erscheinen. DRBD läuft davon unberührt weiter, nur der Controller kann diese Nodes kurz nicht steuern.

REIHENFOLGE

1. Die Nodes, auf denen der Controller **nicht** läuft
2. Die Witness
3. Den Node mit dem aktiven Controller, nachdem er den Controller abgegeben hat

Auf `k0s-w1`, ohne aktiven Controller:

K0S-W1 Schritt 4

```
V=1.35.3-1ppa1~noble1
sudo apt-mark unhold linstor-common linstor-satellite linstor-controller
sudo apt-get install -y linstor-common=$V linstor-satellite=$V linstor-controller=$V
sudo apt-mark hold linstor-common linstor-satellite linstor-controller
systemctl is-active linstor-satellite
systemctl is-enabled linstor-controller
```

Die Version 1.35.3 kommt aus dem PPA. Die Paketinstallation startet den Satellite von selbst neu.

DAS SOLLTEST DU SEHEN

```
Setting up linstor-satellite (1.35.3-1ppa1~noble1) ...
Setting up linstor-controller (1.35.3-1ppa1~noble1) ...
active
disabled
```

`linstor-controller` bleibt `disabled`. Das Paket respektiert das, drbd-reactor behält die Kontrolle. In `linstor node list` steht `k0s-w1` jetzt auf `OFFLINE`, Satellite 1.35 gegen Controller 1.33.

Dasselbe auf `k0s-w3`, auf der Witness `k0s-ctrl` ohne das Paket `linstor-controller`.

Jetzt der Node mit dem aktiven Controller, `k0s-w2`. Im Lab wurde das Paket dort direkt installiert, und das Journal zeigt, was dann geschah:

DAS SOLLTEST DU SEHEN

```
drbd-reactor: INFO  systemd_start: systemctl start drbd-services@linstor_db.target
drbd-linstor_db: linstor_db: State change failed: (-1) Multiple primaries not allowed by
config
drbd-reactor: WARN  Starting 'linstor_db' failed: Return code not status success
systemd: Dependency failed for linstor-controller.service - drbd-reactor controlled
linstor-controller.
```

Das Paket hat den laufenden Controller gestoppt. drbd-reactor hat die ganze Kette heruntergefahren, und `k0s-w3`, bereits auf 1.35, hat übernommen. Ein Versuch, auf `k0s-w2` neu zu starten, scheitert richtigerweise, weil `k0s-w3` jetzt Primary ist.

Am Ende lief alles, aber über einen ungeplanten Failover. Sauber ist es, den Controller vorher abzugeben:

K0S-W2 Schritt 5

```
sudo drbd-reactorctl evict linstor_db
V=1.35.3-1ppa1~noble1
sudo apt-mark unhold linstor-common linstor-satellite linstor-controller
sudo apt-get install -y linstor-common=$V linstor-satellite=$V linstor-controller=$V
sudo apt-mark hold linstor-common linstor-satellite linstor-controller
```

Der Controller startet auf einem Node, der schon 1.35 hat, und erst dann wird das Paket auf `k0s-w2` getauscht.

K0S-W3 Schritt 6

```
linstor controller version
linstor node list
linstor resource list --faulty
```

Prüft das Ergebnis.

DAS SOLLTEST DU SEHEN

```
linstor controller 1.35.3; GIT-hash: 75d5fa72d504ccfe55c421ccccda9505af8371f8
| Node     | Platform | NodeType  | Addresses                 | State  |
| k0s-ctrl | LINUX    | SATELLITE | 172.16.0.176:3366 (PLAIN) | Online |
| k0s-w1   | LINUX    | SATELLITE | 10.10.0.11:3366 (PLAIN)   | Online |
| k0s-w2   | LINUX    | SATELLITE | 10.10.0.12:3366 (PLAIN)   | Online |
| k0s-w3   | LINUX    | SATELLITE | 10.10.0.13:3366 (PLAIN)   | Online |
```

`--faulty` ist leer. Neu ist die Spalte `Platform`.

15.5 Was mit der Controller-Datenbank passiert

Beim ersten Start von 1.35 migriert der Controller seine Datenbank auf ein neues H2-Format. Das steht in der Unit-Datei:

K0S-W1 Schritt 7

```
systemctl cat linstor-controller | grep ExecStartPre
sudo ls -la /var/lib/linstor/
```

Die Startschritte und der Inhalt der Datenbank-Ressource, auf dem Node mit dem aktiven Controller.

DAS SOLLTEST DU SEHEN

```
ExecStartPre=/usr/share/linstor-server/bin/linstor-database migrate-h2 --yes ...
-rw-r--r-- 1 linstor-controller root   6442 Nov 12 20:16
auto_db_export_2026-11-12_20-16-01.json.gz
-rw-r--r-- 1 linstor-controller root 135168 Nov 12 20:19 linstordb.mv.db
-rw-r--r-- 1 linstor-controller root 839680 Nov 12 20:15 linstordb.mv.db.h2v1.bak
```

Der Controller hat vor der Migration selbst zwei Sicherungen angelegt: die alte Datei als `.h2v1.bak` und einen JSON-Export. Beide liegen auf `linstor_db` und sind damit auf drei Nodes repliziert. Ab jetzt läuft der Controller als eigener Benutzer `linstor-controller`.

WARUM DAS WICHTIG IST

Die Migration ist eine Einbahnstraße: Ein Controller 1.33 kann die neue Datei nicht mehr lesen. Ein Rückweg auf 1.33 ginge nur über die Sicherung, die vor der Migration entstanden ist. Kopiere `auto_db_export_*.json.gz` deshalb nach dem Update zusätzlich an einen anderen Ort.

DATENBANK-EXPORT VON HAND: NICHT ERFOLGREICH GETESTET

LINSTOR hat ein Werkzeug `linstor-database export-db`. Laut Handbuch muss dafür der Controller gestoppt sein. Mit drbd-reactor reicht ein `systemctl stop linstor-controller` nicht: drbd-reactor fährt dann die Kette herunter, und ein anderer Node übernimmt. Im Lab funktionierte der Stopp so: `drbd-reactorctl disable linstor_db` auf den Standby-Nodes, dann `drbd-reactorctl disable --now linstor_db` auf dem aktiven Node, danach `linstor_db` von Hand zum Primary machen und mounten. Der Export scheiterte dann aber mit `Schema "LINSTOR" not found ... The database has pending migrations`. Die Ursache wurde nicht geklärt. Verlass dich für einen Rückweg auf die automatische Sicherung oben und auf `drbd-reactorctl enable linstor_db` auf allen Kandidaten, um den Wartungsmodus zu beenden.

15.6 CSI aktualisieren

Jetzt darf der Pin fallen:

DATEI `lab/piraeus/linstor-cluster-values.yaml (Ausschnitt)`

```
        - name: linstor-csi
          image: quay.io/piraeusdatastore/piraeus-csi:v1.13.1
```

MAC — Schritt 8

```
helm upgrade linstor-cluster piraeus-charts/linstor-cluster --version 1.2.0 \
  -n piraeus-datastore -f lab/piraeus/linstor-cluster-values.yaml
kubectl -n piraeus-datastore rollout status ds/linstor-csi-node
kubectl get linstorcluster -o \
  jsonpath='{range .items[].status.conditions[*]}{.type}={.status} {.message}{"\n"}{end}'
```

Rollt CSI 1.13.1 aus.

DAS SOLLTEST DU SEHEN

```
Applied=True Resources applied
Available=True Controller 1.35.3 (API: 1.29.1, Git: 75d5fa72...) reachable at
'http://10.10.0.100:3370'
Configured=True Properties applied
```

Der Client aus Kapitel 10 meldete während des gesamten LINSTOR- und CSI-Updates keinen Fehler.

15.7 Was 1.35 ändert und was nicht

Thema	1.33.3	1.35.3
Snapshots mit klassischem LVM	nicht möglich	**möglich**
Vergrößern mit Writecache	Writecache-Gerät wächst nicht mit	**unverändert fehlerhaft**

K0S-W2 Schritt 9

```
linstor storage-pool list -s pool_data
```

`CanSnapshots` ist jetzt `True`.

Der Snapshot-Versuch aus Kapitel 13 gelingt jetzt:

DAS SOLLTEST DU SEHEN

```
pg-vor-update   true   data-pg-0   2Gi   linstor-snapshots   snapcontent-10131a8f-...
| pvc-55d1f760-... | snapshot-10131a8f-... | k0s-w2, k0s-w3 | 0: 2 GiB | ... |
Successful |
  pvc-55d1f760-..._00000                        data owi-aos---   2.00g
  pvc-55d1f760-..._00000_snapshot-10131a8f-... data sri-a-s--k   2.00g
pvc-55d1f760-..._00000
```

Ein PVC aus diesem Snapshot (`lab/k8s/aus-snapshot.yaml`) enthielt den vollständigen Datenbestand.

STOLPERFALLE

Ein Snapshot mit klassischem LVM reserviert sofort **die volle Größe** des Volumes, hier 2 GiB pro Node. Zehn Snapshots einer 100-GB-Datenbank brauchen 1 TB. Und laut Changelog lassen sich Volumes mit solchen Snapshots nicht vergrößern. Snapshots sind hier ein Werkzeug für kurze Zeit, etwa vor einem riskanten Update, kein Backup-Ersatz.

Das Vergrößern eines Writecache-Volumes von 1 GiB auf 1500 MiB unter 1.35.3 ergab dasselbe Bild wie in Kapitel 12: Kubernetes meldet `1500Mi`, das Daten-LV hat 3080192 Sektoren, das Writecache-Gerät weiter 2105344, das Dateisystem 977 MB. Zusätzlich wird das Vergrößern verweigert, solange ein Volume noch synchronisiert

(`Cannot resize volume, because we have a non-UpToDate DRBD device`). Der Resizer versucht es später von selbst noch einmal.

15.8 k0s aktualisieren

k0s ist der einfachste Teil. Die neue Version kommt in `lab/k0sctl.yaml`:

DATEI `lab/k0sctl.yaml (Ausschnitt)`

```
  k0s:
    version: v1.37.2+k0s.1
```

MAC Schritt 10

```
k0sctl apply --config lab/k0sctl.yaml --concurrency 1
```

k0sctl aktualisiert erst den Controller, dann die Worker nacheinander. Jeden Worker leert es vorher (`drain`) und gibt ihn danach wieder frei.

DAS SOLLTEST DU SEHEN

```
level=info msg="==> Running phase: Upgrade controllers"
level=info msg="172.16.0.176:22: starting upgrade"
level=info msg="==> Running phase: Upgrade workers"
level=info msg="172.16.0.186:22: starting upgrade"
level=info msg="172.16.0.186:22: upgrade finished"
...
level=info msg="==> Finished in 5m14s"
```

MAC Schritt 11

```
kubectl get nodes
```

Eine Minute später.

DAS SOLLTEST DU SEHEN

```
NAME     STATUS   ROLES    AGE    VERSION
k0s-w1   Ready    <none>   ...    v1.37.2+k0s
k0s-w2   Ready    <none>   ...    v1.37.2+k0s
k0s-w3   Ready    <none>   ...    v1.37.2+k0s
```

Direkt nach dem Ende von k0sctl zeigte `k0s-w3` noch die alte Version, weil der kubelet sich gerade neu meldete.

Zwei Beobachtungen aus dem Lab:

- Der Drain von `k0s-w3` hat `pg-0` verschoben. Bis das Volume auf `k0s-w3` gelöst war, wartete der Pod etwa 30 Sekunden (`Volume is already exclusively attached to one node, waiting on detach`). Eine Datenbank mit einer einzigen Instanz hat bei jedem Drain ihres Nodes eine kurze Pause.

- Der Client aus Kapitel 10 lief zu diesem Zeitpunkt als einzelner Pod ohne Deployment. Der Drain hat ihn gelöscht, und niemand hat ihn neu gestartet. Alles, was einen Drain überleben soll, braucht einen Controller wie ein Deployment. `lab/k8s/pg-client.yaml` ist deshalb inzwischen ein Deployment.

WAS DU GELERNT HAST

- Erst LINSTOR, dann CSI. CSI 1.13 braucht LINSTOR 1.35.
- Satellites mit anderer Version als der Controller sind `OFFLINE`. DRBD läuft weiter.
- Den Node mit dem aktiven Controller zuletzt aktualisieren und vorher mit `evict` abgeben.
- LINSTOR 1.35 migriert seine Datenbank beim ersten Start und legt vorher Sicherungen auf `linstor_db` an.
- 1.35 bringt Snapshots für klassisches LVM, die je die volle Volume-Größe belegen.
- Der Writecache-Fehler beim Vergrößern besteht in 1.35.3 weiter.
- `k0sctl apply --concurrency 1` aktualisiert k0s Node für Node mit Drain.

KAPITEL 16

Dasselbe mit Ansible

Die Handarbeit aus den Kapiteln 5 und 6 als zwei Ansible-Rollen: `linstor` für alle Nodes und `linstor-controller-ha` für die Controller-Kandidaten. Du verstehst, was jede Aufgabe tut, weil du sie vorher von Hand gemacht hast.

Dauer: 30 Minuten **Voraussetzung:** Ansible auf dem Mac (getestet mit ansible-core 2.20 und `community.general` 12.2), die VMs aus den vorigen Kapiteln

WARUM DAS WICHTIG IST

In Produktion installiert eine Automatisierung wie Ansible LINSTOR auf vielen Hosts. Wer dort etwas ändern oder einen Fehler suchen muss, liest Rollen, keine Anleitung. Dieses Kapitel ist die Brücke: Jede Aufgabe der Rollen entspricht einem Befehl, den du in Kapitel 5 oder 6 von Hand ausgeführt hast.

16.1 Aufbau

Datei	Inhalt
`lab/ansible/inventory.ini`	Nodes, Gruppen und je Node die beiden IPs und die Zone
`lab/ansible/group_vars/all.yml`	LINSTOR-Version, VIP, PPA
`lab/ansible/roles/linstor/`	Pakete, DRBD-Modul, LVM-Filter, Client, Satellite, Metriken
`lab/ansible/roles/linstor-controller-ha/`	Controller-Paket, Mount-Unit, Promoter
`lab/ansible/site.yml`	Wendet beide Rollen auf die passenden Gruppen an

DATEI `lab/ansible/inventory.ini`

```
# Alle Nodes mit DRBD. storage_ip leer = kein Storage-Netz (Witness).
[linstor_satellites]
k0s-ctrl ansible_host=172.16.0.176 mgmt_ip=172.16.0.176 storage_ip=
k0s-w1   ansible_host=172.16.0.186 mgmt_ip=172.16.0.186 storage_ip=10.10.0.11 zone=a
k0s-w2   ansible_host=172.16.0.36  mgmt_ip=172.16.0.36  storage_ip=10.10.0.12 zone=b
k0s-w3   ansible_host=172.16.0.72  mgmt_ip=172.16.0.72  storage_ip=10.10.0.13 zone=a

# Nodes, die den Controller übernehmen können
[linstor_controllers]
k0s-w1
k0s-w2
k0s-w3
```

```
[all:vars]
ansible_user=ubuntu
ansible_ssh_private_key_file=~/.ssh/homelab
```

Zwei Gruppen: `linstor_satellites` sind alle Nodes mit DRBD, `linstor_controllers` die Kandidaten für den Controller. Die Witness hat keine `storage_ip`.

DATEI `lab/ansible/group_vars/all.yml`

```
# Eine Stelle für die Version. Für ein Update hier ändern (Kapitel 15).
linstor_version: "1.35.3-1ppa1~noble1"
linstor_vip: "10.10.0.100"
linstor_vip_nic: ens19
linstor_ppa: "ppa:linbit/linbit-drbd9-stack"
```

Die Version steht an genau einer Stelle. In größeren Umgebungen steht sie oft in `roles/linstor/defaults/main.yml`, mit einer möglichen Überschreibung je Host.

16.2 Die Rolle linstor

DATEI `lab/ansible/roles/linstor/tasks/main.yml`

```
# add-apt-repository legt das PPA im deb822-Format an, wie bei der Handinstallation in Kapitel 5.
# apt_repository würde daneben eine zweite .list-Datei anlegen.
- name: LINBIT-PPA einbinden
  ansible.builtin.command:
    cmd: "add-apt-repository -y {{ linstor_ppa }}"
    creates: /etc/apt/sources.list.d/linbit-ubuntu-linbit-drbd9-stack-noble.sources

- name: DRBD, Werkzeuge und Kernel-Header installieren
  ansible.builtin.apt:
    name:
      - "linux-headers-{{ ansible_kernel }}"
      - drbd-dkms
      - drbd-utils
      - drbd-reactor
      - linstor-client
      - lvm2
      - thin-provisioning-tools
      - default-jre-headless
    state: present

- name: LINSTOR-Satellite in fester Version
  ansible.builtin.apt:
    name:
      - "linstor-common={{ linstor_version }}"
      - "linstor-satellite={{ linstor_version }}"
    state: present
    allow_downgrade: false
    # Die Pakete sind gehalten. Ohne diese Option verweigert apt eine neue Version.
    allow_change_held_packages: true

- name: Versionen festhalten
  ansible.builtin.dpkg_selections:
    name: "{{ item }}"
    selection: hold
  loop: [linstor-common, linstor-satellite, drbd-dkms, drbd-utils]
```

```
- name: DRBD beim Start laden
  ansible.builtin.copy:
    dest: /etc/modules-load.d/drbd.conf
    content: "drbd\n"
    mode: "0644"

- name: DRBD jetzt laden
  community.general.modprobe:
    name: drbd
    state: present

- name: LVM soll DRBD-Geräte nicht scannen
  ansible.builtin.lineinfile:
    path: /etc/lvm/lvm.conf
    regexp: '^\s*#?\s*global_filter\s*='
    line: ' global_filter = [ "r|^/dev/drbd|" ]'
    insertafter: '^devices \{'

- name: Client-Konfiguration
  ansible.builtin.template:
    src: linstor-client.conf.j2
    dest: /etc/linstor/linstor-client.conf
    mode: "0644"

- name: DRBD-Metriken über drbd-reactor
  ansible.builtin.copy:
    dest: /etc/drbd-reactor.d/prometheus.toml
    content: |
      [[prometheus]]
      enums = true
      address = ":9942"
    mode: "0644"
  notify: drbd-reactor neu laden

- name: Satellite läuft
  ansible.builtin.systemd_service:
    name: linstor-satellite
    enabled: true
    state: started

- name: drbd-reactor läuft
  ansible.builtin.systemd_service:
    name: drbd-reactor
    enabled: true
    state: started
```

Aufgabe	Von Hand in
PPA, Pakete, Header	Kapitel 5, „Die Pakete installieren“
Version festhalten	Kapitel 5, `apt-mark hold`
Modul laden, `modules-load.d`	Kapitel 5
LVM-Filter	Kapitel 5, „LVM darf DRBD-Geräte nicht anfassen“
Client-Konfiguration	Kapitel 6, VIP bzw. Kandidatenliste für die Witness
`prometheus.toml`	Kapitel 17

STOLPERFALLE

Die erste Fassung band das PPA mit `ansible.builtin.apt_repository` ein. Der Trockenlauf zeigte, dass das Modul eine zweite Quelle als `.list`-Datei angelegt hätte, neben der `.sources`-Datei, die `add-apt-repository` im deb822-Format erzeugt. Doppelte Quellen sind harmlos, aber verwirrend. Deshalb ruft die Rolle `add-apt-repository` auf, mit `creates` als Schutz vor Wiederholung.

Die Vorlage für den Client unterscheidet zwischen Workern und Witness:

DATEI `lab/ansible/roles/linstor/templates/linstor-client.conf.j2`

```
[global]
{% if storage_ip %}
controllers={{ linstor_vip }}
{% else %}
controllers={{ groups['linstor_controllers'] | map('extract', hostvars, 'mgmt_ip') | join(',') }}
{% endif %}
```

16.3 Die Rolle linstor-controller-ha

DATEI `lab/ansible/roles/linstor-controller-ha/tasks/main.yml`

```
- name: Controller-Paket und VIP-Agent
  ansible.builtin.apt:
    name:
      - "linstor-controller={{ linstor_version }}"
      - resource-agents-base
    state: present
    allow_change_held_packages: true

- name: Version festhalten
  ansible.builtin.dpkg_selections:
    name: linstor-controller
    selection: hold

# Nur drbd-reactor startet den Controller. Kein state: ein laufender Controller bleibt laufen.
- name: Controller nicht von systemd starten lassen
  ansible.builtin.systemd_service:
    name: linstor-controller
    enabled: false

- name: Mount-Unit für die Controller-Datenbank
  ansible.builtin.copy:
    src: var-lib-linstor.mount
    dest: /etc/systemd/system/var-lib-linstor.mount
    mode: "0644"
  notify: systemd neu laden

- name: Promoter für linstor_db
  ansible.builtin.template:
    src: linstor_db.toml.j2
    dest: /etc/drbd-reactor.d/linstor_db.toml
    mode: "0644"
  notify: drbd-reactor neu laden
```

Die Mount-Unit liegt unverändert unter `files/`, der Promoter als Vorlage unter `templates/`, damit VIP und Netzwerkkarte aus den Variablen kommen.

STOLPERFALLE

Die Aufgabe für `linstor-controller` setzt nur `enabled: false`, aber **keinen** `state`. Mit `state: stopped` würde Ansible bei jedem Lauf den aktiven Controller anhalten, und drbd-reactor würde die Kette auf einen anderen Node schwenken (Kapitel 15). Die Rolle darf den Controller nie selbst starten oder stoppen.

16.4 Ausprobieren

MAC Schritt 1

```
cd lab/ansible
ansible-playbook --syntax-check site.yml
ansible-playbook site.yml --check --diff
```

Erst Syntax, dann ein Trockenlauf, der zeigt, was Ansible ändern würde, ohne es zu tun.

DAS SOLLTEST DU SEHEN

```
PLAY RECAP *********************************************************************
k0s-ctrl   : ok=12   changed=0    unreachable=0    failed=0    skipped=0
k0s-w1     : ok=18   changed=0    unreachable=0    failed=0    skipped=0
k0s-w2     : ok=18   changed=0    unreachable=0    failed=0    skipped=0
k0s-w3     : ok=18   changed=0    unreachable=0    failed=0    skipped=0
```

MAC Schritt 2

```
ansible-playbook site.yml
ansible-playbook site.yml
```

Zwei echte Läufe hintereinander.

DAS SOLLTEST DU SEHEN

Beide Läufe enden mit `changed=0` auf allen Nodes. Die Rollen beschreiben genau den Zustand, den du in den Kapiteln 5, 6 und 17 von Hand hergestellt hast, und sie verändern nichts, wenn nichts zu tun ist.

WAS DIESER TEST ZEIGT UND WAS NICHT

Getestet ist, dass die Rollen zur Handinstallation passen und idempotent sind. Nicht getestet ist ein Lauf gegen einen leeren Node. Dabei würden die Pakete installiert und das Modul gebaut. Die Rollen enthalten außerdem nicht, was einmalig am Controller passiert: Nodes registrieren, Schnittstellen, Pfade, Zonen, Pools, `Aux/topology/linbit.com/hostname`, `linstor_db` anlegen und die Datenbank umziehen. Das sind die Befehle aus den Kapiteln 5 bis 8. In einer bestehenden Umgebung ist zu prüfen, wo diese Schritte automatisiert sind.

16.5 Ein Update mit Ansible

Für das LINSTOR-Update aus Kapitel 15 änderst du `linstor_version` in `group_vars/all.yml`. Die Reihenfolge bleibt dieselbe: Mit `--limit` aktualisierst du erst die Nodes ohne aktiven Controller, dann die Witness, und den aktiven Node erst nach `drbd-reactorctl evict`.

MAC — Schritt 3

```
ansible-playbook site.yml --limit k0s-w1,k0s-w3
ansible-playbook site.yml --limit k0s-ctrl
ssh k0s-w2 'sudo drbd-reactorctl evict linstor_db'
ansible-playbook site.yml --limit k0s-w2
```

Beispiel, wenn der Controller auf `k0s-w2` läuft. Nicht im Lab ausgeführt, das Update lief in Kapitel 15 von Hand.

STOLPERFALLE

Die Rolle hält die Pakete mit `dpkg_selections: hold` fest. Für gehaltene Pakete verweigert `apt` eine andere Version, auch wenn sie ausdrücklich angefordert wird. Deshalb steht in den Paket-Aufgaben `allow_change_held_packages: true`. Ein `apt upgrade` von Hand bleibt blockiert, die Rolle darf die Version ändern. So soll es sein: Updates nur über die Variable. Ob ein Versionswechsel über die Rolle wirklich durchläuft, wurde im Lab nicht getestet.

WAS DU GELERNT HAST

- Jede Aufgabe der Rollen entspricht einem Schritt aus der Handinstallation.
- `--check --diff` zeigt vor dem Lauf, was sich ändern würde.
- Zwei Läufe mit `changed=0` beweisen Idempotenz.
- Die Rolle darf den Controller-Dienst nie starten oder stoppen, nur deaktivieren.
- Registrierung, Pools und die Controller-Datenbank sind einmalige Schritte außerhalb der Rollen.

KAPITEL 17

Überwachung: Was du messen und alarmieren solltest

Du kennst die beiden Metrik-Quellen für LINSTOR und DRBD, baust Prometheus, Alertmanager, Grafana, Loki und Alloy im Lab auf und verfolgst einen echten DRBD-Fehler vom Kernel-Log bis zur Alarm-Mail.

Dauer: 90 Minuten **Voraussetzung:** Den Cluster, Helm auf dem Mac und etwa 1,5 GiB freien Arbeitsspeicher über die drei Worker verteilt.

Bis hierher hast du Zustände mit `linstor` und `drbdadm` von Hand gelesen. Im Betrieb schaut niemand alle paar Minuten nach. Dieses Kapitel baut deshalb einen kleinen, vollständigen Monitoring-Stack ins Lab und richtet ihn auf LINSTOR und DRBD aus. Danach trennst du absichtlich eine DRBD-Verbindung und siehst, was Metriken, Logs und Alarme daraus machen.

Der Stack ist bewusst auf einen Cluster zugeschnitten. Wie Prometheus, Alertmanager und Grafana für viele Cluster und lange Aufbewahrung zusammenspielen, beschreibt das Buch „Kubernetes beobachten“.

17.1 Zwei Quellen

Quelle	Endpunkt	Was sie weiß
LINSTOR-Controller	`http://10.10.0.100:3370/metrics`	Den Soll-Zustand und das, was die Satellites melden: Nodes, Pools, Volumes, Fehlerberichte
drbd-reactor	`http://<node>:9942/metrics`	Den Ist-Zustand von DRBD auf genau diesem Node: Verbindungen, Quorum, Rolle, Out-of-Sync, Suspend

Wie in Kapitel 9 gilt: Der Controller kann veraltet sein, DRBD auf dem Node nicht. Deshalb brauchst du beide.

17.2 LINSTOR-Metriken

Der Controller liefert sie ohne weitere Einrichtung:

K0S-W2 Schritt 1

```
curl -s http://10.10.0.100:3370/metrics | grep -oE "^linstor_[a-z_]+" | sort -u
```

Alle LINSTOR-Metriken. Dazu kommen die üblichen JVM-Werte.

Die wichtigsten:

Metrik	Bedeutung
`linstor_node_state`	`2` = ONLINE, `0` = OFFLINE, `3` = VERSION_MISMATCH (Kapitel 15)
`linstor_volume_state`	`1` = UpToDate, `4` = Diskless, `5` = Inconsistent, `6` = Failed
`linstor_storage_pool_capacity_free_bytes`	Freier Platz je Pool und Node, auch für `pool_nvme`
`linstor_storage_pool_capacity_total_bytes`	Gesamtgröße. Bei disklosen Pools 0.
`linstor_error_reports_count`	Anzahl der Fehlerberichte

Die Zahlencodes erklärt der Endpunkt selbst in seinen `# HELP`-Zeilen:

K0S-W2 Schritt 2

```
curl -s http://10.10.0.100:3370/metrics | grep "^# HELP linstor_node_state"
```

Die Bedeutung der Zustandswerte.

DAS SOLLTEST DU SEHEN

```
# HELP linstor_node_state 0="OFFLINE", 1="CONNECTED", 2="ONLINE", 3="VERSION_MISMATCH",
  4="FULL_SYNC_FAILED", 5="AUTHENTICATION_ERROR", 6="UNKNOWN", ...
```

STOLPERFALLE

Der Endpunkt hängt an der VIP und damit am Storage-Netz. Ein Prometheus, der das Storage-Netz nicht erreicht, bekommt keine LINSTOR-Metriken. Bei einem Failover des Controllers fehlen sie für die Dauer des Wechsels, etwa 15 bis 40 Sekunden (Kapitel 6 und 10). Ein Alarm auf `up == 0` für dieses Ziel braucht deshalb ein `for` von einigen Minuten.

17.3 DRBD-Metriken mit drbd-reactor

drbd-reactor läuft auf den Controller-Kandidaten schon für die HA. Er hat ein Plugin, das DRBD-Zustände als Prometheus-Metriken ausgibt. Es braucht nur eine Konfigurationsdatei, und zwar auf **allen** Nodes mit DRBD, auch der Witness. Auf `k0s-ctrl` lief drbd-reactor bisher ohne Konfiguration.

DATEI `lab/monitoring/prometheus.toml`

```
[[prometheus]]
enums = true
address = ":9942"
```

`enums = true` gibt Zustände als eigene Zeitreihen je möglichem Wert aus, mit 1 für den aktuellen und 0 für alle anderen. Das lässt sich in PromQL einfach filtern.

MAC Schritt 3

```
scp lab/monitoring/prometheus.toml k0s-w1:/tmp/
ssh k0s-w1 'sudo install -m 644 /tmp/prometheus.toml /etc/drbd-reactor.d/'
ssh k0s-w1 'sudo systemctl reload drbd-reactor'
```

Für alle vier Nodes. `reload` liest die neue Datei, ohne den Promoter für `linstor_db` zu stören.

K0S-W2 Schritt 4

```
curl -s localhost:9942/metrics | grep -E "^drbd_(device_quorum|resource_suspended)"
```

Zwei der wichtigsten Werte.

DAS SOLLTEST DU SEHEN

```
drbd_device_quorum{name="pvc-55d1f760-...",volume="0",minor="1001"} 1
drbd_device_quorum{name="linstor_db",volume="0",minor="1000"} 1
drbd_resource_suspended{name="pvc-55d1f760-..."} 0
drbd_resource_suspended{name="linstor_db"} 0
```

Metrik	Bezug zu den Betriebskapiteln
`drbd_connection_state{drbd_connection_state="StandAlone"}`	Split-Brain (Kapitel 11, Versuch E)
`drbd_device_quorum`	Quorum-Verlust (Kapitel 11, Versuch C)
`drbd_resource_suspended`	Angehaltene Schreibvorgänge, die Anwendung hängt
`drbd_resource_role`	Primary oder Secondary. Entscheidet, ob ein Problem die Anwendung trifft.
`drbd_peerdevice_outofsync_bytes`	Ein Partner ist nicht auf Stand, etwa nach einem Node-Ausfall
`drbd_device_unintentionaldiskless`	Eine Kopie hat ihre Platte verloren (Kapitel 14)

17.4 Der Stack im Überblick

Baustein	Läuft als	Aufgabe
Prometheus 3.16	StatefulSet, 1 Pod	Sammelt Metriken ein, wertet Alarmregeln aus
Alertmanager 0.35	StatefulSet, 1 Pod	Gruppiert Alarme und verschickt Mails
Grafana 13.3	Deployment	Dashboards für Metriken und Logs
node-exporter, kube-state-metrics	DaemonSet, Deployment	Host- und Kubernetes-Metriken
Loki 3.6	StatefulSet, 1 Pod	Speichert Logs, durchsuchbar mit LogQL
Alloy 1.21	DaemonSet	Liest Journal und Pod-Logs auf jedem Worker, schickt sie an Loki
Mailpit 1.33	Deployment	Fängt die Alarm-Mails ab und zeigt sie im Browser

Die ersten vier kommen gemeinsam aus dem Helm-Chart `kube-prometheus-stack` 93.2.2, Loki und Alloy aus den Charts von Grafana. Im Lab brauchte der ganze Stack nach dem Start etwa 1,35 GiB Arbeitsspeicher.

STOLPERFALLE

LINSTOR und DRBD laufen auf den Hosts, nicht als Pods. Ein ServiceMonitor findet sie deshalb nicht. Prometheus bekommt sie als statische Ziele. Die Witness `k0s-ctrl` ist außerdem kein Kubernetes-Node: Weder node-exporter noch Alloy laufen dort. Ihre DRBD-Metriken erreicht Prometheus über `172.16.0.176`, weil sie kein Storage-Netz hat.

17.5 Prometheus, Alertmanager und Grafana

DATEI `lab/monitoring/kube-prometheus-stack-values.yaml`

```
# kube-prometheus-stack für das k0s-Lab: Prometheus, Alertmanager, Grafana,
# node-exporter und kube-state-metrics, klein dimensioniert für 3 x 4 GiB.

# k0s betreibt Controller-Komponenten außerhalb des Clusters auf k0s-ctrl.
# Prometheus erreicht sie nicht, also gar nicht erst danach suchen.
kubeControllerManager: {enabled: false}
kubeScheduler: {enabled: false}
kubeEtcd: {enabled: false}
kubeProxy: {enabled: false}
defaultRules:
  rules:
    etcd: false
    kubeControllerManager: false
    kubeSchedulerAlerting: false
    kubeSchedulerRecording: false
```

```
    kubeProxy: false

prometheus:
  prometheusSpec:
    retention: 7d
    resources:
      requests: {cpu: 100m, memory: 400Mi}
      limits: {memory: 1Gi}
    # Auch Regeln und Monitore finden, die nicht vom Chart selbst stammen.
    ruleSelectorNilUsesHelmValues: false
    serviceMonitorSelectorNilUsesHelmValues: false
    podMonitorSelectorNilUsesHelmValues: false
    storageSpec:
      volumeClaimTemplate:
        spec:
          storageClassName: linstor-replicated
          accessModes: [ReadWriteOnce]
          resources: {requests: {storage: 5Gi}}
    # LINSTOR und DRBD laufen auf den Hosts, nicht als Pods.
    # Deshalb statische Ziele statt ServiceMonitor.
    additionalScrapeConfigs:
      - job_name: linstor-controller
        static_configs:
          - targets: ["10.10.0.100:3370"]
            labels: {instance: linstor-controller}
        # Das Community-Dashboard 15917 erwartet den LINSTOR-Node als
        # exported_node (Label-Schema des Piraeus-Operators).
        metric_relabel_configs:
          - source_labels: [node]
            target_label: exported_node
      - job_name: drbd-reactor
        static_configs:
          - targets: ["10.10.0.11:9942"]
            labels: {instance: k0s-w1, node: k0s-w1}
          - targets: ["10.10.0.12:9942"]
            labels: {instance: k0s-w2, node: k0s-w2}
          - targets: ["10.10.0.13:9942"]
            labels: {instance: k0s-w3, node: k0s-w3}
          # Die Witness hat kein Storage-Netz.
          - targets: ["172.16.0.176:9942"]
            labels: {instance: k0s-ctrl, node: k0s-ctrl}

alertmanager:
  alertmanagerSpec:
    resources:
      requests: {cpu: 10m, memory: 50Mi}
      limits: {memory: 128Mi}
  config:
    global:
      smtp_smarthost: mailpit.monitoring.svc:1025
      smtp_from: alertmanager@k0s-lab.local
      smtp_require_tls: false
    route:
      receiver: storage-team
      group_by: [alertname, name]
      group_wait: 30s
      group_interval: 5m
      repeat_interval: 4h
      routes:
        # Watchdog feuert immer (Totmannschalter) und InfoInhibitor
        # unterdrückt Info-Alarme. Beides gehört nicht ins Postfach.
        - receiver: "null"
          matchers: ['alertname=~"Watchdog|InfoInhibitor"']
    receivers:
```

```
      - name: "null"
      - name: storage-team
        email_configs:
          - to: storage-team@k0s-lab.local
            send_resolved: true

grafana:
  adminPassword: lab-admin   # nur im Lab, sonst ein Secret
  persistence: {enabled: false}
  resources:
    requests: {cpu: 50m, memory: 128Mi}
    limits: {memory: 384Mi}
  additionalDataSources:
    - name: Loki
      type: loki
      uid: loki
      url: http://loki.monitoring.svc:3100
  sidecar:
    dashboards:
      enabled: true
      label: grafana_dashboard
      searchNamespace: monitoring

prometheus-node-exporter:
  resources:
    requests: {cpu: 10m, memory: 32Mi}
    limits: {memory: 64Mi}

kube-state-metrics:
  resources:
    requests: {cpu: 10m, memory: 64Mi}
    limits: {memory: 128Mi}

prometheusOperator:
  resources:
    requests: {cpu: 20m, memory: 64Mi}
    limits: {memory: 192Mi}
```

Die Stellen, die für dieses Lab wichtig sind:

- **k0s-Controller-Komponenten aus.** k0s betreibt API-Server, Scheduler, Controller-Manager und etcd auf `k0s-ctrl` außerhalb des Clusters. Der API-Server ist über den Service `kubernetes` erreichbar, die anderen nicht. Ohne die Schalter oben stünden drei Ziele dauerhaft auf `down` und feuerten Alarme.
- **`…SelectorNilUsesHelmValues: false`.** Sonst sieht Prometheus nur Regeln und Monitore mit dem Label des Helm-Releases. So findet er auch die eigene `PrometheusRule` weiter unten.
- **Statische Ziele mit Labels.** `instance` trägt den Node-Namen statt `IP:Port`. In Alarmen und Dashboards steht dann `k0s-w2` und nicht `10.10.0.12:9942`. Die Labels `node` und `exported_node` brauchst du erst für das Community-Dashboard weiter unten.
- **Route `null` für `Watchdog`.** Der Chart bringt einen Alarm mit, der **immer** feuert. Er ist ein Totmannschalter für die Alarmkette: Kommt er bei einem externen Dienst nicht mehr an, ist Prometheus oder Alertmanager kaputt. Ins Postfach des Storage-Teams gehört er nicht.

MAC Schritt 5

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm repo add grafana https://grafana.github.io/helm-charts
helm repo update
kubectl create namespace monitoring
```

Einmalig.

DATEI `lab/monitoring/mailpit.yaml`

```
# Mailpit fängt alle Mails ab und zeigt sie in einer Weboberfläche.
# Im Lab der Empfänger für Alertmanager, in Produktion ein echter Mailserver.
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mailpit
  namespace: monitoring
spec:
  replicas: 1
  selector:
    matchLabels: {app: mailpit}
  template:
    metadata:
      labels: {app: mailpit}
    spec:
      containers:
        - name: mailpit
          image: docker.io/axllent/mailpit:v1.33.1
          ports:
            - {name: smtp, containerPort: 1025}
            - {name: http, containerPort: 8025}
          resources:
            requests: {cpu: 10m, memory: 32Mi}
            limits: {memory: 128Mi}
---
apiVersion: v1
kind: Service
metadata:
  name: mailpit
  namespace: monitoring
spec:
  selector: {app: mailpit}
  ports:
    - {name: smtp, port: 1025}
    - {name: http, port: 8025}
```

Mailpit nimmt auf Port 1025 jede Mail an und zeigt sie auf Port 8025. In Produktion steht in `smtp_smarthost` dein Mailserver, oder du nimmst einen anderen Empfänger wie Teams, Slack oder PagerDuty.

MAC Schritt 6

```
kubectl apply -f lab/monitoring/mailpit.yaml
helm install kps prometheus-community/kube-prometheus-stack --version 93.2.2 \
  -n monitoring -f lab/monitoring/kube-prometheus-stack-values.yaml --wait --timeout 10m
```

Dauert im Lab etwa fünf Minuten. Die meiste Zeit geht für die Images und für das Anlegen des PVC drauf.

MAC Schritt 7

```
kubectl -n monitoring get pvc
```

Prometheus speichert auf LINSTOR.

DAS SOLLTEST DU SEHEN

```
NAME                                    STATUS   CAPACITY   STORAGECLASS
prometheus-kps-....-prometheus-0        Bound    5Gi        linstor-replicated
```

WARUM DAS WICHTIG IST

Hier überwacht etwas den Speicher, auf dem es selbst liegt. Solange eine Kopie ein Quorum hat, schreibt Prometheus weiter, und genau das willst du sehen. Fällt die Ressource von Prometheus aber selbst aus, wird das Monitoring blind, wenn du es am dringendsten brauchst. In Produktion liegt das Monitoring deshalb auf lokalem Speicher, in einem anderen Cluster oder schickt zumindest seine Alarme über einen Weg, der nicht vom überwachten Speicher abhängt. Für ein Lab ist `linstor-replicated` in Ordnung und zeigt nebenbei, dass die StorageClass trägt.

Für den Zugriff vom Mac reichen Port-Forwards:

MAC Schritt 8

```
kubectl -n monitoring port-forward svc/kps-kube-prometheus-stack-prometheus 9090 &
kubectl -n monitoring port-forward svc/kps-kube-prometheus-stack-alertmanager 9093 &
kubectl -n monitoring port-forward svc/kps-grafana 3000:80 &
kubectl -n monitoring port-forward svc/mailpit 8025 &
```

Prometheus unter `localhost:9090`, Alertmanager unter `:9093`, Grafana unter `:3000` (Benutzer `admin`, Passwort aus der Values-Datei), Mailpit unter `:8025`.

Unter **Status → Target health** zeigt Prometheus, ob er alle Ziele erreicht:

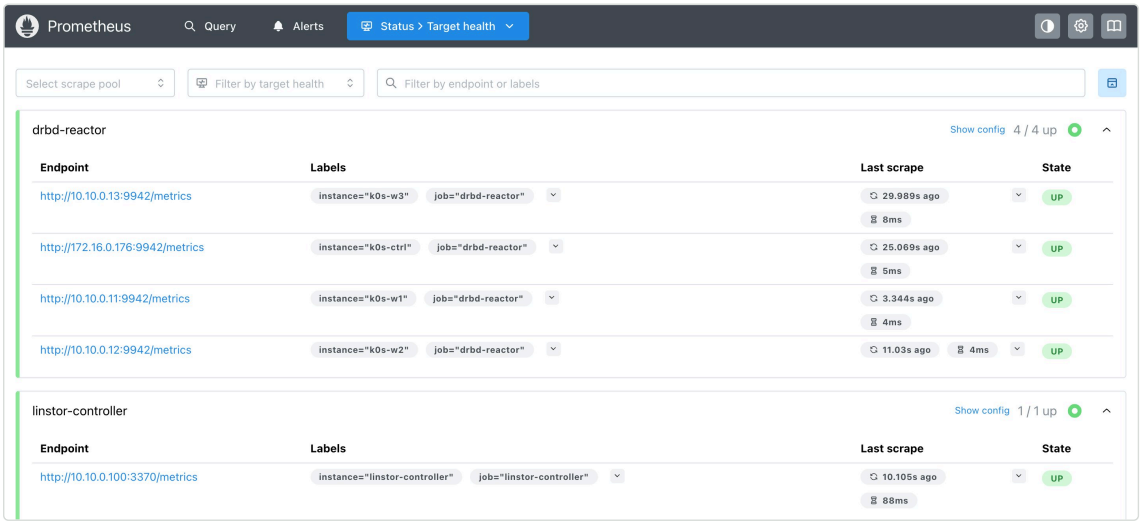


Abbildung 1: Die vier drbd-reactor-Instanzen und der LINSTOR-Controller hinter der VIP sind `UP`. Die Witness `k0s-ctrl` wird über das Management-Netz abgefragt.

17.6 Logs mit Loki und Alloy

Metriken sagen dir, **dass** etwas nicht stimmt. Warum, steht meist im Log: im Journal von `linstor-satellite` und `drbd-reactor` und vor allem in den Kernelmeldungen von DRBD. Loki speichert diese Logs, Alloy sammelt sie ein.

DATEI `lab/monitoring/loki-values.yaml`

```
# Loki als ein einziger Pod mit Dateisystem-Speicher. Reicht für ein Lab
# und kleine Cluster. Für mehr Last: Objektspeicher und SimpleScalable.
deploymentMode: SingleBinary

loki:
  auth_enabled: false
  commonConfig:
    replication_factor: 1
  storage:
    type: filesystem
  schemaConfig:
    configs:
      - from: "2026-01-01"
        store: tsdb
        object_store: filesystem
        schema: v13
        index: {prefix: index_, period: 24h}
  limits_config:
    retention_period: 168h
  compactor:
    retention_enabled: true
    delete_request_store: filesystem

singleBinary:
  replicas: 1
  resources:
```

```
    requests: {cpu: 50m, memory: 256Mi}
    limits: {memory: 768Mi}
  persistence:
    enabled: true
    size: 5Gi
    storageClass: linstor-replicated

# Alles abschalten, was ein Single-Binary-Lab nicht braucht.
gateway: {enabled: false}
chunksCache: {enabled: false}
resultsCache: {enabled: false}
lokiCanary: {enabled: false}
test: {enabled: false}
read: {replicas: 0}
write: {replicas: 0}
backend: {replicas: 0}
```

Ein einziger Loki-Pod mit Dateisystem-Speicher auf einem LINSTOR-PVC. Die Caches und das Gateway des Charts sind für ein Lab überdimensioniert und deshalb aus.

DATEI `lab/monitoring/alloy-values.yaml`

```
# Alloy als DaemonSet: liest auf jedem Worker das Journal (LINSTOR-Satellite,
# drbd-reactor, Kernelmeldungen von DRBD) und die Logs aller Pods.
alloy:
  mounts:
    varlog: true          # /var/log/journal
    extra:
      - name: machine-id
        mountPath: /etc/machine-id
        readOnly: true
  # Node-Name, damit jeder Alloy-Pod nur die Pods seines Nodes liest.
  extraEnv:
    - name: NODE_NAME
      valueFrom:
        fieldRef: {fieldPath: spec.nodeName}
  resources:
    requests: {cpu: 20m, memory: 64Mi}
    limits: {memory: 256Mi}
  configMap:
    content: |
      // --- Journal der Hosts ------------------------------------------
      loki.relabel "journal" {
        forward_to = []
        rule {
          source_labels = ["__journal__systemd_unit"]
          target_label  = "unit"
        }
        rule {
          source_labels = ["__journal__hostname"]
          target_label  = "node"
        }
        rule {
          source_labels = ["__journal__transport"]
          target_label  = "transport"
        }
        rule {
          source_labels = ["__journal_priority_keyword"]
          target_label  = "level"
        }
      }
```

```
      loki.source.journal "host" {
        max_age       = "12h"
        relabel_rules = loki.relabel.journal.rules
        labels        = {job = "journal"}
        forward_to    = [loki.write.lab.receiver]
      }

      // --- Pod-Logs über die Kubernetes-API ------------------------------
      discovery.kubernetes "pods" {
        role = "pod"
        selectors {
          role  = "pod"
          field = "spec.nodeName=" + sys.env("NODE_NAME")
        }
      }

      discovery.relabel "pods" {
        targets = discovery.kubernetes.pods.targets
        rule {
          source_labels = ["__meta_kubernetes_namespace"]
          target_label  = "namespace"
        }
        rule {
          source_labels = ["__meta_kubernetes_pod_name"]
          target_label  = "pod"
        }
        rule {
          source_labels = ["__meta_kubernetes_pod_container_name"]
          target_label  = "container"
        }
        rule {
          source_labels = ["__meta_kubernetes_pod_node_name"]
          target_label  = "node"
        }
      }

      loki.source.kubernetes "pods" {
        targets    = discovery.relabel.pods.output
        forward_to = [loki.write.lab.receiver]
      }

      loki.write "lab" {
        endpoint {
          url = "http://loki.monitoring.svc:3100/loki/api/v1/push"
        }
      }

controller:
  type: daemonset
  # Das Journal ist nur mit der machine-id des Hosts lesbar.
  volumes:
    extra:
      - name: machine-id
        hostPath: {path: /etc/machine-id}
```

Zwei Quellen. `loki.source.journal` liest das Journal des Hosts und macht aus den Journal-Feldern die Labels `unit`, `node` und `transport`. DRBD schreibt in den Kernel-Ring-Buffer, diese Zeilen haben `transport="kernel"`. `loki.source.kubernetes` holt die Pod-Logs über die Kubernetes-API. Der Selektor auf `NODE_NAME` sorgt dafür, dass jeder Alloy-Pod nur die Pods seines eigenen Nodes liest.

MAC Schritt 9

```
helm install loki grafana/loki --version 7.6.0 \
  -n monitoring -f lab/monitoring/loki-values.yaml --wait --timeout 10m
helm install alloy grafana/alloy --version 1.15.0 \
  -n monitoring -f lab/monitoring/alloy-values.yaml --wait --timeout 5m
```

Grafana kennt Loki schon, die Datenquelle steht unter `additionalDataSources` in den Values von kube-prometheus-stack.

MAC Schritt 10

```
kubectl -n monitoring port-forward svc/loki 3100 &
curl -s localhost:3100/loki/api/v1/label/unit/values
```

Welche systemd-Units bei Loki angekommen sind.

DAS SOLLTEST DU SEHEN

```
{"status":"success","data":["ModemManager.service", ..., "drbd-reactor.service",
 ..., "k0sworker.service","linstor-controller.service","linstor-
satellite.service", ...]}
```

In Grafana suchst du Logs unter **Explore** mit der Datenquelle **Loki**. Eine Abfrage für alle Zustandswechsel von DRBD:

DATEI LogQL

```
{job="journal", transport="kernel"} |= "drbd" |~ "conn\\(|quorum|susp"
```

Der Teil in geschweiften Klammern wählt die Log-Ströme über Labels aus und ist schnell. `|=` filtert auf einen festen Text, `|~` auf einen regulären Ausdruck. Diese Filter durchsuchen den Text und sind langsamer, deshalb immer erst mit Labels eingrenzen.

STOLPERFALLE

Im Journal des Controllers steht bei jedem Scrape eine Zeile `REST/API RestClient(...; 'Prometheus/3.16.0')/metrics` und dazu `Assembled error reports`. Bei einem Scrape alle 30 Sekunden verdrängen diese Zeilen alles andere. Das Dashboard unten filtert sie mit `!~ "REST/API|/metrics|Assembled error reports"` heraus.

17.7 Alarmregeln

DATEI lab/monitoring/linstor-alerts.yaml

```
groups:
  - name: linstor-drbd
    rules:
      # LINSTOR-Controller (Metriken vom Controller über die VIP)
```

```
      - alert: LinstorNodeNichtOnline
        expr: linstor_node_state != 2
        for: 5m
        labels: {severity: warning}
        annotations:
          summary: "LINSTOR-Node {{ $labels.node }} ist nicht ONLINE (Zustand {{ $value }})"
          description: "0=OFFLINE, 3=VERSION_MISMATCH. Satellite und Storage-Netz des Nodes prüfen."
      - alert: LinstorPoolFastVoll
        expr: |
          linstor_storage_pool_capacity_free_bytes
            / (linstor_storage_pool_capacity_total_bytes > 0) < 0.15
        for: 15m
        labels: {severity: warning}
        annotations:
          summary: "Pool {{ $labels.storage_pool }} auf {{ $labels.node }} hat weniger als 15 % frei"
      - alert: LinstorFehlerberichte
        expr: increase(linstor_error_reports_count[30m]) > 0
        labels: {severity: info}
        annotations:
          summary: "Neue LINSTOR-Fehlerberichte: linstor error-reports list"
      # DRBD (Metriken von drbd-reactor auf jedem Node)
      - alert: DrbdVerbindungGetrennt
        expr: drbd_connection_state{drbd_connection_state=~"StandAlone|Connecting|Unconnected"} == 1
        for: 5m
        labels: {severity: critical}
        annotations:
          summary: "DRBD {{ $labels.name }}: Verbindung zu {{ $labels.conn_name }} ist
{{ $labels.drbd_connection_state }}"
          description: "StandAlone bedeutet fast immer Split-Brain (Kapitel 11)."
      # Nur dort kritisch, wo die Ressource Primary ist: Dort hängt die
      # Anwendung. Ein Secondary ohne Quorum meldet DrbdVerbindungGetrennt.
      - alert: DrbdOhneQuorum
        expr: |
          drbd_device_quorum == 0
            and on (name, instance) drbd_resource_role{drbd_resource_role="Primary"} == 1
        for: 1m
        labels: {severity: critical}
        annotations:
          summary: "DRBD {{ $labels.name }} auf {{ $labels.instance }} hat kein Quorum"
      - alert: DrbdSuspendiert
        expr: |
          drbd_resource_suspended == 1
            and on (name, instance) drbd_resource_role{drbd_resource_role="Primary"} == 1
        for: 1m
        labels: {severity: critical}
        annotations:
          summary: "DRBD {{ $labels.name }} hält alle Schreibvorgänge an"
      - alert: DrbdOutOfSync
        expr: drbd_peerdevice_outofsync_bytes > 0
        for: 30m
        labels: {severity: warning}
        annotations:
          summary: "DRBD {{ $labels.name }}: {{ $value | humanize1024 }}B nicht synchron mit
{{ $labels.conn_name }}"
```

Die Werte für `for` sind bewusst gewählt:

- `DrbdVerbindungGetrennt` wartet 5 Minuten. Ein Neustart eines Nodes dauert etwa 30 Sekunden (Kapitel 10) und soll niemanden wecken.

- `DrbdOhneQuorum` und `DrbdSuspendiert` warten nur 1 Minute. In diesem Zustand steht die Anwendung. Beide gelten nur dort, wo die Ressource Primary ist. Warum, zeigt der Versuch weiter unten.
- `DrbdOutOfSync` wartet 30 Minuten. Ein Resync nach einem Neustart ist normal, ein Resync, der nicht fertig wird, nicht.

Der Prometheus-Operator liest Regeln aus Objekten vom Typ `PrometheusRule`. Die Datei oben ist eine gewöhnliche Regeldatei, damit `promtool` sie direkt prüfen kann. Für den Cluster wird sie in das Objekt eingepackt:

MAC Schritt 11

```
{ printf 'apiVersion: monitoring.coreos.com/v1\nkind: PrometheusRule\n'
  printf 'metadata:\n  name: linstor-drbd\n  namespace: monitoring\nspec:\n'
  sed 's/^/  /' lab/monitoring/linstor-alerts.yaml; } | kubectl apply -f -
```

Eine Quelle für beides: Der Test prüft genau die Regeln, die im Cluster laufen. Nach etwa einer Minute stehen sie unter **Alerts** in Prometheus.

17.8 Die Regeln testen

`promtool` prüft Regeln auf Syntax und spielt Testfälle mit erfundenen Zeitreihen durch. Ein Ausschnitt des Tests:

DATEI `lab/monitoring/linstor-alerts-test.yaml (Ausschnitt)`

```
    input_series:
      - series: 'drbd_connection_state{name="pvc-a",conn_name="k0s-
w3",drbd_connection_state="StandAlone"}'
        values: '0 0 1x10'
      # Im Lab beobachtet: Der getrennte Secondary verliert das Quorum,
      # die Anwendung auf dem Primary läuft weiter. Kein Alarm.
      - series: 'drbd_device_quorum{name="pvc-b",instance="k0s-w2"}'
        values: '1 1 0x5'
      - series: 'drbd_resource_role{name="pvc-b",instance="k0s-w2",drbd_resource_role="Primary"}'
        values: '0x8'
    alert_rule_test:
      - eval_time: 9m
        alertname: DrbdVerbindungGetrennt
        exp_alerts:
          - exp_labels: {severity: critical, name: pvc-a, conn_name: k0s-w3, drbd_connection_state:
StandAlone}
```

Ab Minute 2 ist die Verbindung `StandAlone`. Nach 5 Minuten, bei Minute 9 also sicher, muss der Alarm feuern. Weitere Fälle prüfen Quorum-Verlust auf einem Primary (`pvc-a`, Alarm) und auf einem Secondary (`pvc-b`, kein Alarm) sowie einen Pool mit 10 % frei. Der disklose Pool mit Gesamtgröße 0 darf dabei keinen Alarm auslösen.

MAC Schritt 12

```
kubectl create namespace promtool
kubectl -n promtool create configmap regeln \
  --from-file=lab/monitoring/linstor-alerts.yaml \
  --from-file=lab/monitoring/linstor-alerts-test.yaml
kubectl -n promtool run pt --rm -i --restart=Never \
  --image=quay.io/prometheus/prometheus:v3.15.1 --overrides='{"spec":{"containers":[{
    "name":"pt","image":"quay.io/prometheus/prometheus:v3.15.1",
    "command":["sh","-c","cd /r && promtool check rules linstor-alerts.yaml && promtool test rules
linstor-alerts-test.yaml"],
    "volumeMounts":[{"name":"r","mountPath":"/r"}]}],
    "volumes":[{"name":"r","configMap":{"name":"regeln"}}]}}'
kubectl delete namespace promtool
```

Wer `promtool` lokal hat, ruft die beiden Befehle direkt auf. Im Lab läuft es in einem Pod mit dem Prometheus-Image.

DAS SOLLTEST DU SEHEN

```
Checking linstor-alerts.yaml
  SUCCESS: 7 rules found

  SUCCESS
```

17.9 Das Dashboard

Für den täglichen Blick auf LINSTOR und DRBD liegt ein eigenes Dashboard im Repository: `lab/monitoring/linstor-dashboard.json`. Grafana lädt es über den Dashboard-Sidecar aus jeder ConfigMap mit dem Label `grafana_dashboard`:

MAC Schritt 13

```
kubectl -n monitoring create configmap linstor-dashboard \
  --from-file=lab/monitoring/linstor-dashboard.json
kubectl -n monitoring label configmap linstor-dashboard grafana_dashboard=1
```

Nach etwa 30 Sekunden steht es unter **Dashboards** als „LINSTOR und DRBD“. Änderungen an der Datei spielst du genauso ein, mit `--dry-run=client -o yaml | kubectl apply -f -`.

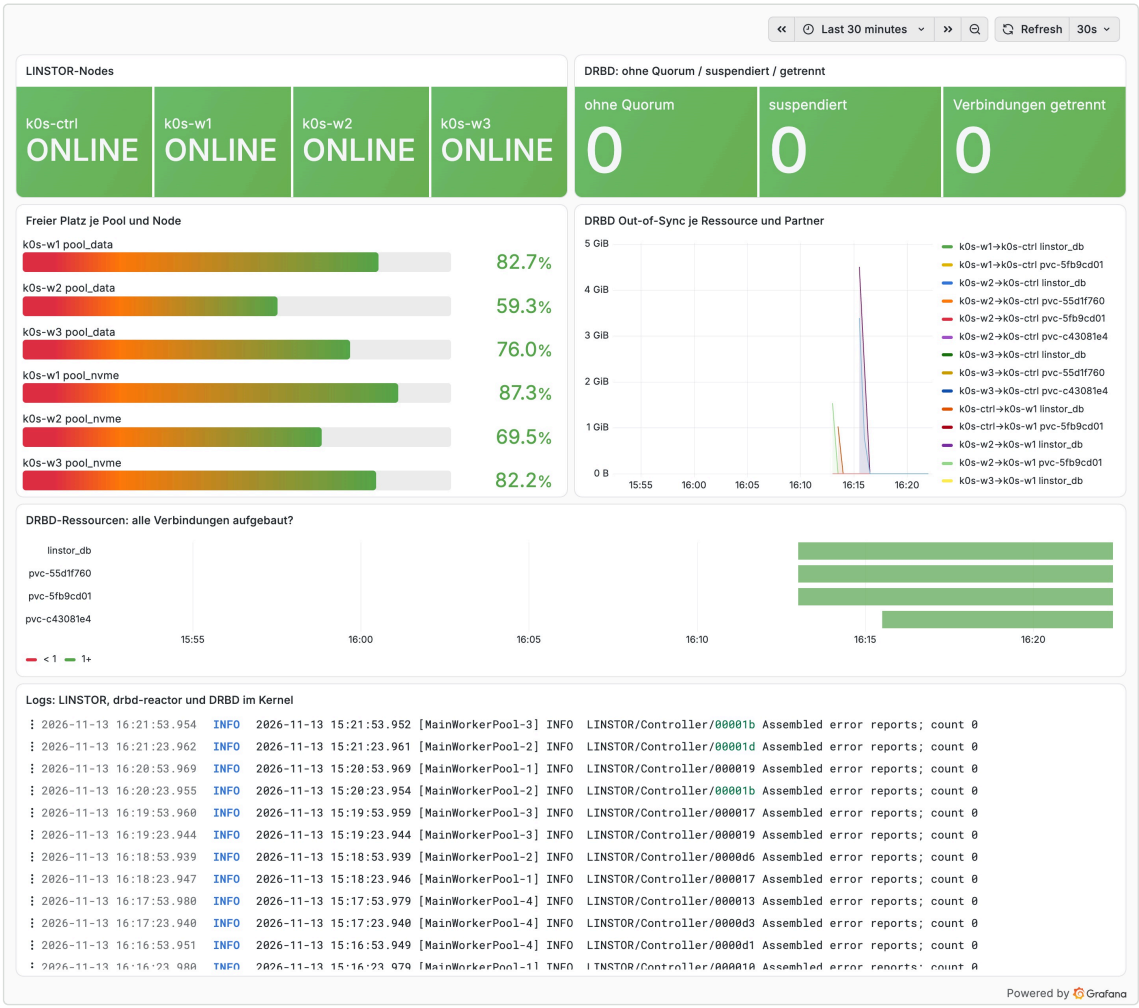


Abbildung 2: Das Dashboard im Normalzustand. Die Spitzen bei Out-of-Sync um 16:15 sind die Erstsynchronisation der gerade angelegten PVCs von Prometheus und Loki, kein Fehler.

Die sechs Panels beantworten die Fragen aus den Betriebskapiteln:

Panel	Frage
LINSTOR-Nodes	Sind alle Satellites ONLINE? (Kapitel 10, 15)
ohne Quorum / suspendiert / getrennt	Hängt gerade irgendwo eine Anwendung? Alles außer 0 ist rot.
Freier Platz	Wann wird ein Pool knapp? Rot unter 15 % (Kapitel 12)
Out-of-Sync	Läuft ein Resync, und wird er fertig?
Verbindungen aufgebaut?	Eine Zeile je Ressource. Rot, sobald **irgendeine** Verbindung fehlt.
Logs	Was LINSTOR und drbd-reactor dazu sagen

STOLPERFALLE

Die PVC-Namen sind 40 Zeichen lang. In Legenden passen sie nicht. Das Dashboard kürzt sie mit `label_replace(..., "name", "$1", "name", "(pvc-[0-9a-f]{8}).*")` auf die ersten acht Stellen. Das reicht, um sie in `linstor resource list` wiederzufinden.

17.10 Fertige Dashboards

Zu kube-prometheus-stack gehören gut zwei Dutzend Dashboards. Für dieses Buch sind zwei interessant: „Node Exporter / Nodes“ für CPU, Speicher und Platten der Worker und „Kubernetes / Persistent Volumes“ für die Belegung der PVCs aus Sicht von Kubernetes.

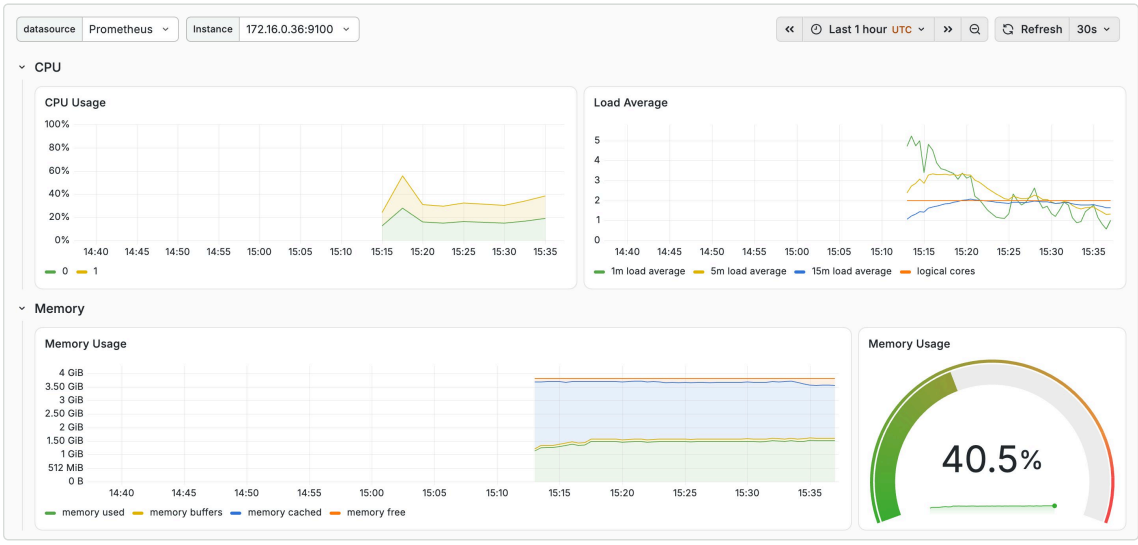


Abbildung 3: „Node Exporter / Nodes“ für `k0s-w2`. Die Geräte `dm-*` unter **Disk I/O** sind die LVM-Volumes, auf denen DRBD liegt.

Ein eigenes Dashboard für k0s gibt es nicht. Die Kubernetes-Dashboards funktionieren, nur die für Scheduler, Controller-Manager und etcd bleiben leer, weil diese Komponenten nicht abgefragt werden.

Für LINSTOR und DRBD gibt es auf grafana.com das Dashboard 15917 „LINSTOR / DRBD“ vom Piraeus-Projekt. Es ist für das Label-Schema des Piraeus-Operators gebaut: DRBD-Metriken mit `node`, LINSTOR-Metriken mit `exported_node`. Deshalb stehen in den Values oben die zusätzlichen Labels an den drbd-reactor-Zielen und die Regel unter `metric_relabel_configs` für den Controller.

MAC Schritt 14

```
curl -sL https://grafana.com/api/dashboards/15917/revisions/latest/download \
  | sed 's/${ds_prometheus}/prometheus/g' > lab/monitoring/dashboard-15917-linstor-drbd.json
kubectl -n monitoring create configmap dashboard-15917 \
  --from-file=lab/monitoring/dashboard-15917-linstor-drbd.json
kubectl -n monitoring label configmap dashboard-15917 grafana_dashboard=1
```

Der Sidecar löst Platzhalter für Datenquellen nicht auf. `sed` ersetzt sie durch die UID der Prometheus-Datenquelle.

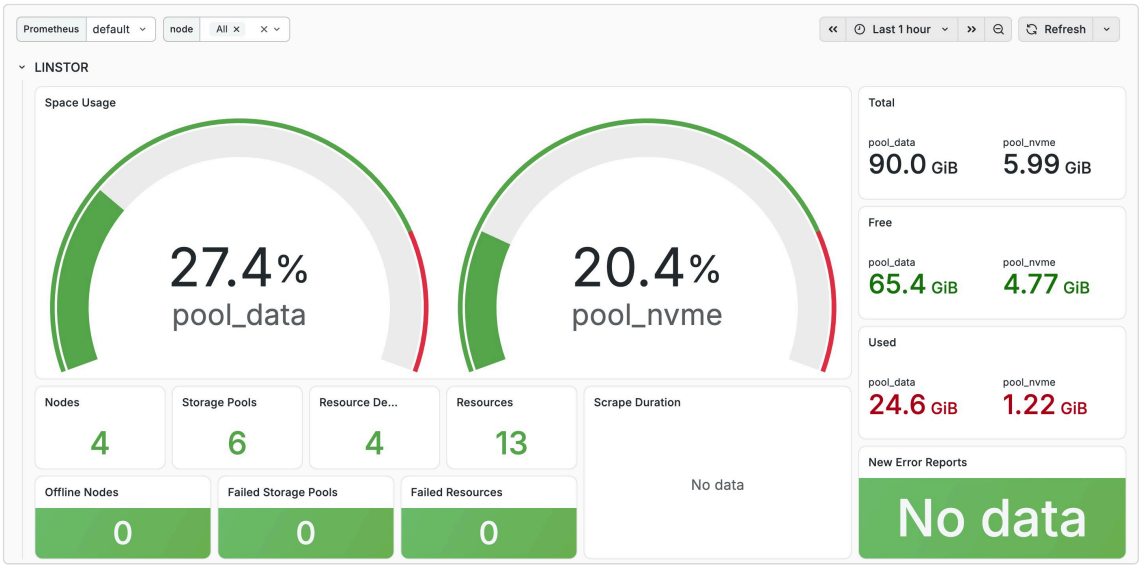


Abbildung 4: Dashboard 15917 mit den angepassten Labels. „Scrape Duration“ und „New Error Reports“ bleiben leer, weil sie Metriken erwarten, die nur der Piraeus-Operator erzeugt.

WARUM DAS WICHTIG IST

Ein Community-Dashboard ist ein guter Startpunkt, aber kein Vertrag. Es passt zu der Umgebung, in der es gebaut wurde. Prüfe nach dem Import jedes Panel: Ein leeres Panel ist harmlos, ein Panel, das falsch filtert und deshalb grün zeigt, ist es nicht.

17.11 Ein Alarm von Anfang bis Ende

Jetzt wird es ernst. Auf `k0s-w2` trennst du die DRBD-Verbindung der Postgres-Ressource aus Kapitel 8 zu `k0s-w3`. Postgres läuft auf `k0s-w3`, dort ist die Ressource Primary.

K0S-W2 Schritt 15

```
sudo drbdadm disconnect pvc-55d1f760-7f7b-4d71-8a03-0e1e8cd11250:k0s-w3
sudo drbdadm status pvc-55d1f760-7f7b-4d71-8a03-0e1e8cd11250
```

Nur diese eine Verbindung. LINSTOR baut sie nicht von selbst wieder auf.

DAS SOLLTEST DU SEHEN

```
pvc-55d1f760-... role:Secondary suspended:quorum
  disk:Outdated quorum:no open:no blocked:upper
  k0s-ctrl role:Secondary
    peer-disk:Diskless peer-client:yes
  k0s-w3 connection:StandAlone
```

Beim Trennen markiert DRBD die Kopie auf `k0s-w2` über einen clusterweiten Zustandswechsel als `Outdated`. Die Witness reicht die Nachricht weiter. Eine Kopie, die weiß, dass es irgendwo neuere Daten gibt, bekommt kein Quorum, auch wenn sie die Witness noch sieht (Kapitel 11). `k0s-w2` hält deshalb Schreibvorgänge an. Auf `k0s-w3` dagegen bleibt alles beim Alten: Primary, `UpToDate`, Quorum über die Witness. Postgres schreibt weiter, und jeder geschriebene Block landet als Out-of-Sync für `k0s-w2` in der Bitmap.

MAC Schritt 16

```
kubectl -n demo exec pg-0 -- psql -U postgres -c \
  "create table last(x text); insert into last select repeat(md5(random()::text),30) from
generate_series(1,200000);"
```

Etwa 200 MB Schreiblast, damit Out-of-Sync sichtbar wächst.

Der Ablauf im Lab, aus `lab/protokolle/17-alarm.log` und den Zeitstempeln von Prometheus und Mailpit:

Uhrzeit	Ereignis
16:22:59	`drbdadm disconnect` auf `k0s-w2`
16:23:00	Kernel auf beiden Seiten: `conn( Connected -> Disconnecting )`, auf `k0s-w2` `quorum( yes -> no )`
16:23:18	`DrbdVerbindungGetrennt` für `k0s-w3` → `k0s-w2` (`Connecting`) wird **pending**
16:23:48	Dieselbe Regel für die Gegenrichtung (`StandAlone`), dazu `DrbdOhneQuorum`, `DrbdSuspendiert` und `DrbdOutOfSync`: **pending**
16:25:18	Mails zu `DrbdOhneQuorum` und `DrbdSuspendiert`: 1 Minute `for` plus 30 Sekunden `group_wait`
16:28:48	Mail zu `DrbdVerbindungGetrennt` mit beiden Richtungen in **einer** Mail
16:31:43	`drbdadm connect` auf `k0s-w2`, Resync von etwa 410 MiB
16:32:10	beide Seiten wieder `UpToDate`
16:33:48, 16:35:18	Mails mit `[RESOLVED]`

`DrbdOutOfSync` wurde nie zugestellt. Der Resync war nach weniger als 30 Sekunden fertig, lange vor den 30 Minuten `for`. Genau so soll es sein.

Prometheus: pending

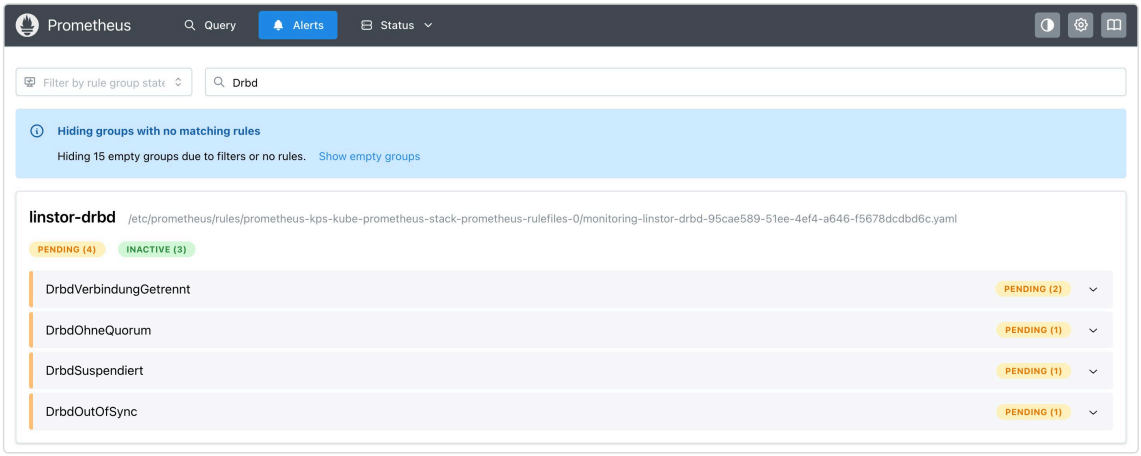


Abbildung 5: Prometheus unter **Alerts** kurz nach der Trennung. Die Bedingungen sind erfüllt, aber die Zeit in `for` ist noch nicht um.

Ein Alarm hat drei Zustände. **Inactive**: Der Ausdruck liefert nichts. **Pending**: Er liefert etwas, aber kürzer als `for`. **Firing**: länger als `for`. Erst dann geht er an Alertmanager. Bei einem Node-Neustart von 30 Sekunden siehst du hier kurz **pending** und dann wieder **inactive**, und niemand wird geweckt.

Grafana: was ist los?



Abbildung 6: Dasselbe Dashboard während der Störung. Oben rechts drei rote Kacheln, in der Zeitleiste ist `pvc-55d1f760` ab 16:23 rot, die Out-of-Sync-Kurve steigt auf etwa 410 MiB.

Die LINSTOR-Nodes bleiben grün. Aus Sicht des Controllers ist jeder Satellite erreichbar. Dass eine einzelne DRBD-Verbindung fehlt, weiß nur drbd-reactor. Das ist der Grund für zwei Quellen.

Loki: warum?

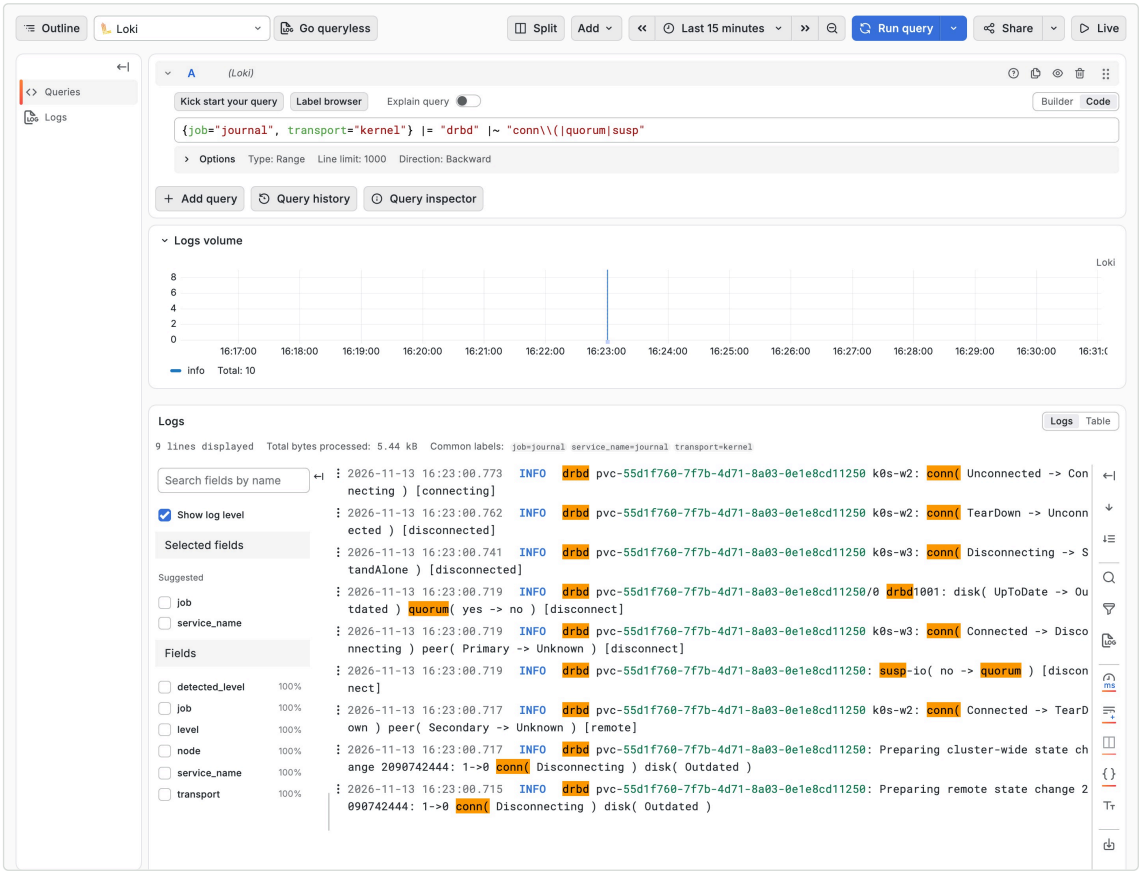


Abbildung 7: Explore mit der LogQL-Abfrage von oben. Die Kernelmeldungen beider Nodes zeigen die Trennung, `quorum( yes -> no )` und `susp-io( no -> quorum )` auf `k0s-w2`.

Die Zeile `drbd1001: disk( UpToDate -> Outdated ) quorum( yes -> no )` erklärt beide Quorum-Alarme. Auf einem Node, auf dem DRBD-Meldungen nicht im Journal ankommen, sähest du hier nichts. Prüfe deshalb einmal mit einer harmlosen Aktion, dass die Kernelzeilen wirklich in Loki landen, bevor du dich im Ernstfall darauf verlässt.

Alertmanager und Mail

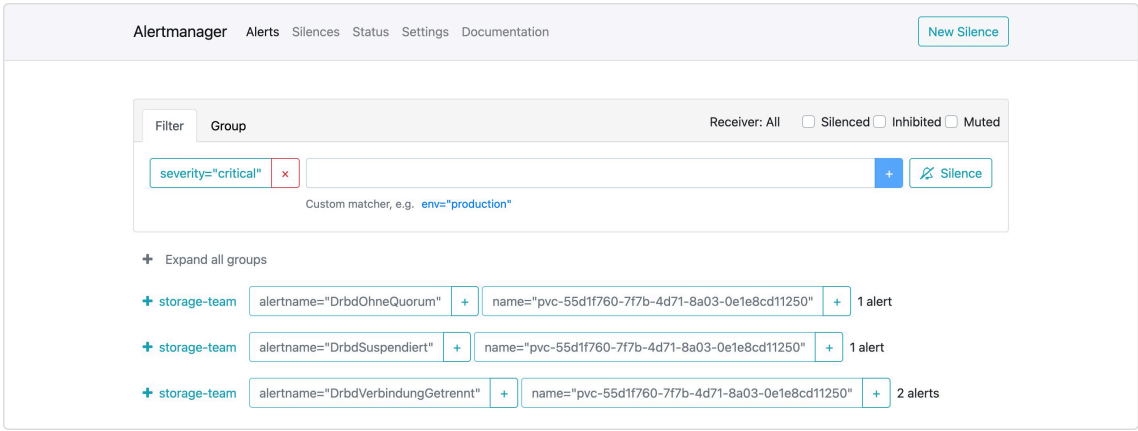


Abbildung 8: Alertmanager gruppiert nach `alertname` und `name`. Die beiden Richtungen der getrennten Verbindung sind **eine** Gruppe mit zwei Alarmen.

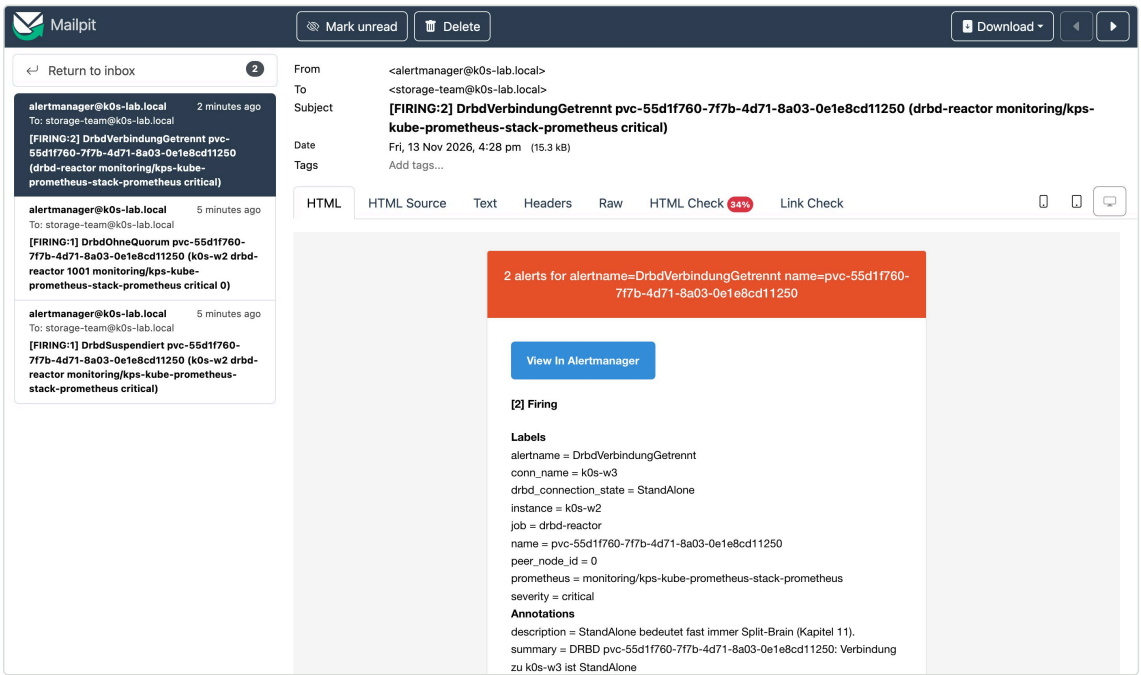


Abbildung 9: Die Mail zu `DrbdVerbindungGetrennt` in Mailpit. Labels und Annotationen aus der Regel stehen im Text, `summary` nennt Ressource und Partner.

`group_by: [alertname, name]` in den Values bündelt alles, was zu einer Ressource und einer Ursache gehört. Ohne Gruppierung kämen bei einem Node-Ausfall eine Mail je Ressource und Richtung. Bei 50 Volumes sind das 100 Mails in derselben Minute.

K0S-W2 Schritt 17

```
sudo drbdadm connect pvc-55d1f760-7f7b-4d71-8a03-0e1e8cd11250:k0s-w3
```

Verbindung wiederherstellen. DRBD gleicht nur die Blöcke aus der Bitmap ab, nicht das ganze Volume.

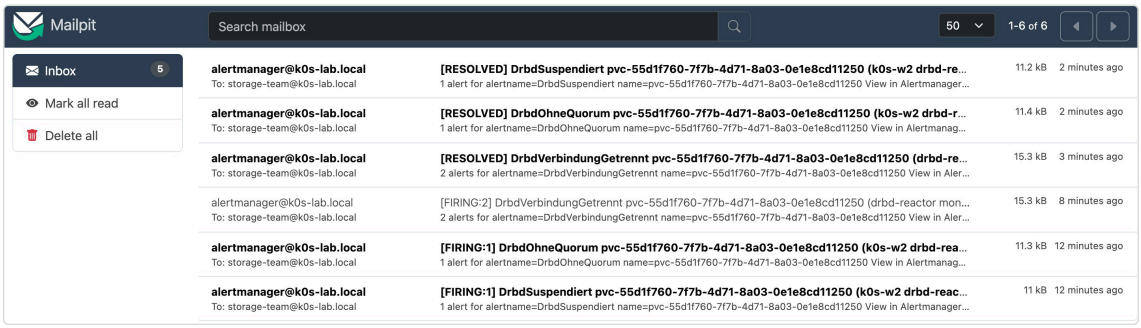


Abbildung 10: Der Posteingang nach dem Versuch. Jede Gruppe meldet sich einmal mit `FIRING` und einmal mit `RESOLVED`.

Die Entwarnung kommt nicht sofort, sondern nach bis zu `group_interval` (5 Minuten). So entsteht bei einer flatternden Verbindung keine Mailflut.

17.12 Was der Versuch über die Regeln lehrt

In der ersten Fassung lauteten die beiden Regeln einfach `drbd_device_quorum == 0` und `drbd_resource_suspended == 1`. Im Versuch feuerten sie beide als `critical`, für `k0s-w2`. Dort ist die Ressource aber Secondary und nicht geöffnet. Die Anwendung lief auf `k0s-w3` die ganze Zeit weiter. Die Beschreibung „die Anwendung steht“ war also falsch.

Die Verbindung zur Rolle stellt `and on (name, instance)` her:

DATEI PromQL

```
drbd_device_quorum == 0
  and on (name, instance) drbd_resource_role{drbd_resource_role="Primary"} == 1
```

`and` behält nur die Zeitreihen links, zu denen es rechts eine mit denselben Werten für `name` und `instance` gibt. Übrig bleiben Geräte ohne Quorum auf dem Node, auf dem die Ressource Primary ist. Der getrennte Secondary meldet sich trotzdem, über `DrbdVerbindungGetrennt` mit 5 Minuten Geduld.

Der Fall steht jetzt als `pvc-b` im Test. Wer die Regel später vereinfacht, merkt es an `promtool test rules`.

WARUM DAS WICHTIG IST

Ein Alarm, der `critical` sagt und sich als harmlos herausstellt, kostet beim nächsten Mal Vertrauen. Ein solcher Versuch im Lab ist die billigste Stelle, solche Regeln zu finden. Die Screenshots oben zeigen noch die alte Fassung.

WANN THANOS ODER MIMIR?

Ein Prometheus mit 7 Tagen Aufbewahrung reicht für einen Cluster. Thanos oder Mimir lohnen sich, wenn du eine gemeinsame Sicht über mehrere Cluster brauchst, Metriken über Monate behalten willst oder zwei Prometheus-Instanzen als HA-Paar deduplizieren musst. Beide legen die Daten in Objektspeicher ab. Den hätte das Lab mit SeaweedFS aus Kapitel 13 sogar, aber Sidecar, Query, Store und Compactor kosten mehr Speicher, als die drei Worker übrig haben. Der Aufbau steht in „Kubernetes beobachten“.

17.13 Aufräumen

Der Stack kann im Lab bleiben, die folgenden Kapitel stören sich nicht daran. Wenn du den Speicher brauchst:

MAC — Schritt 18

```
helm -n monitoring uninstall alloy loki kps
kubectl delete namespace monitoring
kubectl get crd -o name | grep monitoring.coreos.com | xargs kubectl delete
```

Der Namespace nimmt die PVCs mit, LINSTOR löscht die Ressourcen dank `reclaimPolicy: Delete`. Die CRDs bleiben bei `helm uninstall` stehen und müssen von Hand weg.

WAS DU GELERNT HAST

- Der Controller liefert LINSTOR-Metriken über die VIP, drbd-reactor DRBD-Metriken auf jedem Node, auch auf der Witness.
- Dienste auf den Hosts kommen als statische Ziele zu Prometheus, mit `instance` als lesbarem Node-Namen.
- Alloy liest Journal und Kernelmeldungen, Loki macht sie mit LogQL durchsuchbar. Die Ursache eines DRBD-Alarms steht im Kernel-Log.
- `for`, `group_by` und `group_interval` entscheiden, ob ein Alarm hilft oder nervt.
- Quorum-Verlust und Suspend sind nur auf dem Primary kritisch. Erst der Versuch im Lab hat das gezeigt.
- Community-Dashboards setzen ein Label-Schema voraus. Passe die Scrape-Konfiguration an und prüfe jedes Panel.

KAPITEL 18

Vom Lab zur Produktion

Du weißt, was in Produktion anders ist als im Lab, welche Entscheidungen dort anstehen und welche Fragen du einem bestehenden Setup stellen solltest.

Dauer: 20 Minuten Lesen

NICHT IM LAB GETESTET

Dieses Kapitel beschreibt Architektur und Abwägungen. Was hier steht, wurde nicht auf den Lab-VMs ausgeführt, außer wo ausdrücklich ein Kapitel genannt ist.

18.1 Lab und Produktion im Vergleich

Thema	Lab	typische Produktion
Hosts	4 VMs	8 Hosts in zwei Zonen, Witness an einem dritten Standort
k0s-Controller	1	Zu prüfen. Mit einem Controller ist die Control Plane ein Single Point of Failure.
LINSTOR-Controller	1 aktiv, 3 Kandidaten	1 aktiv, Kandidaten laut Gruppe `linstor_controllers`
`linstor_db`	3 Kopien + Witness	Kopien auf allen Hosts + Witness
Datenplatten	eine VM-Platte, in zwei Partitionen geteilt	HDDs für `pool_data`, NVMe für `pool_nvme`
Storage-Netz	Proxmox-Bridge ohne Port	eigenes VLAN auf eigenen Bonds
Installation	von Hand, dann Ansible	Ansible

18.2 Mehrere k0s-Controller

Mit einem k0s-Controller laufen die Pods bei dessen Ausfall weiter, aber niemand kann den Cluster ändern, und Pods auf ausgefallenen Workern werden nicht ersetzt. Für Produktion nimmt man drei Controller:

- In `k0sctl.yaml` drei Hosts mit `role: controller`. k0s bildet daraus einen etcd-Cluster mit drei Mitgliedern.

- Ein Load-Balancer oder eine virtuelle IP vor der API auf Port 6443, 9443 (Join) und 8132 (konnectivity). k0s bringt dafür einen eigenen Mechanismus mit, **Control Plane Load Balancing** (CPLB) mit Keepalived. Alternativ ein externer Load-Balancer, dessen Adresse in `spec.api.externalAddress` steht.
- `spec.api.sans` um die VIP ergänzen.

Prüfe in einer bestehenden Umgebung: Wie viele Controller laufen? Was steht in `k0s etcd member-list`?

18.3 Das Storage-Netz

- Replikation braucht Bandbreite und vor allem niedrige Latenz. Jeder Schreibvorgang wartet auf die Gegenseite (Kapitel 1).
- Ein Bond aus zwei Ports schützt vor einem Kabel- oder Switch-Ausfall. Die Versuche aus Kapitel 11 zeigen, was ohne Redundanz passiert: Schreibpausen von etwa 10 Sekunden bei jeder Unterbrechung.
- Satellite-Verkehr und Replikation laufen im Lab über dasselbe Netz. In Produktion ist zu klären, ob das auch so ist.
- Die Witness sitzt außerhalb des Storage-Netzes. Das ist gewollt, siehe Kapitel 11, Versuch B: Fällt nur das Storage-Netz aus, entscheidet die Witness über das Management-Netz.

18.4 Writecache: Nutzen und Preis

Der Writecache auf NVMe macht Schreibvorgänge auf HDDs schnell. Dafür nimmst du drei Dinge in Kauf, die dieses Buch im Lab gefunden hat:

1. **Vergrößern scheitert**, wenn der Zuwachs größer ist als der freie Platz in `pool_nvme` (Kapitel 12).
2. **Vergrößern gelingt scheinbar und ist wirkungslos**: Das Writecache-Gerät wächst nicht mit. Der Fehler besteht in LINSTOR 1.33.3 und 1.35.3 (Kapitel 12 und 15). Kontrolle mit `df` im Pod ist Pflicht.
3. **Snapshots** mit klassischem LVM ab LINSTOR 1.35 reservieren die volle Größe (Kapitel 15).

Kläre in einer bestehenden Umgebung, ob Volumes je vergrößert wurden, und prüfe sie mit `dmsetup table` gegen `blockdev --getsz` des Daten-LVs.

18.5 Der Piraeus-Operator am externen Controller

Der Operator verändert globale Eigenschaften des Controllers (Kapitel 8). Am wichtigsten: `DrbdOptions/AutoEvictAllowEviction=false`. Ausgefallene Nodes werden also nie automatisch ge-

räumt. Zusammen mit dem abgeschalteten HA-Controller heißt das: Bei einem dauerhaften Ausfall braucht es einen Menschen, der das Out-of-Service-Taint setzt (Kapitel 10). Das gehört ins Betriebshandbuch, mit Telefonnummer.

18.6 Fragen an ein bestehendes Setup

Wer ein laufendes System übernimmt, kann mit diesen Befehlen in einer Stunde viel lernen:

Frage	Befehl
Welche Versionen laufen?	`linstor controller version`, `cat /proc/drbd`, `k0s version`
Wo läuft der Controller?	`drbd-reactorctl status linstor_db`
Sind alle Nodes online?	`linstor node list`
Gibt es kaputte Ressourcen?	`linstor resource list --faulty`
Gibt es Split-Brains?	`drbdadm status | grep -B3 StandAlone` auf jedem Node
Wie voll sind die Pools?	`linstor storage-pool list`
Was setzt der Operator?	`linstor controller list-properties`
Welche StorageClasses gibt es, was steht drin?	`kubectl get sc -o yaml`, danach `linstor resource-group list`
Gibt es Fehlerberichte?	`linstor error-reports list`
Haben alle CSI-Nodes die Topologie-Eigenschaft?	`linstor node list-properties <node> | grep topology`

WAS DU GELERNT HAST

- Drei k0s-Controller mit VIP oder Load-Balancer beseitigen den letzten Single Point of Failure.
- Das Storage-Netz braucht Redundanz. Unterbrechungen kosten Schreibpausen.
- Writecache-Volumes brauchen beim Vergrößern Handarbeit und Kontrolle.
- Ohne HA-Controller und Auto-Eviction braucht ein dauerhafter Ausfall einen Menschen.

KAPITEL 19

Anhang

19.1 Befehle für den Alltag

Befehl	Wofür
`drbdadm status`	Zustand aller DRBD-Ressourcen auf diesem Node
`drbdsetup status --verbose --statistics <res>`	Quorum, Out-of-Sync, Backing-Device
`linstor node list`	Sind alle Satellites online?
`linstor resource list --faulty`	Nur Ressourcen mit Problemen
`linstor storage-pool list`	Freier Platz je Pool und Node
`linstor error-reports list`	Fehlerberichte des Controllers
`drbd-reactorctl status linstor_db`	Wo läuft der Controller?
`drbd-reactorctl evict linstor_db`	Controller geplant abgeben
`linstor controller list-properties`	Globale Einstellungen, auch die des Operators
`k0s status`	Rolle und Version des lokalen k0s
`k0s sysinfo`	Voraussetzungen des Hosts prüfen
`journalctl -u k0sworker` / `-u linstor-satellite`	Logs des Nodes

19.2 Fehlerbilder

Du siehst	Wahrscheinliche Ursache	Kapitel
CSI-Pods `Pending`, `didn't match Pod's node affinity/selector`	`nodeSelector` des Linstor-Cluster gilt auch für CSI	8
CSI-Node `Init:0/1`, `/var/lib/kubelet/plugins_registry is not a directory`	kubelet-Pfad von k0s	8
CSI-Node-Pods werden alle paar Sekunden neu erzeugt	`Aux/topology/linbit.com/hostname` fehlt am LINSTOR-Node	8
Volume hat 1 Replikat statt 2, kein Writecache	StorageClass ohne vollständige Parameter überschreibt die Resource-Group	8

Du siehst	Wahrscheinliche Ursache	Kapitel
CSI `CrashLoopBackOff`, `REST API 1.27.0 is not supported`	CSI 1.13 gegen LINSTOR 1.33	15
`Cannot grow the volume definition ... StorPool: pool_nvme`	Zuwachs größer als freier Cache-Pool	12
PVC größer, `df` im Pod unverändert	Writecache-Gerät nicht mitgewachsen	12
`Storage driver 'LVM' does not support snapshots`	LINSTOR 1.33 mit klassischem LVM	13
Node `OFFLINE` direkt nach Paket-Update	Satellite und Controller haben verschiedene Versionen	15
`Connection refused` von der VIP	Controller startet gerade, Failover läuft	6, 10
`connection:StandAlone`	Split-Brain	11
`suspended:quorum`, Anwendung hängt	Primary ohne Mehrheit	11
I/O-Fehler, `force-io-failures:yes`	Veralteter Primary wurde abgesetzt	11
Pod hängt in `Terminating` auf `NotReady`-Node	Node ist aus, Out-of-Service-Taint fehlt	10
Keine Pods auf einem reparierten Node	Out-of-Service-Taint noch gesetzt	10
`driver name linstor.csi.linbit.com not found`	CSI-Node nach Neustart noch nicht registriert	10
Zwei LINSTOR-Controller antworten unterschiedlich	`linstor-controller` nach Paketinstallation auf mehreren Nodes aktiv	5

19.3 Ports

Port	Dienst	Netz
3366	LINSTOR-Satellite	Storage (Worker), Management (Witness)

Port	Dienst	Netz
3370	LINSTOR-Controller REST und Metriken	Storage, über die VIP
7000 und folgende	DRBD, je Ressource ein Port	Storage, zur Witness Management
9942	drbd-reactor Metriken	jeder Node
6443, 9443, 8132	k0s API, Join, konnectivity	Management
8333	SeaweedFS S3 auf `k0s-ctrl`	Management

19.4 Quellen

- k0s-Dokumentation: `docs.k0sproject.io`
- LINSTOR User Guide: `linbit.com/drbd-user-guide/linstor-guide-1_0-en/`, Abschnitte „Creating a highly available LINSTOR cluster“, „Auto-evict“, „Backup and restore database“
- DRBD 9 User Guide: `linbit.com/drbd-user-guide/drbd-guide-9_0-en/`, Abschnitte zu Quorum und drbd-reactor
- Piraeus-Operator: `github.com/piraeusdatastore/piraeus-operator`, `docs/how-to/external-controller.md` und `docs/how-to/k0s.md`
- Velero: `velero.io/docs`, File System Backup

K0S UND LINSTOR

Vier Ubuntu-VMs, ein Laptop und eine Frage: Was passiert mit meinen Daten, wenn ein Node ausfällt, ein Netz reißt oder eine Platte stirbt?

Dieses Buch baut Kubernetes mit k0s und replizierten Block-Speicher mit LINSTOR und DRBD Befehl für Befehl auf. LINSTOR läuft außerhalb des Clusters auf den Hosts, sein Controller ist hochverfügbar, Kubernetes nutzt ihn über CSI. Danach wird alles absichtlich kaputtgemacht und repariert, mit gemessenen Ausfallzeiten aus dem Lab.

AUS DEM INHALT

Aufbau	k0s mit k0sctl, LINSTOR und DRBD als Host-Pakete
Hochverfügbarkeit	Controller-Failover mit drbd-reactor und VIP
Speicher	Zonen, Witness, Writecache, Piraeus als reiner CSI-Treiber
Ausfälle	Node weg, Netz weg, Quorum, Split-Brain, Plattentausch
Betrieb	Kapazität, Backup mit Velero, Updates in der richtigen Reihenfolge
Automatisierung	Ansible-Rollen und getestete Alarmregeln

Thomas Zachmann
Autor auf Leanpub



Hands-on-Handbuch · Stand: 12. November 2026